

Source Code Appendix

Phiz — Physics Simulator Android App

107 source files

build.gradle.kts

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.gms.google.services)
}

android {
    namespace = "com.example.phiz"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.phiz"
        minSdk = 26
        targetSdk = 34
        versionCode = 2
        versionName = "1.1"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    implementation(libs.recyclerview)
    implementation(libs.firebase.ai)
    implementation(libs.firebase.firestore)
    implementation(libs.firebase.auth)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
    implementation(platform("com.google.firebase:firebase-bom:34.6.0"))
    implementation("com.google.firebase:firebase-analytics")

    // Firebase Cloud Messaging for push notifications
    implementation("com.google.firebase:firebase-messaging")

    // WorkManager for scheduled local notifications
    implementation("androidx.work:work-runtime:2.9.0")
}
```

proguard-rules.pro

app/proguard-rules.pro

```
# Add project specific ProGuard rules here.
# You can control the set of applied configuration files using the
# proguardFiles setting in build.gradle.
#
# For more details, see
# http://developer.android.com/guide/developing/tools/proguard.html

# If your project uses WebView with JS, uncomment the following
# and specify the fully qualified class name to the JavaScript interface
# class:
#-keepclassmembers class fqcn.of.javascript.interface.for.webview {
#    public *;
#}

# Uncomment this to preserve the line number information for
# debugging stack traces.
#-keepattributes SourceFile,LineNumberTable

# If you keep the line number information, uncomment this to
# hide the original source file name.
#-renamesourcefileattribute SourceFile
```

ExampleInstrumentedTest.java

app/src/androidTest/java/com/example/phiz/ExampleInstrumentedTest.java

```
package com.example.phiz;

import android.content.Context;

import androidx.test.platform.app.InstrumentationRegistry;
import androidx.test.ext.junit.runners.AndroidJUnit4;

import org.junit.Test;
import org.junit.runner.RunWith;

import static org.junit.Assert.*;

/**
 * Instrumented test, which will execute on an Android device.
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
@RunWith(AndroidJUnit4.class)
public class ExampleInstrumentedTest {
    @Test
    public void useAppContext() {
        // Context of the app under test.
        Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();
        assertEquals("com.example.phiz", appContext.getPackageName());
    }
}
```

AndroidManifest.xml

app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

    <!-- Notification permissions -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <uses-permission android:name="android.permission.SCHEDULE_EXACT_ALARM" />

    <application
        android:name=".PhizApplication"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Phiz">
        <activity
            android:name=".activities.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".activities.LoginActivity"
            android:exported="false" />
        <activity
            android:name=".activities.RegisterActivity"
            android:exported="false" />
        <activity
            android:name=".activities.StudentHomeActivity"
            android:exported="false" />
        <activity
            android:name=".activities.TeacherHomeActivity"
            android:exported="false" />
        <activity
            android:name=".activities.PhysicsSimulationActivity"
            android:exported="false" />
        <activity
            android:name=".activities.QuizActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.VIEW" />
                <category android:name="android.intent.category.DEFAULT" />
                <data
                    android:host="quiz"
                    android:scheme="phiz" />
            </intent-filter>
        </activity>
        <activity
            android:name=".activities.GradesActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.VIEW" />
                <category android:name="android.intent.category.DEFAULT" />
                <data
                    android:host="grades"
                    android:scheme="phiz" />
            </intent-filter>
        </activity>
        <activity
            android:name=".activities.CreateQuestionActivity"
            android:exported="false" />
        <activity
            android:name=".activities.ViewQuestionsActivity"
            android:exported="false" />
        <activity
            android:name=".activities.AllGradesActivity"
            android:exported="false" />
        <activity
            android:name=".activities.NotificationSettingsActivity"
```

```

        android:exported="false" />

<!-- Firebase Cloud Messaging Service -->
<service
    android:name=".services.PhizFirebaseMessagingService"
    android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>

<!-- Boot Receiver for rescheduling notifications -->
<receiver
    android:name=".receivers.BootReceiver"
    android:exported="false">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>

<!-- Notification Action Receiver -->
<receiver
    android:name=".receivers.NotificationActionReceiver"
    android:exported="false">
    <intent-filter>
        <action android:name="com.example.phiz.NOTIFICATION_ACTION" />
    </intent-filter>
</receiver>
</application>
</manifest>

```

AllGradesActivity.java

app/src/main/java/com/example/phiz/activities/AllGradesActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.LinearLayout;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.phiz.R;
import com.example.phiz.helpers.FirestoreHelper;
import com.example.phiz.models.QuizResult;
import com.google.android.material.card.MaterialCardView;
import com.google.firebase.Timestamp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class AllGradesActivity extends AppCompatActivity {
    private RecyclerView allGradesRecyclerView;
    private LinearLayout emptyStateLayout;
    private MaterialCardView backButton;
    private ProgressBar progressBar;
    private AllGradesAdapter adapter;

    private FirebaseAuth mAuth;
    private FirestoreHelper firestoreHelper;
    private List<UserGrade> userGradesList = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_all_grades);

        mAuth = FirebaseAuth.getInstance();
        firestoreHelper = FirestoreHelper.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            startActivity(new Intent(this, LoginActivity.class));
            finish();
            return;
        }

        allGradesRecyclerView = findViewById(R.id.allGradesRecyclerView);
        emptyStateLayout = findViewById(R.id.emptyStateLayout);
        backButton = findViewById(R.id.backButton);
        progressBar = findViewById(R.id.progressBar);

        allGradesRecyclerView.setLayoutManager(new LinearLayoutManager(this));
        adapter = new AllGradesAdapter(userGradesList);
        allGradesRecyclerView.setAdapter(adapter);

        backButton.setOnClickListener(v -> finish());

        loadAllGrades();
    }

    private void loadAllGrades() {
        progressBar.setVisibility(View.VISIBLE);

        // Use FirestoreHelper to get all grades from all users' subcollections
        firestoreHelper.getAllGrades(
            grades -> {
                userGradesList.clear();
                for (FirestoreHelper.UserGrade userGrade : grades) {
                    userGradesList.add(new UserGrade(userGrade.userName, userGrade.grade));
                }
            }
        );
    }
}
```

```

    }

    adapter.notifyDataSetChanged();
    progressBar.setVisibility(View.GONE);

    if (userGradesList.isEmpty()) {
        allGradesRecyclerView.setVisibility(View.GONE);
        emptyStateLayout.setVisibility(View.VISIBLE);
    } else {
        allGradesRecyclerView.setVisibility(View.VISIBLE);
        emptyStateLayout.setVisibility(View.GONE);
    }
},
e -> {
    progressBar.setVisibility(View.GONE);
    Toast.makeText(this, "Error loading grades: " + e.getMessage(),
        Toast.LENGTH_SHORT).show();
});
});

// Data class to hold user name with quiz result
private static class UserGrade {
    String userName;
    QuizResult quizResult;

    UserGrade(String userName, QuizResult quizResult) {
        this.userName = userName;
        this.quizResult = quizResult;
    }
}

private static class AllGradesAdapter extends RecyclerView.Adapter<AllGradesAdapter.GradeViewHolder> {
    private List<UserGrade> userGrades;
    private SimpleDateFormat dateFormat = new SimpleDateFormat("MMM dd, yyyy HH:mm", Locale.US);

    public AllGradesAdapter(List<UserGrade> userGrades) {
        this.userGrades = userGrades;
    }

    @NonNull
    @Override
    public GradeViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_all_grades, parent, false);
        return new GradeViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull GradeViewHolder holder, int position) {
        UserGrade userGrade = userGrades.get(position);
        QuizResult result = userGrade.quizResult;

        holder.userNameTextView.setText(userGrade.userName);
        holder.quizNameTextView.setText(result.getQuizName());

        Timestamp timestamp = result.getTimestamp();
        if (timestamp != null) {
            Date date = timestamp.toDate();
            holder.dateTextView.setText(dateFormat.format(date));
        } else {
            holder.dateTextView.setText("");
        }

        int maxScore = result.getTotalQuestions() * 20;
        holder.scoreTextView.setText(String.format(Locale.US, "Score: %d/%d",
            result.getScore(), maxScore));

        int percentage = maxScore > 0 ? (int) (((float) result.getScore() / maxScore) * 100) : 0;
        if (percentage > 100) percentage = 100; // Cap at 100%
        holder.percentageTextView.setText(percentage + "%");
    }

    @Override
    public int getItemCount() {
        return userGrades.size();
    }
}

static class GradeViewHolder extends RecyclerView.ViewHolder {
    TextView userNameTextView, quizNameTextView, dateTextView, scoreTextView, percentageTextView;

    public GradeViewHolder(@NonNull View itemView) {
        super(itemView);
        userNameTextView = itemView.findViewById(R.id.userNameTextView);
        quizNameTextView = itemView.findViewById(R.id.quizNameTextView);
    }
}

```



```
        dateTextView = itemView.findViewById(R.id.dateTextView);
        scoreTextView = itemView.findViewById(R.id.scoreTextView);
        percentageTextView = itemView.findViewById(R.id.percentageTextView);
    }
}
}
```

CreateQuestionActivity.java

app/src/main/java/com/example/phiz/activities/CreateQuestionActivity.java

```
package com.example.phiz.activities;

import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.ProgressBar;
import android.widget.RadioGroup;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.phiz.R;
import com.example.phiz.helpers.NotificationHelper;
import com.example.phiz.models.Question;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.messaging.FirebaseMessaging;

import java.util.Arrays;
import java.util.List;
import java.util.UUID;

public class CreateQuestionActivity extends AppCompatActivity {
    private static final String TAG = "CreateQuestionActivity";

    private TextInputEditText questionTextInput;
    private TextInputEditText option1Input, option2Input, option3Input, option4Input;
    private RadioGroup correctAnswerRadioGroup;
    private Spinner difficultySpinner;
    private TextInputEditText explanationInput;
    private TextInputEditText pointValueInput;
    private Button saveQuestionButton;
    private ProgressBar progressBar;
    private MaterialToolbar toolbar;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_create_question);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            finish();
            return;
        }

        initializeViews();
        setupDifficultySpinner();
        setupToolbar();
        setupSaveButton();
    }

    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        questionTextInput = findViewById(R.id.questionTextInput);
        option1Input = findViewById(R.id.option1Input);
        option2Input = findViewById(R.id.option2Input);
        option3Input = findViewById(R.id.option3Input);
        option4Input = findViewById(R.id.option4Input);
        correctAnswerRadioGroup = findViewById(R.id.correctAnswerRadioGroup);
        difficultySpinner = findViewById(R.id.difficultySpinner);
        explanationInput = findViewById(R.id.explanationInput);
        pointValueInput = findViewById(R.id.pointValueInput);
        saveQuestionButton = findViewById(R.id.saveQuestionButton);
        progressBar = findViewById(R.id.progressBar);
    }

    private void setupDifficultySpinner() {
        String[] difficulties = {"easy", "medium", "hard"};
    }
}
```

```

        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this,
            android.R.layout.simple_spinner_item,
            difficulties
        );
        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        difficultySpinner.setAdapter(adapter);
        difficultySpinner.setSelection(1); // Default to "medium"
    }

    private void setupToolbar() {
        toolbar.setNavigationOnClickListener(v -> finish());
    }

    private void setupSaveButton() {
        saveQuestionButton.setOnClickListener(v -> saveQuestion());
    }

    private void saveQuestion() {
        String questionText = getTextFromInput(questionTextInput);
        String option1 = getTextFromInput(option1Input);
        String option2 = getTextFromInput(option2Input);
        String option3 = getTextFromInput(option3Input);
        String option4 = getTextFromInput(option4Input);
        String explanation = getTextFromInput(explanationInput);
        String pointValueStr = getTextFromInput(pointValueInput);
        String difficulty = difficultySpinner.getSelectedItem().toString();

        // Validate inputs
        if (questionText.isEmpty()) {
            questionTextInput.setError("Question text is required");
            return;
        }
        if (option1.isEmpty() || option2.isEmpty() || option3.isEmpty() || option4.isEmpty()) {
            Toast.makeText(this, "All four options are required", Toast.LENGTH_SHORT).show();
            return;
        }

        int correctAnswerIndex = getCorrectAnswerIndex();
        if (correctAnswerIndex == -1) {
            Toast.makeText(this, "Please select the correct answer", Toast.LENGTH_SHORT).show();
            return;
        }

        int pointValue = 20; // Default
        if (!pointValueStr.isEmpty()) {
            try {
                pointValue = Integer.parseInt(pointValueStr);
            } catch (NumberFormatException e) {
                pointValueInput.setError("Invalid point value");
                return;
            }
        }

        // Create question object
        String questionId = UUID.randomUUID().toString();
        List<String> options = Arrays.asList(option1, option2, option3, option4);
        Question question = new Question(
            questionId,
            questionText,
            options,
            correctAnswerIndex,
            explanation,
            pointValue,
            difficulty
        );

        // Show progress
        setLoading(true);

        // Save to Firestore
        db.collection("questions")
            .document(questionId)
            .set(question)
            .addOnSuccessListener(aVoid -> {
                setLoading(false);
                Toast.makeText(this, "Question saved successfully!", Toast.LENGTH_SHORT).show();

                // Notify students of new content via FCM topic
                notifyStudentsOfNewContent();

                clearForm();
            })
            .addOnFailureListener(e -> {

```

```

        setLoading(false);
        Toast.makeText(this, "Error saving question: " + e.getMessage(),
            Toast.LENGTH_LONG).show();
    });
}

private void notifyStudentsOfNewContent() {
    // Send notification to all students topic
    // Note: For server-triggered notifications, you would typically use Cloud Functions
    // Here we show a local notification as a fallback/demo
    NotificationHelper.getInstance(this).showNewContentNotification("questions");
}

private String getTextFromInput(TextInputEditText input) {
    if (input.getText() != null) {
        return input.getText().toString().trim();
    }
    return "";
}

private int getCorrectAnswerIndex() {
    int checkedId = correctAnswerRadioGroup.getCheckedRadioButtonId();
    if (checkedId == R.id.radioOption1) return 0;
    if (checkedId == R.id.radioOption2) return 1;
    if (checkedId == R.id.radioOption3) return 2;
    if (checkedId == R.id.radioOption4) return 3;
    return -1;
}

private void setLoading(boolean isLoading) {
    progressBar.setVisibility(isLoading ? View.VISIBLE : View.GONE);
    saveQuestionButton.setEnabled(!isLoading);
}

private void clearForm() {
    questionTextInput.setText("");
    option1Input.setText("");
    option2Input.setText("");
    option3Input.setText("");
    option4Input.setText("");
    correctAnswerRadioGroup.clearCheck();
    explanationInput.setText("");
    pointValueInput.setText("20");
    difficultySpinner.setSelection(1);
}
}

```

GradesActivity.java

app/src/main/java/com/example/phiz/activities/GradesActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.phiz.R;
import com.example.phiz.helpers.FirestoreHelper;
import com.example.phiz.models.QuizResult;
import com.google.android.material.card.MaterialCardView;
import com.google.firebase.Timestamp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;

public class GradesActivity extends AppCompatActivity {
    private RecyclerView gradesRecyclerView;
    private LinearLayout emptyStateLayout;
    private MaterialCardView backButton;
    private Button takeQuizButton;
    private TextView titleTextView;
    private TextView subtitleTextView;
    private TextView emptyTextView;
    private GradeAdapter adapter;

    private FirebaseAuth mAuth;
    private FirestoreHelper firestoreHelper;
    private List<QuizResult> quizResults = new ArrayList<>();
    private boolean isViewingOtherUser = false;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_grades);

        mAuth = FirebaseAuth.getInstance();
        firestoreHelper = FirestoreHelper.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            startActivity(new Intent(this, LoginActivity.class));
            finish();
            return;
        }

        gradesRecyclerView = findViewById(R.id.gradesRecyclerView);
        emptyStateLayout = findViewById(R.id.emptyStateLayout);
        backButton = findViewById(R.id.backButton);
        takeQuizButton = findViewById(R.id.takeQuizButton);
        titleTextView = findViewById(R.id.titleTextView);
        subtitleTextView = findViewById(R.id.subtitleTextView);
        emptyTextView = findViewById(R.id.emptyTextView);

        gradesRecyclerView.setLayoutManager(new LinearLayoutManager(this));
        adapter = new GradeAdapter(quizResults);
        gradesRecyclerView.setAdapter(adapter);

        // Check if viewing another user's grades (teacher viewing student)
        String userId = getIntent().getStringExtra("userId");
        String studentName = getIntent().getStringExtra("studentName");

        if (userId != null && !userId.isEmpty()) {
            // Teacher viewing a student's grades
            isViewingOtherUser = true;
        }
    }
}
```

```

        loadGrades(userId);
        if (titleTextView != null) {
            titleTextView.setText(studentName != null ? studentName + "'s Grades" : "Student Grades");
        }
        if (subtitleTextView != null) {
            subtitleTextView.setText("View student's quiz results");
        }
        if (emptyTextView != null) {
            emptyTextView.setText("This student has no quiz results yet");
        }
        // Hide "Take Quiz" button when viewing other user's grades
        takeQuizButton.setVisibility(View.GONE);
    } else {
        // User viewing their own grades
        loadGrades(currentUser.getId());
    }

    backButton.setOnClickListener(v -> finish());

    takeQuizButton.setOnClickListener(v -> {
        startActivity(new Intent(GradesActivity.this, QuizActivity.class));
        finish();
    });
}

private void loadGrades(String orderId) {
    // Use FirestoreHelper to get grades from user's subcollection
    firestoreHelper.getUserGrades(orderId,
        grades -> {
            quizResults.clear();
            quizResults.addAll(grades);
            adapter.notifyDataSetChanged();

            if (quizResults.isEmpty()) {
                gradesRecyclerView.setVisibility(View.GONE);
                emptyStateLayout.setVisibility(View.VISIBLE);
            } else {
                gradesRecyclerView.setVisibility(View.VISIBLE);
                emptyStateLayout.setVisibility(View.GONE);
            }
        },
        e -> {
            Toast.makeText(this, "Error loading grades: " + e.getMessage(),
                Toast.LENGTH_SHORT).show();
        });
}

private static class GradeAdapter extends RecyclerView.Adapter<GradeAdapter.GradeViewHolder> {
    private List<QuizResult> results;
    private SimpleDateFormat dateFormat = new SimpleDateFormat("MMM dd, yyyy HH:mm", Locale.US);

    public GradeAdapter(List<QuizResult> results) {
        this.results = results;
    }

    @NonNull
    @Override
    public GradeViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_grade, parent, false);
        return new GradeViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull GradeViewHolder holder, int position) {
        QuizResult result = results.get(position);
        holder.quizNameTextView.setText(result.getQuizName());

        Timestamp timestamp = result.getTimestamp();
        if (timestamp != null) {
            Date date = timestamp.toDate();
            holder.dateTextView.setText(dateFormat.format(date));
        }

        int maxScore = result.getTotalQuestions() * 20;
        holder.scoreTextView.setText(String.format(Locale.US, "Score: %d/%d",
            result.getScore(), maxScore));

        int percentage = maxScore > 0 ? (int) (((float) result.getScore() / maxScore) * 100) : 0;
        if (percentage > 100) percentage = 100; // Cap at 100%
        holder.percentageTextView.setText(percentage + "%");
    }

    @Override

```

```

public int getItemCount() {
    return results.size();
}

static class GradeViewHolder extends RecyclerView.ViewHolder {
    TextView quizNameTextView, dateTextView, scoreTextView, percentageTextView;

    public GradeViewHolder(@NonNull View itemView) {
        super(itemView);
        quizNameTextView = itemView.findViewById(R.id.quizNameTextView);
        dateTextView = itemView.findViewById(R.id.dateTextView);
        scoreTextView = itemView.findViewById(R.id.scoreTextView);
        percentageTextView = itemView.findViewById(R.id.percentageTextView);
    }
}
}

```

LoginActivity.java

app/src/main/java/com/example/phiz/activities/LoginActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.phiz.R;
import com.example.phiz.helpers.FCMTokenManager;
import com.example.phiz.helpers.FirestoreHelper;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

public class LoginActivity extends AppCompatActivity {
    private TextInputEditText emailEditText, passwordEditText;
    private Button loginButton;
    private TextView registerTextView;
    private ProgressBar progressBar;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        emailEditText = findViewById(R.id.emailEditText);
        passwordEditText = findViewById(R.id.passwordEditText);
        loginButton = findViewById(R.id.loginButton);
        registerTextView = findViewById(R.id.registerTextView);
        progressBar = findViewById(R.id.progressBar);

        loginButton.setOnClickListener(v -> loginUser());

        registerTextView.setOnClickListener(v -> {
            startActivity(new Intent(LoginActivity.this, RegisterActivity.class));
        });
    }

    @Override
    protected void onStart() {
        super.onStart();
        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser != null) {
            checkUserRoleAndNavigate(currentUser.getUid());
        }
    }

    private void loginUser() {
        String email = emailEditText.getText().toString().trim();
        String password = passwordEditText.getText().toString().trim();

        if (email.isEmpty() || password.isEmpty()) {
            Toast.makeText(this, "Please fill all fields", Toast.LENGTH_SHORT).show();
            return;
        }

        progressBar.setVisibility(View.VISIBLE);
        loginButton.setEnabled(false);

        mAuth.signInWithEmailAndPassword(email, password)
            .addOnCompleteListener(task -> {
                progressBar.setVisibility(View.GONE);
                loginButton.setEnabled(true);

                if (task.isSuccessful()) {
                    FirebaseUser user = mAuth.getCurrentUser();
                    if (user != null) {
                        checkUserRoleAndNavigate(user.getUid());
                    }
                }
            });
    }
}
```



```

        }
    } else {
        Toast.makeText(LoginActivity.this,
            "Login failed: " + task.getException().getMessage(),
            Toast.LENGTH_LONG).show();
    }
});

}

private void checkUserRoleAndNavigate(String uid) {
    db.collection("users").document(uid).get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                String role = documentSnapshot.getString("role");

                // Initialize FCM token after successful login
                FCMTokenManager tokenManager = FCMTokenManager.getInstance(this);
                tokenManager.initializeToken();
                tokenManager.subscribeToRoleTopics(role);

                // Update last activity
                FirestoreHelper.getInstance().updateLastActivity(uid, null);

                Intent intent;
                if ("teacher".equals(role)) {
                    intent = new Intent(LoginActivity.this, TeacherHomeActivity.class);
                } else {
                    intent = new Intent(LoginActivity.this, StudentHomeActivity.class);
                }
                startActivity(intent);
                finish();
            } else {
                Toast.makeText(this, "User data not found", Toast.LENGTH_SHORT).show();
            }
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
        });
}
}
}

```

MainActivity.java

app/src/main/java/com/example/phiz/activities/MainActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Redirect to LoginActivity
        Intent intent = new Intent(MainActivity.this, LoginActivity.class);
        startActivity(intent);
        finish();
    }
}
```

NotificationSettingsActivity.java

app/src/main/java/com/example/phiz/activities/NotificationSettingsActivity.java

```
package com.example.phiz.activities;

import android.app.TimePickerDialog;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.example.phiz.R;
import com.example.phiz.helpers.FirestoreHelper;
import com.example.phiz.helpers.WorkerScheduler;
import com.example.phiz.models.NotificationPreferences;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.switchmaterial.SwitchMaterial;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;

import java.util.Locale;

/**
 * Activity for managing notification preferences.
 */
public class NotificationSettingsActivity extends AppCompatActivity {
    private static final String TAG = "NotificationSettings";
    public static final String EXTRA_USER_ROLE = "user_role";

    private MaterialToolbar toolbar;
    private SwitchMaterial switchQuizNotifications;
    private SwitchMaterial switchGradeNotifications;
    private SwitchMaterial switchAchievementNotifications;
    private SwitchMaterial switchStudyReminders;
    private SwitchMaterial switchWeeklyProgress;
    private MaterialCardView reminderTimeCard;
    private TextView reminderTimeText;
    private Button saveButton;
    private ProgressBar progressBar;

    // Reminder section views (hidden for teachers)
    private View studyRemindersRow;
    private View weeklyProgressRow;
    private View dividerStudyReminders;
    private View dividerWeeklyProgress;

    private FirebaseAuth mAuth;
    private NotificationPreferences currentPreferences;
    private int selectedHour = 18;
    private int selectedMinute = 0;
    private boolean isTeacher = false;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_notification_settings);

        mAuth = FirebaseAuth.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            finish();
            return;
        }

        // Check if user is a teacher
        isTeacher = "teacher".equals(getIntent().getStringExtra(EXTRA_USER_ROLE));

        initializeViews();
        setupToolbar();

        // Hide reminder-related settings for teachers
        if (isTeacher) {
            hideStudentOnlySettings();
        }

        loadPreferences();
    }
}
```

```

reminderTimeCard.setOnClickListener(v -> showTimePicker());
saveButton.setOnClickListener(v -> savePreferences());

// Update reminder time card visibility based on switch state
switchStudyReminders.setOnCheckedChangeListener((buttonView, isChecked) -> {
    if (!isTeacher) {
        reminderTimeCard.setVisibility(isChecked ? View.VISIBLE : View.GONE);
    }
});
}

private void initializeViews() {
    toolbar = findViewById(R.id.toolbar);
    switchQuizNotifications = findViewById(R.id.switchQuizNotifications);
    switchGradeNotifications = findViewById(R.id.switchGradeNotifications);
    switchAchievementNotifications = findViewById(R.id.switchAchievementNotifications);
    switchStudyReminders = findViewById(R.id.switchStudyReminders);
    switchWeeklyProgress = findViewById(R.id.switchWeeklyProgress);
    reminderTimeCard = findViewById(R.id.reminderTimeCard);
    reminderTimeText = findViewById(R.id.reminderTimeText);
    saveButton = findViewById(R.id.saveButton);
    progressBar = findViewById(R.id.progressBar);

    studyRemindersRow = findViewById(R.id.studyRemindersRow);
    weeklyProgressRow = findViewById(R.id.weeklyProgressRow);
    dividerStudyReminders = findViewById(R.id.dividerStudyReminders);
    dividerWeeklyProgress = findViewById(R.id.dividerWeeklyProgress);
}

private void hideStudentOnlySettings() {
    studyRemindersRow.setVisibility(View.GONE);
    weeklyProgressRow.setVisibility(View.GONE);
    dividerStudyReminders.setVisibility(View.GONE);
    dividerWeeklyProgress.setVisibility(View.GONE);
    reminderTimeCard.setVisibility(View.GONE);

    // Force these off for teachers
    switchStudyReminders.setChecked(false);
    switchWeeklyProgress.setChecked(false);
}

private void setupToolbar() {
    toolbar.setNavigationOnClickListener(v -> finish());
}

private void loadPreferences() {
    setLoading(true);

    String userId = mAuth.getCurrentUser().getUid();
    FirestoreHelper.getInstance().getNotificationPreferences(userId,
        prefs -> {
            setLoading(false);
            currentPreferences = prefs;
            updateUI(prefs);
        },
        e -> {
            setLoading(false);
            // Use defaults if loading fails
            currentPreferences = NotificationPreferences.createDefault();
            updateUI(currentPreferences);
            Toast.makeText(this, "Using default settings", Toast.LENGTH_SHORT).show();
        }
    );
}

private void updateUI(NotificationPreferences prefs) {
    switchQuizNotifications.setChecked(prefs.isQuizNotifications());
    switchGradeNotifications.setChecked(prefs.isGradeNotifications());
    switchAchievementNotifications.setChecked(prefs.isAchievementNotifications());
    switchStudyReminders.setChecked(prefs.isStudyReminders());
    switchWeeklyProgress.setChecked(prefs.isWeeklyProgress());

    selectedHour = prefs.getReminderHour();
    selectedMinute = prefs.getReminderMinute();
    updateReminderTimeText();

    // Show/hide reminder time card based on study reminders toggle (never show for teachers)
    if (!isTeacher) {
        reminderTimeCard.setVisibility(prefs.isStudyReminders() ? View.VISIBLE : View.GONE);
    }
}

private void updateReminderTimeText() {
    String amPm = selectedHour >= 12 ? "PM" : "AM";

```

```

    int displayHour = selectedHour > 12 ? selectedHour - 12 : (selectedHour == 0 ? 12 : selectedHour);
    String timeString = String.format(Locale.getDefault(), "%d:%02d %s", displayHour, selectedMinute, amPm);
    reminderTimeText.setText(timeString);
}

private void showTimePicker() {
    TimePickerDialog timePicker = new TimePickerDialog(
        this,
        (view, hourOfDay, minute) -> {
            selectedHour = hourOfDay;
            selectedMinute = minute;
            updateReminderTimeText();
        },
        selectedHour,
        selectedMinute,
        false
    );
    timePicker.setTitle("Select Reminder Time");
    timePicker.show();
}

private void savePreferences() {
    setLoading(true);

    // Build preferences from UI state
    NotificationPreferences prefs = new NotificationPreferences();
    prefs.setQuizNotifications(switchQuizNotifications.isChecked());
    prefs.setGradeNotifications(switchGradeNotifications.isChecked());
    prefs.setAchievementNotifications(switchAchievementNotifications.isChecked());
    prefs.setStudyReminders(switchStudyReminders.isChecked());
    prefs.setWeeklyProgress(switchWeeklyProgress.isChecked());
    prefs.setReminderTime(String.format(Locale.getDefault(), "%02d:%02d", selectedHour, selectedMinute));

    String userId = mAuth.getCurrentUser().getUid();
    FirestoreHelper.getInstance().saveNotificationPreferences(userId, prefs,
        (success, e) -> {
            setLoading(false);
            if (success) {
                // Only schedule reminder workers for students
                if (!isTeacher) {
                    WorkerScheduler.scheduleAllWorkers(this, prefs);
                }

                Toast.makeText(this, "Settings saved", Toast.LENGTH_SHORT).show();
                finish();
            } else {
                Toast.makeText(this, "Error saving settings: " +
                    (e != null ? e.getMessage() : "Unknown error"), Toast.LENGTH_SHORT).show();
            }
        }
    );
}

private void setLoading(boolean loading) {
    progressBar.setVisibility(loading ? View.VISIBLE : View.GONE);
    saveButton.setEnabled(!loading);
}
}

```

PhysicsSimulationActivity.java

app/src/main/java/com/example/phiz/activities/PhysicsSimulationActivity.java

```
package com.example.phiz.activities;

import android.os.Bundle;
import android.widget.Button;
import android.widget.SeekBar;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import com.example.phiz.R;
import com.example.phiz.views.PhysicsSimulationView;
import com.google.android.material.card.MaterialCardView;

import java.util.Locale;

public class PhysicsSimulationActivity extends AppCompatActivity {
    private PhysicsSimulationView simulationView;
    private SeekBar massSeekBar, forceSeekBar, frictionSeekBar, angleSeekBar;
    private TextView massValueTextView, forceValueTextView, frictionValueTextView,
        angleValueTextView, accelerationTextView, velocityTextView, positionTextView,
        lawDescriptionTextView;
    private Button startButton, resetButton;
    private MaterialCardView backButton;

    private float mass = 5.0f;
    private float appliedForce = 30.0f;
    private float frictionCoefficient = 0.2f;
    private float angle = 0.0f;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_physics_simulation);

        simulationView = findViewById(R.id.simulationView);
        massSeekBar = findViewById(R.id.massSeekBar);
        forceSeekBar = findViewById(R.id.forceSeekBar);
        frictionSeekBar = findViewById(R.id.frictionSeekBar);
        angleSeekBar = findViewById(R.id.angleSeekBar);
        massValueTextView = findViewById(R.id.massValueTextView);
        forceValueTextView = findViewById(R.id.forceValueTextView);
        frictionValueTextView = findViewById(R.id.frictionValueTextView);
        angleValueTextView = findViewById(R.id.angleValueTextView);
        accelerationTextView = findViewById(R.id.accelerationTextView);
        velocityTextView = findViewById(R.id.velocityTextView);
        positionTextView = findViewById(R.id.positionTextView);
        lawDescriptionTextView = findViewById(R.id.lawDescriptionTextView);
        startButton = findViewById(R.id.startButton);
        resetButton = findViewById(R.id.resetButton);
        backButton = findViewById(R.id.backButton);

        updateLawDescription();

        simulationView.setOnPhysicsUpdateListener((acceleration, velocity, position) -> {
            runOnUiThread(() -> {
                accelerationTextView.setText(String.format(Locale.US, "%.2f m/s2", acceleration));
                velocityTextView.setText(String.format(Locale.US, "%.2f m/s", velocity));
                positionTextView.setText(String.format(Locale.US, "%.2f m", position));
            });
        });

        massSeekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                mass = (progress + 1) / 10.0f;
                massValueTextView.setText(String.format(Locale.US, "%.1f kg", mass));
                simulationView.setParameters(mass, appliedForce, frictionCoefficient, angle);
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
        });

        forceSeekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
```

```

        public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
            appliedForce = progress;
            forceValueTextView.setText(String.format(Locale.US, "%.1f N", appliedForce));
            simulationView.setParameters(mass, appliedForce, frictionCoefficient, angle);
        }

        @Override
        public void onStartTrackingTouch(SeekBar seekBar) {
        }

        @Override
        public void onStopTrackingTouch(SeekBar seekBar) {
        }
    });

    frictionSeekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
        @Override
        public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
            frictionCoefficient = progress / 100.0f;
            frictionValueTextView.setText(String.format(Locale.US, "%.2f", frictionCoefficient));
            simulationView.setParameters(mass, appliedForce, frictionCoefficient, angle);
        }

        @Override
        public void onStartTrackingTouch(SeekBar seekBar) {
        }

        @Override
        public void onStopTrackingTouch(SeekBar seekBar) {
        }
    });

    angleSeekBar.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
        @Override
        public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
            angle = progress;
            angleValueTextView.setText(String.format(Locale.US, "%d°", (int) angle));
            simulationView.setParameters(mass, appliedForce, frictionCoefficient, angle);
        }

        @Override
        public void onStartTrackingTouch(SeekBar seekBar) {
        }

        @Override
        public void onStopTrackingTouch(SeekBar seekBar) {
        }
    });

    startButton.setOnClickListener(v -> {
        simulationView.startAnimation();
    });

    resetButton.setOnClickListener(v -> {
        simulationView.reset();
    });

    backButton.setOnClickListener(v -> {
        finish();
    });

    simulationView.setParameters(mass, appliedForce, frictionCoefficient, angle);
}

private void updateLawDescription() {
    String description = "Newton's Laws:\n\n" +
        "1st Law: An object remains at rest or in uniform motion unless acted upon by a force.\n\n" +
        "2nd Law:  $F = ma$  (Force equals mass times acceleration)\n\n" +
        "3rd Law: For every action, there is an equal and opposite reaction.\n\n" +
        "Adjust the parameters to see how forces affect motion!";
    lawDescriptionTextView.setText(description);
}

@Override
protected void onDestroy() {
    super.onDestroy();
    simulationView.stopAnimation();
}
}

```

QuizActivity.java

app/src/main/java/com/example/phiz/activities/QuizActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.os.Handler;
import android.os.Looper;
import android.view.View;
import android.widget.Button;
import android.widget.RadioButton;
import android.widget.RadioGroup;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.example.phiz.R;
import com.example.phiz.helpers.FirestoreHelper;
import com.example.phiz.helpers.NotificationHelper;
import com.example.phiz.models.Question;
import com.example.phiz.models.QuizResult;
import com.example.phiz.views.DoodleView;
import com.example.phiz.workers.InactivityCheckWorker;
import com.example.phiz.workers.WeeklyProgressWorker;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class QuizActivity extends AppCompatActivity {
    private TextView questionNumberTextView, questionTextView, feedbackTextView, timerTextView;
    private RadioGroup answersRadioGroup;
    private RadioButton option1, option2, option3, option4;
    private Button submitButton, nextButton;
    private MaterialCardView feedbackCard;

    private List<Question> questions;
    private int currentQuestionIndex = 0;
    private int score = 0;
    private int correctAnswers = 0;
    private boolean answerSubmitted = false;
    private int shuffledCorrectAnswerIndex = 0;

    // Timer
    private CountDownTimer countDownTimer;
    private static final long QUESTION_TIME_MS = 30000; // 30 seconds per question
    private static final long AUTO_PROGRESS_DELAY_MS = 2000; // 2 seconds delay before auto-progress

    // Handler for auto-progression
    private Handler autoProgressHandler = new Handler(Looper.getMainLooper());

    // Doodle
    private DoodleView doodleView;
    private FloatingActionButton doodleToggleButton, clearDoodleButton;

    // Point system constants
    private static final int POINTS_PER_CORRECT = 20;
    private static final int COMPLETION_BONUS = 10;
    private static final int PERFECT_SCORE_BONUS = 50;

    private static final int MAX_QUESTIONS = 5;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_quiz);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();
    }
}
```



```

questionNumberTextView = findViewById(R.id.questionNumberTextView);
questionTextView = findViewById(R.id.questionTextView);
feedbackTextView = findViewById(R.id.feedbackTextView);
feedbackCard = findViewById(R.id.feedbackCard);
timerTextView = findViewById(R.id.timerTextView);
answersRadioGroup = findViewById(R.id.answersRadioGroup);
option1 = findViewById(R.id.option1RadioButton);
option2 = findViewById(R.id.option2RadioButton);
option3 = findViewById(R.id.option3RadioButton);
option4 = findViewById(R.id.option4RadioButton);
submitButton = findViewById(R.id.submitButton);
nextButton = findViewById(R.id.nextButton);

// Doodle views
doodleView = findViewById(R.id.doodleView);
doodleToggleButton = findViewById(R.id.doodleToggleButton);
clearDoodleButton = findViewById(R.id.clearDoodleButton);

submitButton.setOnClickListener(v -> submitAnswer());
nextButton.setOnClickListener(v -> nextQuestion());

// Doodle toggle button
if (doodleToggleButton != null) {
    doodleToggleButton.setOnClickListener(v -> toggleDoodle());
}
if (clearDoodleButton != null) {
    clearDoodleButton.setOnClickListener(v -> {
        if (doodleView != null) {
            doodleView.clearDoodle();
        }
        Toast.makeText(this, "Doodle cleared", Toast.LENGTH_SHORT).show();
    });
}

submitButton.setEnabled(false);
questionTextView.setText("Loading questions...");
loadQuestionsFromFirebase();
}

private void loadQuestionsFromFirebase() {
    db.collection("questions")
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            try {
                if (!queryDocumentSnapshots.isEmpty()) {
                    questions = new ArrayList<>();

                    for (DocumentSnapshot doc : queryDocumentSnapshots.getDocuments()) {
                        Question question = new Question();
                        question.setQuestionId(doc.getString("questionId"));
                        question.setQuestionText(doc.getString("questionText"));

                        List<String> options = (List<String>) doc.get("options");
                        question.setOptions(options);

                        Long correctIndex = doc.getLong("correctAnswerIndex");
                        question.setCorrectAnswerIndex(correctIndex != null ? correctIndex.intValue() : 0);

                        question.setExplanation(doc.getString("explanation"));

                        Long pointValue = doc.getLong("pointValue");
                        question.setPointValue(pointValue != null ? pointValue.intValue() : POINTS_PER_CORRECT);

                        question.setDifficulty(doc.getString("difficulty"));

                        // Only add questions that have valid data
                        if (question.getQuestionText() != null && options != null && options.size() >= 4) {
                            questions.add(question);
                        }
                    }

                    if (!questions.isEmpty()) {
                        // Randomize question order
                        Collections.shuffle(questions);

                        // Limit to configured number of questions
                        if (questions.size() > MAX_QUESTIONS) {
                            questions = new ArrayList<>(questions.subList(0, MAX_QUESTIONS));
                        }

                        // Start the quiz
                        submitButton.setEnabled(true);
                        displayQuestion();
                    } else {

```

```

        Toast.makeText(this, "No valid questions found", Toast.LENGTH_SHORT).show();
        finish();
    }
} else {
    Toast.makeText(this, "No questions available", Toast.LENGTH_SHORT).show();
    finish();
}
} catch (Exception e) {
    Toast.makeText(this, "Error parsing questions: " + e.getMessage(), Toast.LENGTH_SHORT).show();
    finish();
}
}
})
.addOnFailureListener(e -> {
    Toast.makeText(this, "Error loading questions: " + e.getMessage(), Toast.LENGTH_SHORT).show();
    finish();
});
});

private void toggleDoodle() {
    if (doodleView == null) return;

    boolean isEnabled = doodleView.isDrawingEnabled();
    doodleView.setDrawingEnabled(!isEnabled);

    if (!isEnabled) {
        if (doodleToggleButton != null) {
            doodleToggleButton.setBackgroundTintList(android.content.res.ColorStateList.valueOf(ContextCompat.getColor(this,
            }
            if (clearDoodleButton != null) {
                clearDoodleButton.setVisibility(View.VISIBLE);
            }
            Toast.makeText(this, "Doodle mode ON", Toast.LENGTH_SHORT).show();
        } else {
            if (doodleToggleButton != null) {
                doodleToggleButton.setBackgroundTintList(android.content.res.ColorStateList.valueOf(ContextCompat.getColor(this,
            }
            if (clearDoodleButton != null) {
                clearDoodleButton.setVisibility(View.GONE);
            }
            Toast.makeText(this, "Doodle mode OFF", Toast.LENGTH_SHORT).show();
        }
    }
}

@Override
protected void onDestroy() {
    super.onDestroy();
    if (countDownTimer != null) {
        countDownTimer.cancel();
    }
    autoProgressHandler.removeCallbacksAndMessages(null);
}

private void displayQuestion() {
    if (questions == null || questions.isEmpty()) return;

    if (currentQuestionIndex < questions.size()) {
        Question q = questions.get(currentQuestionIndex);
        questionNumberTextView.setText("Question " + (currentQuestionIndex + 1) + " of " + questions.size());
        questionTextView.setText(q.getQuestionText());

        List<String> options = q.getOptions();
        if (options == null || options.size() < 4) {
            Toast.makeText(this, "Invalid question format", Toast.LENGTH_SHORT).show();
            return;
        }

        // Create a list of indices and shuffle them
        List<Integer> indices = new ArrayList<>();
        for (int i = 0; i < options.size(); i++) {
            indices.add(i);
        }
        Collections.shuffle(indices);

        // Set options in shuffled order and track correct answer position
        option1.setText(options.get(indices.get(0)));
        option2.setText(options.get(indices.get(1)));
        option3.setText(options.get(indices.get(2)));
        option4.setText(options.get(indices.get(3)));

        // Find where the correct answer ended up after shuffle
        shuffledCorrectAnswerIndex = indices.indexOf(q.getCorrectAnswerIndex());

        answersRadioGroup.clearCheck();
        feedbackCard.setVisibility(View.GONE);
    }
}

```

```

        answerSubmitted = false;

        // Enable radio buttons
        setRadioButtonsEnabled(true);

        // Show submit button, hide next button
        submitButton.setVisibility(View.VISIBLE);
        nextButton.setVisibility(View.GONE);
        nextButton.setOnClickListener(v -> nextQuestion());

        // Clear doodle for new question
        if (doodleView != null) {
            doodleView.clearDoodle();
        }

        // Start the countdown timer
        startTimer();
    }
}

private void setRadioButtonsEnabled(boolean enabled) {
    option1.setEnabled(enabled);
    option2.setEnabled(enabled);
    option3.setEnabled(enabled);
    option4.setEnabled(enabled);
}

private void startTimer() {
    // Cancel any existing timer
    if (countDownTimer != null) {
        countDownTimer.cancel();
    }

    timerTextView.setTextColor(ContextCompat.getColor(this, R.color.orange_primary));

    countDownTimer = new CountDownTimer(QUESTION_TIME_MS, 1000) {
        @Override
        public void onTick(long millisUntilFinished) {
            long secondsLeft = millisUntilFinished / 1000;
            timerTextView.setText(String.valueOf(secondsLeft));

            // Change color to red when 10 seconds or less
            if (secondsLeft <= 10) {
                timerTextView.setTextColor(ContextCompat.getColor(QuizActivity.this, R.color.red_primary));
            }
        }

        @Override
        public void onFinish() {
            timerTextView.setText("0");
            onTimeUp();
        }
    }.start();
}

private void onTimeUp() {
    if (answerSubmitted) return;

    Question q = questions.get(currentQuestionIndex);
    String correctAnswer = q.getCorrectAnswer();
    feedbackTextView.setText("Time's up! The correct answer is: " + correctAnswer);
    feedbackTextView.setTextColor(ContextCompat.getColor(this, R.color.red_primary));
    feedbackCard.setVisibility(View.VISIBLE);

    answerSubmitted = true;
    submitButton.setVisibility(View.GONE);
    setRadioButtonsEnabled(false);

    // Auto-progress after delay
    scheduleAutoProgress();
}

private void submitAnswer() {
    if (answerSubmitted) return;

    // Stop the timer when answer is submitted
    if (countDownTimer != null) {
        countDownTimer.cancel();
    }

    int selectedId = answersRadioGroup.getCheckedRadioButtonId();
    if (selectedId == -1) {
        Toast.makeText(this, "Please select an answer", Toast.LENGTH_SHORT).show();
        // Restart timer since they didn't submit
    }
}

```

```

        startTimer();
        return;
    }

    int selectedAnswer = -1;
    if (selectedId == R.id.option1RadioButton) selectedAnswer = 0;
    else if (selectedId == R.id.option2RadioButton) selectedAnswer = 1;
    else if (selectedId == R.id.option3RadioButton) selectedAnswer = 2;
    else if (selectedId == R.id.option4RadioButton) selectedAnswer = 3;

    Question q = questions.get(currentQuestionIndex);
    boolean isCorrect = (selectedAnswer == shuffledCorrectAnswerIndex);

    int pointValue = q.getPointValue() > 0 ? q.getPointValue() : POINTS_PER_CORRECT;

    if (isCorrect) {
        score += pointValue;
        correctAnswers++;
        feedbackTextView.setText("Correct! +" + pointValue + " points");
        feedbackTextView.setTextColor(ContextCompat.getColor(this, R.color.green_dark));
    } else {
        String correctAnswer = q.getCorrectAnswer();
        feedbackTextView.setText("Incorrect. The correct answer is: " + correctAnswer);
        feedbackTextView.setTextColor(ContextCompat.getColor(this, R.color.red_primary));
    }

    feedbackCard.setVisibility(View.VISIBLE);
    answerSubmitted = true;
    submitButton.setVisibility(View.GONE);
    setRadioButtonsEnabled(false);

    // Auto-progress after delay (for both correct and incorrect)
    scheduleAutoProgress();
}

private void scheduleAutoProgress() {
    // Show next button for last question only
    if (currentQuestionIndex == questions.size() - 1) {
        nextButton.setText("Go Home");
        nextButton.setVisibility(View.VISIBLE);
        // Set click listener to go home
        nextButton.setOnClickListener(v -> {
            finishQuiz();
            goHome();
        });
    } else {
        // Auto-progress to next question after delay
        autoProgressHandler.postDelayed(() -> {
            nextQuestion();
        }, AUTO_PROGRESS_DELAY_MS);
    }
}

private void goHome() {
    Intent intent = new Intent(this, StudentHomeActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_NEW_TASK);
    startActivity(intent);
    finish();
}

private void nextQuestion() {
    // Cancel any pending auto-progress
    autoProgressHandler.removeCallbacksAndMessages(null);

    currentQuestionIndex++;
    if (currentQuestionIndex < questions.size()) {
        displayQuestion();
    }
}

private void finishQuiz() {
    // Calculate bonuses
    int completionBonus = COMPLETION_BONUS;
    int perfectBonus = (correctAnswers == questions.size()) ? PERFECT_SCORE_BONUS : 0;
    int totalPointsEarned = score + completionBonus + perfectBonus;

    FirebaseUser user = mAuth.getCurrentUser();
    if (user != null) {
        String userId = user.getId();

        QuizResult result = new QuizResult(
            "physics_quiz",
            userId,
            "Physics Quiz",

```

```

        totalPointsEarned,
        questions.size()
    );

    // Use FirestoreHelper to save grade to user's grades subcollection
    FirestoreHelper.getInstance().saveGradeAndUpdateScore(userId, result, (success, e) -> {
        if (success) {
            // Show grade posted notification to student
            NotificationHelper.getInstance(this)
                .showGradePostedNotification(totalPointsEarned, questions.size());

            // Update activity tracking
            InactivityCheckWorker.updateLastActivity(this);
            FirestoreHelper.getInstance().updateLastActivity(userId, null);

            // Update weekly progress tracking
            updateProgressTracking(userId, totalPointsEarned);

            // Check for achievements
            checkAchievements(userId, totalPointsEarned);

            // Notify teachers of quiz completion
            notifyTeachersOfCompletion(userId, totalPointsEarned);
        } else if (e != null) {
            Toast.makeText(this, "Error saving results: " + e.getMessage(),
                Toast.LENGTH_SHORT).show();
        }
    });
}

private void updateProgressTracking(String userId, int score) {
    // Update the weekly progress worker with the new score
    WeeklyProgressWorker.updateCurrentScore(this, score);

    // Get and update total score for progress tracking
    FirestoreHelper.getInstance().getUser(userId,
        user -> {
            if (user != null) {
                WeeklyProgressWorker.updateCurrentScore(this, user.getTotalScore());
            }
        },
        e -> { /* Ignore errors */ }
    );
}

private void checkAchievements(String userId, int quizScore) {
    FirestoreHelper.getInstance().getUser(userId,
        user -> {
            if (user == null) return;

            int totalScore = user.getTotalScore();
            int testsCompleted = user.getTestsCompleted();
            NotificationHelper notificationHelper = NotificationHelper.getInstance(this);

            // Check for first quiz achievement
            if (testsCompleted == 1) {
                notificationHelper.showAchievementNotification(
                    "First Steps!",
                    "You completed your first physics quiz!"
                );
            }

            // Check for 5 quizzes achievement
            if (testsCompleted == 5) {
                notificationHelper.showAchievementNotification(
                    "Quiz Enthusiast!",
                    "You've completed 5 quizzes. Keep it up!"
                );
            }

            // Check for 10 quizzes achievement
            if (testsCompleted == 10) {
                notificationHelper.showAchievementNotification(
                    "Physics Regular!",
                    "10 quizzes completed! You're on a roll!"
                );
            }

            // Check for score milestones
            if (totalScore >= 100 && totalScore - quizScore < 100) {
                notificationHelper.showAchievementNotification(
                    "Century Club!",
                    "You've earned 100+ points total!"
                );
            }
        }
    );
}

```

```

    });
}

if (totalScore >= 500 && totalScore - quizScore < 500) {
    notificationHelper.showAchievementNotification(
        "High Achiever!",
        "You've earned 500+ points! Impressive!"
    );
}

if (totalScore >= 1000 && totalScore - quizScore < 1000) {
    notificationHelper.showAchievementNotification(
        "Physics Master!",
        "1000+ points! You're a true physics expert!"
    );
}

// Check for perfect score on quiz
if (correctAnswers == questions.size()) {
    notificationHelper.showAchievementNotification(
        "Perfect Score!",
        "You got all questions right! Amazing!"
    );
}
},
e -> { /* Ignore achievement check errors */ }
);
}

private void notifyTeachersOfCompletion(String studentId, int score) {
    // Get student name for the notification
    FirestoreHelper.getInstance().getUser(studentId,
        student -> {
            if (student == null) return;

            String studentName = student.getName();
            int percentage = (questions.size() > 0) ?
                (correctAnswers * 100 / questions.size()) : 0;

            // Check if low score alert is needed (below 50%)
            if (percentage < 50) {
                NotificationHelper.getInstance(this)
                    .showLowScoreAlertForTeacher(studentName, score, percentage);
            }
        },
        e -> { /* Ignore errors */ }
    );
}
}
}

```

RegisterActivity.java

app/src/main/java/com/example/phiz/activities/RegisterActivity.java

```
package com.example.phiz.activities;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.example.phiz.R;
import com.example.phiz.models.User;
import com.google.android.material.card.MaterialCardView;
import com.google.android.material.textfield.TextInputEditText;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

public class RegisterActivity extends AppCompatActivity {
    private static final String TAG = "RegisterActivity";
    private TextInputEditText nameEditText, emailEditText, passwordEditText;
    private MaterialCardView studentCard, teacherCard;
    private String selectedRole = "student";
    private Button registerButton;
    private TextView loginTextView;
    private ProgressBar progressBar;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        nameEditText = findViewById(R.id.nameEditText);
        emailEditText = findViewById(R.id.emailEditText);
        passwordEditText = findViewById(R.id.passwordEditText);
        studentCard = findViewById(R.id.studentCard);
        teacherCard = findViewById(R.id.teacherCard);
        registerButton = findViewById(R.id.registerButton);
        loginTextView = findViewById(R.id.loginTextView);
        progressBar = findViewById(R.id.progressBar);

        // Set up role selection card clicks
        studentCard.setOnClickListener(v -> selectRole("student"));
        teacherCard.setOnClickListener(v -> selectRole("teacher"));

        registerButton.setOnClickListener(v -> registerUser());

        loginTextView.setOnClickListener(v -> {
            finish();
        });
    }

    private void selectRole(String role) {
        selectedRole = role;
        if ("student".equals(role)) {
            studentCard.setStrokeColor(ContextCompat.getColor(this, R.color.green_dark));
            studentCard.setStrokeWidth(6);
            studentCard.setCardElevation(8);
            teacherCard.setStrokeColor(ContextCompat.getColor(this, R.color.divider));
            teacherCard.setStrokeWidth(2);
            teacherCard.setCardElevation(2);
        } else {
            teacherCard.setStrokeColor(ContextCompat.getColor(this, R.color.blue_dark));
            teacherCard.setStrokeWidth(6);
            teacherCard.setCardElevation(8);
            studentCard.setStrokeColor(ContextCompat.getColor(this, R.color.divider));
            studentCard.setStrokeWidth(2);
            studentCard.setCardElevation(2);
        }
    }
}
```

```

private void registerUser() {
    String name = nameEditText.getText().toString().trim();
    String email = emailEditText.getText().toString().trim();
    String password = passwordEditText.getText().toString().trim();
    String role = selectedRole;

    if (name.isEmpty() || email.isEmpty() || password.isEmpty()) {
        Toast.makeText(this, "Please fill all fields", Toast.LENGTH_SHORT).show();
        return;
    }

    if (password.length() < 6) {
        Toast.makeText(this, "Password must be at least 6 characters", Toast.LENGTH_SHORT).show();
        return;
    }

    progressBar.setVisibility(View.VISIBLE);
    registerButton.setEnabled(false);

    Log.d(TAG, "Starting user registration for email: " + email);

    mAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                Log.d(TAG, "User authentication successful");
                FirebaseUser firebaseUser = mAuth.getCurrentUser();
                if (firebaseUser != null) {
                    Log.d(TAG, "Creating user object for UID: " + firebaseUser.getUid());
                    User user = new User(firebaseUser.getUid(), email, name, role);
                    saveUserToFirestore(user);
                } else {
                    Log.e(TAG, "Firebase user is null after successful authentication");
                    progressBar.setVisibility(View.GONE);
                    registerButton.setEnabled(true);
                    Toast.makeText(RegisterActivity.this,
                        "Registration error: User is null",
                        Toast.LENGTH_LONG).show();
                }
            } else {
                Log.e(TAG, "User authentication failed", task.getException());
                progressBar.setVisibility(View.GONE);
                registerButton.setEnabled(true);
                Toast.makeText(RegisterActivity.this,
                    "Registration failed: " + task.getException().getMessage(),
                    Toast.LENGTH_LONG).show();
            }
        });
}

private void saveUserToFirestore(User user) {
    Log.d(TAG, "Saving user to Firestore: " + user.getEmail() + " with role: " + user.getRole());

    db.collection("users").document(user.getUid())
        .set(user)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "User data saved to Firestore successfully");
            progressBar.setVisibility(View.GONE);
            registerButton.setEnabled(true);
            Toast.makeText(RegisterActivity.this,
                "Registration successful!", Toast.LENGTH_SHORT).show();

            Intent intent;
            if ("teacher".equals(user.getRole())) {
                Log.d(TAG, "Navigating to TeacherHomeActivity");
                intent = new Intent(RegisterActivity.this, TeacherHomeActivity.class);
            } else {
                Log.d(TAG, "Navigating to StudentHomeActivity");
                intent = new Intent(RegisterActivity.this, StudentHomeActivity.class);
            }
            startActivity(intent);
            finish();
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Failed to save user data to Firestore", e);
            progressBar.setVisibility(View.GONE);
            registerButton.setEnabled(true);
            Toast.makeText(RegisterActivity.this,
                "Error saving user data: " + e.getMessage(),
                Toast.LENGTH_LONG).show();
        });
}
}

```


StudentHomeActivity.java

app/src/main/java/com/example/phiz/activities/StudentHomeActivity.java

```
package com.example.phiz.activities;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

import com.example.phiz.R;
import com.example.phiz.helpers.FCMTokenManager;
import com.example.phiz.helpers.FirestoreHelper;
import com.example.phiz.helpers.WorkerScheduler;
import com.example.phiz.models.NotificationPreferences;
import com.example.phiz.workers.InactivityCheckWorker;
import com.google.android.material.card.MaterialCardView;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

public class StudentHomeActivity extends AppCompatActivity {
    private TextView welcomeTextView, scoreTextView;
    private MaterialCardView simulationCard, quizCard, gradesCard, logoutButton, settingsButton;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private String userId;

    // Permission request launcher
    private final ActivityResultLauncher<String> requestPermissionLauncher =
        registerForActivityResult(new ActivityResultContracts.RequestPermission(), isGranted -> {
            if (isGranted) {
                // Permission granted, initialize notifications
                initializeNotifications();
            }
        });

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_student_home);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            startActivity(new Intent(this, LoginActivity.class));
            finish();
            return;
        }
        userId = currentUser.getId();

        welcomeTextView = findViewById(R.id.welcomeTextView);
        scoreTextView = findViewById(R.id.scoreTextView);
        simulationCard = findViewById(R.id.simulationCard);
        quizCard = findViewById(R.id.quizCard);
        gradesCard = findViewById(R.id.gradesCard);
        logoutButton = findViewById(R.id.logoutButton);
        settingsButton = findViewById(R.id.settingsButton);

        loadUserData();

        // Request notification permission for Android 13+
        requestNotificationPermission();

        // Update last activity
        InactivityCheckWorker.updateLastActivity(this);
        FirestoreHelper.getInstance().updateLastActivity(userId, null);

        simulationCard.setOnClickListener(v -> {
            startActivity(new Intent(StudentHomeActivity.this, PhysicsSimulationActivity.class));
        });
    }
}
```

```

quizCard.setOnClickListener(v -> {
    startActivity(new Intent(StudentHomeActivity.this, QuizActivity.class));
});

gradesCard.setOnClickListener(v -> {
    startActivity(new Intent(StudentHomeActivity.this, GradesActivity.class));
});

if (settingsButton != null) {
    settingsButton.setOnClickListener(v -> {
        startActivity(new Intent(StudentHomeActivity.this, NotificationSettingsActivity.class));
    });
}

logoutButton.setOnClickListener(v -> {
    // Delete FCM token and unsubscribe from topics before logout
    FCMTokenManager tokenManager = FCMTokenManager.getInstance(this);
    tokenManager.deleteToken();
    tokenManager.unsubscribeFromAllTopics();

    mAuth.signOut();
    startActivity(new Intent(StudentHomeActivity.this, LoginActivity.class));
    finish();
});
}

private void requestNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
            != PackageManager.PERMISSION_GRANTED) {
            requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
        } else {
            initializeNotifications();
        }
    } else {
        initializeNotifications();
    }
}

private void initializeNotifications() {
    // Schedule notification workers with user preferences
    FirestoreHelper.getInstance().getNotificationPreferences(userId,
        prefs -> WorkerScheduler.scheduleAllWorkers(this, prefs),
        e -> WorkerScheduler.scheduleAllWorkers(this, NotificationPreferences.createDefault()));
}

@Override
protected void onResume() {
    super.onResume();
    loadUserData();
}

private void loadUserData() {
    db.collection("users").document(userId).get()
        .addOnSuccessListener(documentSnapshot -> {
            if (documentSnapshot.exists()) {
                String name = documentSnapshot.getString("name");
                Long totalScore = documentSnapshot.getLong("totalScore");

                welcomeTextView.setText(name);
                scoreTextView.setText((totalScore != null ? totalScore : 0) + " points");
            }
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Error loading data: " + e.getMessage(),
                Toast.LENGTH_SHORT).show();
        });
}
}

```

TeacherHomeActivity.java

app/src/main/java/com/example/phiz/activities/TeacherHomeActivity.java

```
package com.example.phiz.activities;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.phiz.R;
import com.example.phiz.helpers.FCMTokenManager;
import com.example.phiz.models.User;
import com.google.android.material.card.MaterialCardView;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;

import java.util.ArrayList;
import java.util.List;

public class TeacherHomeActivity extends AppCompatActivity {
    private RecyclerView studentsRecyclerView;
    private LinearLayout emptyStateLayout;
    private MaterialCardView logoutButton;
    private MaterialCardView settingsButton;
    private MaterialCardView createQuestionButton;
    private MaterialCardView viewQuestionsButton;
    private MaterialCardView allGradesButton;
    private StudentAdapter adapter;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private List<User> studentsList = new ArrayList<>();

    // Permission request launcher
    private final ActivityResultLauncher<String> requestPermissionLauncher =
        registerForActivityResult(new ActivityResultContracts.RequestPermission(), isGranted -> {
            if (isGranted) {
                initializeNotifications();
            }
        });

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_teacher_home);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            startActivity(new Intent(this, LoginActivity.class));
            finish();
            return;
        }

        studentsRecyclerView = findViewById(R.id.studentsRecyclerView);
        emptyStateLayout = findViewById(R.id.emptyStateLayout);
        logoutButton = findViewById(R.id.logoutButton);
        settingsButton = findViewById(R.id.settingsButton);
        createQuestionButton = findViewById(R.id.createQuestionButton);
        viewQuestionsButton = findViewById(R.id.viewQuestionsButton);
        allGradesButton = findViewById(R.id.allGradesButton);

        // Request notification permission for Android 13+
    }
```

```

requestNotificationPermission();

studentsRecyclerView.setLayoutManager(new LinearLayoutManager(this));
adapter = new StudentAdapter(studentsList, student -> {
    Intent intent = new Intent(TeacherHomeActivity.this, GradesActivity.class);
    intent.putExtra("userId", student.getId());
    intent.putExtra("studentName", student.getName());
    startActivity(intent);
});
studentsRecyclerView.setAdapter(adapter);

loadStudents();

createQuestionButton.setOnClickListener(v -> {
    startActivity(new Intent(TeacherHomeActivity.this, CreateQuestionActivity.class));
});

viewQuestionsButton.setOnClickListener(v -> {
    startActivity(new Intent(TeacherHomeActivity.this, ViewQuestionsActivity.class));
});

allGradesButton.setOnClickListener(v -> {
    startActivity(new Intent(TeacherHomeActivity.this, AllGradesActivity.class));
});

if (settingsButton != null) {
    settingsButton.setOnClickListener(v -> {
        Intent settingsIntent = new Intent(TeacherHomeActivity.this, NotificationSettingsActivity.class);
        settingsIntent.putExtra(NotificationSettingsActivity.EXTRA_USER_ROLE, "teacher");
        startActivity(settingsIntent);
    });
}

logoutButton.setOnClickListener(v -> {
    // Delete FCM token and unsubscribe from topics before logout
    FCMTokenManager tokenManager = FCMTokenManager.getInstance(this);
    tokenManager.deleteToken();
    tokenManager.unsubscribeFromAllTopics();

    mAuth.signOut();
    startActivity(new Intent(TeacherHomeActivity.this, LoginActivity.class));
    finish();
});
}

private void requestNotificationPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
            != PackageManager.PERMISSION_GRANTED) {
            requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
        } else {
            initializeNotifications();
        }
    } else {
        initializeNotifications();
    }
}

private void initializeNotifications() {
    // Teachers don't need study reminders, inactivity checks, or weekly progress workers.
    // Those are student-only features. No workers to schedule for teachers.
}

private void loadStudents() {
    db.collection("users")
        .whereEqualTo("role", "student")
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            studentsList.clear();
            for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                User user = document.toObject(User.class);
                studentsList.add(user);
            }
            adapter.notifyDataSetChanged();

            if (studentsList.isEmpty()) {
                studentsRecyclerView.setVisibility(View.GONE);
                emptyStateLayout.setVisibility(View.VISIBLE);
            } else {
                studentsRecyclerView.setVisibility(View.VISIBLE);
                emptyStateLayout.setVisibility(View.GONE);
            }
        })
        .addOnFailureListener(e -> {

```

```

        Toast.makeText(this, "Error loading students: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
    });
}

private static class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentViewHolder> {
    private List<User> students;
    private OnStudentClickListener listener;

    public interface OnStudentClickListener {
        void onStudentClick(User student);
    }

    public StudentAdapter(List<User> students, OnStudentClickListener listener) {
        this.students = students;
        this.listener = listener;
    }

    @NonNull
    @Override
    public StudentViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_student, parent, false);
        return new StudentViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull StudentViewHolder holder, int position) {
        User student = students.get(position);
        holder.nameTextView.setText(student.getName());
        holder.emailTextView.setText(student.getEmail());
        holder.scoreTextView.setText(student.getTotalScore() + " points");

        holder.itemView.setOnClickListener(v -> {
            if (listener != null) {
                listener.onStudentClick(student);
            }
        });
    }

    @Override
    public int getItemCount() {
        return students.size();
    }

    static class StudentViewHolder extends RecyclerView.ViewHolder {
        TextView nameTextView, emailTextView, scoreTextView;

        public StudentViewHolder(@NonNull View itemView) {
            super(itemView);
            nameTextView = itemView.findViewById(R.id.studentNameTextView);
            emailTextView = itemView.findViewById(R.id.studentEmailTextView);
            scoreTextView = itemView.findViewById(R.id.studentScoreTextView);
        }
    }
}

```

ViewQuestionsActivity.java

app/src/main/java/com/example/phiz/activities/ViewQuestionsActivity.java

```
package com.example.phiz.activities;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageButton;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.example.phiz.R;
import com.example.phiz.models.Question;
import com.google.android.material.appbar.MaterialToolbar;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;

import java.util.ArrayList;
import java.util.List;

public class ViewQuestionsActivity extends AppCompatActivity {
    private static final String TAG = "ViewQuestionsActivity";

    private RecyclerView questionsRecyclerView;
    private TextView emptyTextView;
    private ProgressBar progressBar;
    private MaterialToolbar toolbar;
    private QuestionAdapter adapter;

    private FirebaseAuth mAuth;
    private FirebaseFirestore db;
    private List<Question> questionsList = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_view_questions);

        mAuth = FirebaseAuth.getInstance();
        db = FirebaseFirestore.getInstance();

        FirebaseUser currentUser = mAuth.getCurrentUser();
        if (currentUser == null) {
            finish();
            return;
        }

        initializeViews();
        setupToolbar();
        setupRecyclerView();
        loadQuestions();
    }

    private void initializeViews() {
        toolbar = findViewById(R.id.toolbar);
        questionsRecyclerView = findViewById(R.id.questionsRecyclerView);
        emptyTextView = findViewById(R.id.emptyTextView);
        progressBar = findViewById(R.id.progressBar);
    }

    private void setupToolbar() {
        toolbar.setNavigationOnClickListener(v -> finish());
    }

    private void setupRecyclerView() {
        questionsRecyclerView.setLayoutManager(new LinearLayoutManager(this));
        adapter = new QuestionAdapter(questionsList, this::confirmDeleteQuestion);
        questionsRecyclerView.setAdapter(adapter);
    }

    private void loadQuestions() {
        setLoading(true);
    }
}
```

```

db.collection("questions")
    .get()
    .addOnSuccessListener(queryDocumentSnapshots -> {
        setLoading(false);
        questionsList.clear();
        for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
            Question question = document.toObject(Question.class);
            questionsList.add(question);
        }
        adapter.notifyDataSetChanged();
        updateEmptyState();
    })
    .addOnFailureListener(e -> {
        setLoading(false);
        Toast.makeText(this, "Error loading questions: " + e.getMessage(),
            Toast.LENGTH_SHORT).show();
        updateEmptyState();
    });
}

private void confirmDeleteQuestion(Question question) {
    new AlertDialog.Builder(this)
        .setTitle("Delete Question")
        .setMessage("Are you sure you want to delete this question?")
        .setPositiveButton("Delete", (dialog, which) -> deleteQuestion(question))
        .setNegativeButton("Cancel", null)
        .show();
}

private void deleteQuestion(Question question) {
    db.collection("questions")
        .document(question.getId())
        .delete()
        .addOnSuccessListener(aVoid -> {
            questionsList.remove(question);
            adapter.notifyDataSetChanged();
            updateEmptyState();
            Toast.makeText(this, "Question deleted", Toast.LENGTH_SHORT).show();
        })
        .addOnFailureListener(e -> {
            Toast.makeText(this, "Error deleting question: " + e.getMessage(),
                Toast.LENGTH_SHORT).show();
        });
}

private void setLoading(boolean isLoading) {
    progressBar.setVisibility(isLoading ? View.VISIBLE : View.GONE);
}

private void updateEmptyState() {
    if (questionsList.isEmpty()) {
        questionsRecyclerView.setVisibility(View.GONE);
        emptyTextView.setVisibility(View.VISIBLE);
    } else {
        questionsRecyclerView.setVisibility(View.VISIBLE);
        emptyTextView.setVisibility(View.GONE);
    }
}

// Interface for delete callback
interface OnDeleteClickListener {
    void onDeleteClick(Question question);
}

// RecyclerView Adapter
private static class QuestionAdapter extends RecyclerView.Adapter<QuestionAdapter.QuestionViewHolder> {
    private final List<Question> questions;
    private final OnDeleteClickListener deleteListener;

    public QuestionAdapter(List<Question> questions, OnDeleteClickListener deleteListener) {
        this.questions = questions;
        this.deleteListener = deleteListener;
    }

    @NonNull
    @Override
    public QuestionViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.item_question, parent, false);
        return new QuestionViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull QuestionViewHolder holder, int position) {

```

```

        Question question = questions.get(position);
        holder.bind(question, deleteListener);
    }

    @Override
    public int getItemCount() {
        return questions.size();
    }

    static class QuestionViewHolder extends RecyclerView.ViewHolder {
        private final TextView difficultyBadge;
        private final TextView pointsTextView;
        private final TextView questionTextView;
        private final TextView option1TextView;
        private final TextView option2TextView;
        private final TextView option3TextView;
        private final TextView option4TextView;
        private final TextView correctAnswerTextView;
        private final ImageButton deleteButton;

        public QuestionViewHolder(@NonNull View itemView) {
            super(itemView);
            difficultyBadge = itemView.findViewById(R.id.difficultyBadge);
            pointsTextView = itemView.findViewById(R.id.pointsTextView);
            questionTextView = itemView.findViewById(R.id.questionTextView);
            option1TextView = itemView.findViewById(R.id.option1TextView);
            option2TextView = itemView.findViewById(R.id.option2TextView);
            option3TextView = itemView.findViewById(R.id.option3TextView);
            option4TextView = itemView.findViewById(R.id.option4TextView);
            correctAnswerTextView = itemView.findViewById(R.id.correctAnswerTextView);
            deleteButton = itemView.findViewById(R.id.deleteButton);
        }

        public void bind(Question question, OnDeleteClickListener deleteListener) {
            questionTextView.setText(question.getQuestionText());
            pointsTextView.setText(question.getPointValue() + " pts");

            // Set difficulty badge
            String difficulty = question.getDifficulty();
            difficultyBadge.setText(difficulty != null ? difficulty : "medium");

            // Set badge color based on difficulty
            int badgeColor;
            if ("easy".equals(difficulty)) {
                badgeColor = 0xFF4CAF50; // Green
            } else if ("hard".equals(difficulty)) {
                badgeColor = 0xFFFF4433; // Red
            } else {
                badgeColor = 0xFFFF9800; // Orange for medium
            }
            difficultyBadge.getBackground().setTint(badgeColor);

            // Set options
            List<String> options = question.getOptions();
            if (options != null && options.size() >= 4) {
                option1TextView.setText("1. " + options.get(0));
                option2TextView.setText("2. " + options.get(1));
                option3TextView.setText("3. " + options.get(2));
                option4TextView.setText("4. " + options.get(3));

                // Highlight correct answer
                int correctIndex = question.getCorrectAnswerIndex();
                correctAnswerTextView.setText("Correct: " + options.get(correctIndex));
            }

            deleteButton.setOnClickListener(v -> deleteListener.onDeleteClick(question));
        }
    }
}

```


FCMTokenManager.java

app/src/main/java/com/example/phiz/helpers/FCMTokenManager.java

```
package com.example.phiz.helpers;

import android.content.Context;
import android.content.SharedPreferences;
import android.util.Log;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.messaging.FirebaseMessaging;

import java.util.HashMap;
import java.util.Map;

/**
 * Manager class for FCM token lifecycle.
 * Handles token retrieval, storage, and cleanup.
 */
public class FCMTokenManager {
    private static final String TAG = "FCMTokenManager";
    private static final String PREFS_NAME = "fcm_prefs";
    private static final String KEY_FCM_TOKEN = "fcm_token";

    private static FCMTokenManager instance;
    private final Context context;
    private final FirebaseFirestore db;
    private final FirebaseAuth auth;

    private FCMTokenManager(Context context) {
        this.context = context.getApplicationContext();
        this.db = FirebaseFirestore.getInstance();
        this.auth = FirebaseAuth.getInstance();
    }

    public static synchronized FCMTokenManager getInstance(Context context) {
        if (instance == null) {
            instance = new FCMTokenManager(context);
        }
        return instance;
    }

    /**
     * Get the current FCM token and save it to Firestore.
     * Call this after successful login.
     */
    public void initializeToken() {
        FirebaseUser user = auth.getCurrentUser();
        if (user == null) {
            Log.w(TAG, "Cannot initialize token: no authenticated user");
            return;
        }

        FirebaseMessaging.getInstance().getToken()
            .addOnCompleteListener(task -> {
                if (!task.isSuccessful()) {
                    Log.w(TAG, "Failed to get FCM token", task.getException());
                    return;
                }

                String token = task.getResult();
                Log.d(TAG, "FCM token retrieved: " + token);

                // Save token locally
                saveTokenLocally(token);

                // Save token to Firestore
                saveTokenToFirestore(token);
            });
    }

    /**
     * Save FCM token to Firestore for the current user
     */
    public void saveTokenToFirestore(String token) {
        FirebaseUser user = auth.getCurrentUser();
        if (user == null) {
            Log.w(TAG, "Cannot save token: no authenticated user");
            return;
        }
    }
}
```

```

String userId = user.getUid();
Map<String, Object> updates = new HashMap<>();
updates.put("fcmToken", token);
updates.put("lastTokenUpdate", FieldValue.serverTimestamp());

db.collection(FirestoreHelper.COLLECTION_USERS)
    .document(userId)
    .update(updates)
    .addOnSuccessListener(aVoid -> {
        Log.d(TAG, "FCM token saved to Firestore for user: " + userId);
        saveTokenLocally(token);
    })
    .addOnFailureListener(e -> {
        Log.e(TAG, "Error saving FCM token to Firestore", e);
    });
}

/**
 * Delete FCM token from Firestore on logout
 */
public void deleteToken() {
    FirebaseUser user = auth.getCurrentUser();
    if (user == null) {
        // Just clear local token
        clearLocalToken();
        return;
    }

    String userId = user.getUid();

    // Delete from Firestore
    Map<String, Object> updates = new HashMap<>();
    updates.put("fcmToken", FieldValue.delete());

    db.collection(FirestoreHelper.COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "FCM token deleted from Firestore for user: " + userId);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error deleting FCM token from Firestore", e);
        });

    // Clear local token
    clearLocalToken();

    // Delete the FCM token instance
    FirebaseMessaging.getInstance().deleteToken()
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                Log.d(TAG, "FCM token deleted from device");
            } else {
                Log.w(TAG, "Failed to delete FCM token from device", task.getException());
            }
        });
}

/**
 * Save token to SharedPreferences
 */
private void saveTokenLocally(String token) {
    SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    prefs.edit().putString(KEY_FCM_TOKEN, token).apply();
}

/**
 * Get locally stored token
 */
public String getLocalToken() {
    SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    return prefs.getString(KEY_FCM_TOKEN, null);
}

/**
 * Clear locally stored token
 */
private void clearLocalToken() {
    SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    prefs.edit().remove(KEY_FCM_TOKEN).apply();
}

/**

```

```

    * Subscribe to a topic for receiving topic-based messages
    */
    public void subscribeToTopic(String topic) {
        FirebaseMessaging.getInstance().subscribeToTopic(topic)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    Log.d(TAG, "Subscribed to topic: " + topic);
                } else {
                    Log.w(TAG, "Failed to subscribe to topic: " + topic, task.getException());
                }
            });
    }

    /**
     * Unsubscribe from a topic
     */
    public void unsubscribeFromTopic(String topic) {
        FirebaseMessaging.getInstance().unsubscribeFromTopic(topic)
            .addOnCompleteListener(task -> {
                if (task.isSuccessful()) {
                    Log.d(TAG, "Unsubscribed from topic: " + topic);
                } else {
                    Log.w(TAG, "Failed to unsubscribe from topic: " + topic, task.getException());
                }
            });
    }

    /**
     * Subscribe user to role-based topics after login
     */
    public void subscribeToRoleTopics(String role) {
        if ("student".equals(role)) {
            subscribeToTopic("all_students");
            subscribeToTopic("quiz_updates");
        } else if ("teacher".equals(role)) {
            subscribeToTopic("all_teachers");
            subscribeToTopic("student_activity");
        }
    }

    /**
     * Unsubscribe from all topics on logout
     */
    public void unsubscribeFromAllTopics() {
        unsubscribeFromTopic("all_students");
        unsubscribeFromTopic("all_teachers");
        unsubscribeFromTopic("quiz_updates");
        unsubscribeFromTopic("student_activity");
    }
}

```

FirestoreHelper.java

app/src/main/java/com/example/phiz/helpers/FirestoreHelper.java

```
package com.example.phiz.helpers;

import android.util.Log;

import com.example.phiz.models.NotificationPreferences;
import com.example.phiz.models.Question;
import com.example.phiz.models.QuizResult;
import com.example.phiz.models.Test;
import com.example.phiz.models.User;
import com.google.firebase.Timestamp;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.DocumentReference;
import com.google.firebase.firestore.FieldValue;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.Query;
import com.google.firebase.firestore.QueryDocumentSnapshot;
import com.google.firebase.firestore.WriteBatch;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Helper class for Firebase Firestore operations.
 * Provides centralized methods for CRUD operations on all collections.
 *
 * Data Structure:
 * - users/{userId} - User documents
 * - users/{userId}/grades/{gradeId} - Grades subcollection under each user
 * - questions/{questionId} - Question documents
 * - tests/{testId} - Test documents
 */
public class FirestoreHelper {
    private static final String TAG = "FirestoreHelper";

    // Collection names
    public static final String COLLECTION_USERS = "users";
    public static final String COLLECTION_QUESTIONS = "questions";
    public static final String COLLECTION_TESTS = "tests";
    public static final String SUBCOLLECTION_GRADES = "grades";

    // Singleton instance
    private static FirestoreHelper instance;
    private final FirebaseFirestore db;
    private final FirebaseAuth auth;

    // Private constructor for singleton
    private FirestoreHelper() {
        db = FirebaseFirestore.getInstance();
        auth = FirebaseAuth.getInstance();
    }

    /**
     * Get singleton instance
     */
    public static synchronized FirestoreHelper getInstance() {
        if (instance == null) {
            instance = new FirestoreHelper();
        }
        return instance;
    }

    /**
     * Get Firestore instance for direct access if needed
     */
    public FirebaseFirestore getDb() {
        return db;
    }

    /**
     * Get current authenticated user ID
     */
    public String getCurrentUserId() {
        FirebaseUser user = auth.getCurrentUser();
        return user != null ? user.getUid() : null;
    }
}
```

```

// ===== CALLBACK INTERFACES =====

public interface OnSuccessListener<T> {
    void onSuccess(T result);
}

public interface OnFailureListener {
    void onFailure(Exception e);
}

public interface OnCompleteListener {
    void onComplete(boolean success, Exception e);
}

// ===== USER OPERATIONS =====

/**
 * Create a new user in Firestore
 */
public void createUser(User user, OnCompleteListener listener) {
    db.collection(COLLECTION_USERS)
        .document(user.getId())
        .set(user)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "User created successfully: " + user.getId());
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error creating user", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Get user by ID
 */
public void getUser(String userId, OnSuccessListener<User> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .get()
        .addOnSuccessListener(document -> {
            if (document.exists()) {
                User user = document.toObject(User.class);
                if (onSuccess != null) onSuccess.onSuccess(user);
            } else {
                if (onFailure != null) onFailure.onFailure(new Exception("User not found"));
            }
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting user", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
 * Get current authenticated user from Firestore
 */
public void getCurrentUser(OnSuccessListener<User> onSuccess, OnFailureListener onFailure) {
    String userId = getCurrentUserId();
    if (userId == null) {
        if (onFailure != null) onFailure.onFailure(new Exception("No authenticated user"));
        return;
    }
    getUser(userId, onSuccess, onFailure);
}

/**
 * Update user data
 */
public void updateUser(String userId, Map<String, Object> updates, OnCompleteListener listener) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "User updated successfully: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error updating user", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**

```

```

* Update user's total score
*/
public void updateUserScore(String userId, int scoreToAdd, OnCompleteListener listener) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(
            "totalScore", FieldValue.increment(scoreToAdd),
            "testsCompleted", FieldValue.increment(1)
        )
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "User score updated: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error updating user score", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
* Delete user
*/
public void deleteUser(String userId, OnCompleteListener listener) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "User deleted: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error deleting user", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
* Get all users
*/
public void getAllUsers(OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<User> users = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                users.add(doc.toObject(User.class));
            }
            if (onSuccess != null) onSuccess.onSuccess(users);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting all users", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
* Get users by role (student/teacher)
*/
public void getUsersByRole(String role, OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .whereEqualTo("role", role)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<User> users = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                users.add(doc.toObject(User.class));
            }
            if (onSuccess != null) onSuccess.onSuccess(users);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting users by role", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
* Get all students
*/
public void getAllStudents(OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
    getUsersByRole("student", onSuccess, onFailure);
}

/**
* Get all teachers

```

```

    */
    public void getAllTeachers(OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
        getUsersByRole("teacher", onSuccess, onFailure);
    }

    /**
     * Get top students by score
     */
    public void getTopStudents(int limit, OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_USERS)
            .whereEqualTo("role", "student")
            .orderBy("totalScore", Query.Direction.DESENDING)
            .limit(limit)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                List<User> users = new ArrayList<>();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    users.add(doc.toObject(User.class));
                }
                if (onSuccess != null) onSuccess.onSuccess(users);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting top students", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Check if user exists
     */
    public void userExists(String userId, OnSuccessListener<Boolean> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_USERS)
            .document(userId)
            .get()
            .addOnSuccessListener(document -> {
                if (onSuccess != null) onSuccess.onSuccess(document.exists());
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error checking user exists", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    // ===== GRADES SUBCOLLECTION OPERATIONS =====
    // Grades are stored as: users/{userId}/grades/{gradeId}

    /**
     * Save a grade to user's grades subcollection
     */
    public void saveGrade(String userId, QuizResult grade, OnCompleteListener listener) {
        db.collection(COLLECTION_USERS)
            .document(userId)
            .collection(SUBCOLLECTION_GRADES)
            .add(grade)
            .addOnSuccessListener(docRef -> {
                Log.d(TAG, "Grade saved for user: " + userId);
                if (listener != null) listener.onComplete(true, null);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error saving grade", e);
                if (listener != null) listener.onComplete(false, e);
            });
    }

    /**
     * Save grade and update user score in a single batch
     */
    public void saveGradeAndUpdateScore(String userId, QuizResult grade, OnCompleteListener listener) {
        WriteBatch batch = db.batch();

        // Add grade to subcollection
        DocumentReference gradeRef = db.collection(COLLECTION_USERS)
            .document(userId)
            .collection(SUBCOLLECTION_GRADES)
            .document();
        batch.set(gradeRef, grade);

        // Update user score
        DocumentReference userRef = db.collection(COLLECTION_USERS).document(userId);
        batch.update(userRef,
            "totalScore", FieldValue.increment(grade.getScore()),
            "testsCompleted", FieldValue.increment(1));

        batch.commit()
    }

```

```

        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Grade saved and user score updated");
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error in batch operation", e);
            if (listener != null) listener.onComplete(false, e);
        });
    }

    /**
     * Save grade for current user
     */
    public void saveCurrentUserGrade(QuizResult grade, OnCompleteListener listener) {
        String userId = getCurrentUserId();
        if (userId == null) {
            if (listener != null) listener.onComplete(false, new Exception("No authenticated user"));
            return;
        }
        grade.setUserId(userId);
        saveGradeAndUpdateScore(userId, grade, listener);
    }

    /**
     * Get grades for a specific user
     */
    public void getUserGrades(String userId, OnSuccessListener<List<QuizResult>> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_USERS)
            .document(userId)
            .collection(SUBCOLLECTION_GRADES)
            .orderBy("timestamp", Query.Direction.DESENDING)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                List<QuizResult> grades = new ArrayList<>();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    QuizResult grade = doc.toObject(QuizResult.class);
                    grades.add(grade);
                }
                if (onSuccess != null) onSuccess.onSuccess(grades);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting user grades", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Get current user's grades
     */
    public void getCurrentUserGrades(OnSuccessListener<List<QuizResult>> onSuccess, OnFailureListener onFailure) {
        String userId = getCurrentUserId();
        if (userId == null) {
            if (onFailure != null) onFailure.onFailure(new Exception("No authenticated user"));
            return;
        }
        getUserGrades(userId, onSuccess, onFailure);
    }

    /**
     * Data class for grade with user info
     */
    public static class UserGrade {
        public String userName;
        public String oderId;
        public QuizResult grade;

        public UserGrade(String userName, String oderId, QuizResult grade) {
            this.userName = userName;
            this.oderId = oderId;
            this.grade = grade;
        }
    }

    /**
     * Get all grades from all users (for teacher view)
     */
    public void getAllGrades(OnSuccessListener<List<UserGrade>> onSuccess, OnFailureListener onFailure) {
        // First get all users
        db.collection(COLLECTION_USERS)
            .get()
            .addOnSuccessListener(userSnapshots -> {
                List<UserGrade> allGrades = new ArrayList<>();
                List<User> users = new ArrayList<>();
            });
    }

```



```

        for (QueryDocumentSnapshot doc : userSnapshots) {
            users.add(doc.toObject(User.class));
        }

        if (users.isEmpty()) {
            if (onSuccess != null) onSuccess.onSuccess(allGrades);
            return;
        }

        // Counter to track completed queries
        AtomicInteger completedQueries = new AtomicInteger(0);
        int totalUsers = users.size();

        // Get grades for each user
        for (User user : users) {
            db.collection(COLLECTION_USERS)
                .document(user.getUid())
                .collection(SUBCOLLECTION_GRADES)
                .get()
                .addOnSuccessListener(gradeSnapshots -> {
                    synchronized (allGrades) {
                        for (QueryDocumentSnapshot gradeDoc : gradeSnapshots) {
                            QuizResult grade = gradeDoc.toObject(QuizResult.class);
                            allGrades.add(new UserGrade(user.getName(), user.getUid(), grade));
                        }
                    }

                    // Check if all queries completed
                    if (completedQueries.incrementAndGet() == totalUsers) {
                        // Sort by timestamp descending
                        allGrades.sort((a, b) -> {
                            if (a.grade.getTimestamp() == null && b.grade.getTimestamp() == null) return 0;
                            if (a.grade.getTimestamp() == null) return 1;
                            if (b.grade.getTimestamp() == null) return -1;
                            return b.grade.getTimestamp().compareTo(a.grade.getTimestamp());
                        });
                        if (onSuccess != null) onSuccess.onSuccess(allGrades);
                    }
                })
                .addOnFailureListener(e -> {
                    // Still increment counter on failure to avoid hanging
                    if (completedQueries.incrementAndGet() == totalUsers) {
                        allGrades.sort((a, b) -> {
                            if (a.grade.getTimestamp() == null && b.grade.getTimestamp() == null) return 0;
                            if (a.grade.getTimestamp() == null) return 1;
                            if (b.grade.getTimestamp() == null) return -1;
                            return b.grade.getTimestamp().compareTo(a.grade.getTimestamp());
                        });
                        if (onSuccess != null) onSuccess.onSuccess(allGrades);
                    }
                });
        }
    }

    .addOnFailureListener(e -> {
        Log.e(TAG, "Error getting all grades", e);
        if (onFailure != null) onFailure.onFailure(e);
    });
}

/**
 * Delete a grade from user's subcollection
 */
public void deleteGrade(String userId, String gradeId, OnCompleteListener listener) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .collection(SUBCOLLECTION_GRADES)
        .document(gradeId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Grade deleted: " + gradeId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error deleting grade", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Get user's best score for a specific quiz
 */
public void getUserBestScoreForQuiz(String userId, String quizId, OnSuccessListener<Integer> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .document(userId)

```

```

        .collection(SUBCOLLECTION_GRADES)
        .whereEqualTo("quizId", quizId)
        .orderBy("score", Query.Direction.DESENDING)
        .limit(1)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            if (!querySnapshot.isEmpty()) {
                QuizResult result = querySnapshot.getDocuments().get(0).toObject(QuizResult.class);
                if (onSuccess != null) onSuccess.onSuccess(result != null ? result.getScore() : 0);
            } else {
                if (onSuccess != null) onSuccess.onSuccess(0);
            }
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting user best score", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
    }

    /**
     * Get grade count for a user
     */
    public void getUserGradeCount(String userId, OnSuccessListener<Integer> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_USERS)
            .document(userId)
            .collection(SUBCOLLECTION_GRADES)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                if (onSuccess != null) onSuccess.onSuccess(querySnapshot.size());
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting grade count", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    // ===== QUESTION OPERATIONS =====

    /**
     * Create a new question
     */
    public void createQuestion(Question question, OnCompleteListener listener) {
        DocumentReference docRef;
        if (question.getQuestionId() != null && !question.getQuestionId().isEmpty()) {
            docRef = db.collection(COLLECTION_QUESTIONS).document(question.getQuestionId());
        } else {
            docRef = db.collection(COLLECTION_QUESTIONS).document();
            question.setQuestionId(docRef.getId());
        }

        docRef.set(question)
            .addOnSuccessListener(aVoid -> {
                Log.d(TAG, "Question created: " + question.getQuestionId());
                if (listener != null) listener.onComplete(true, null);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error creating question", e);
                if (listener != null) listener.onComplete(false, e);
            });
    }

    /**
     * Get question by ID
     */
    public void getQuestion(String questionId, OnSuccessListener<Question> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_QUESTIONS)
            .document(questionId)
            .get()
            .addOnSuccessListener(document -> {
                if (document.exists()) {
                    Question question = document.toObject(Question.class);
                    if (onSuccess != null) onSuccess.onSuccess(question);
                } else {
                    if (onFailure != null) onFailure.onFailure(new Exception("Question not found"));
                }
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting question", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Get all questions

```

```

    */
    public void getAllQuestions(OnSuccessListener<List<Question>> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_QUESTIONS)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                List<Question> questions = new ArrayList<>();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    questions.add(doc.toObject(Question.class));
                }
                if (onSuccess != null) onSuccess.onSuccess(questions);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting all questions", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Get questions by difficulty
     */
    public void getQuestionsByDifficulty(String difficulty, OnSuccessListener<List<Question>> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_QUESTIONS)
            .whereEqualTo("difficulty", difficulty)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                List<Question> questions = new ArrayList<>();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    questions.add(doc.toObject(Question.class));
                }
                if (onSuccess != null) onSuccess.onSuccess(questions);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting questions by difficulty", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Update question
     */
    public void updateQuestion(String questionId, Map<String, Object> updates, OnCompleteListener listener) {
        db.collection(COLLECTION_QUESTIONS)
            .document(questionId)
            .update(updates)
            .addOnSuccessListener(aVoid -> {
                Log.d(TAG, "Question updated: " + questionId);
                if (listener != null) listener.onComplete(true, null);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error updating question", e);
                if (listener != null) listener.onComplete(false, e);
            });
    }

    /**
     * Delete question
     */
    public void deleteQuestion(String questionId, OnCompleteListener listener) {
        db.collection(COLLECTION_QUESTIONS)
            .document(questionId)
            .delete()
            .addOnSuccessListener(aVoid -> {
                Log.d(TAG, "Question deleted: " + questionId);
                if (listener != null) listener.onComplete(true, null);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error deleting question", e);
                if (listener != null) listener.onComplete(false, e);
            });
    }

    /**
     * Get random questions for a quiz
     */
    public void getRandomQuestions(int count, OnSuccessListener<List<Question>> onSuccess, OnFailureListener onFailure) {
        db.collection(COLLECTION_QUESTIONS)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                List<Question> allQuestions = new ArrayList<>();
                for (QueryDocumentSnapshot doc : querySnapshot) {
                    allQuestions.add(doc.toObject(Question.class));
                }

                // Shuffle and take requested count
            });
    }

```

```

        java.util.Collections.shuffle(allQuestions);
        List<Question> randomQuestions = allQuestions.subList(0, Math.min(count, allQuestions.size()));

        if (onSuccess != null) onSuccess.onSuccess(randomQuestions);
    })
    .addOnFailureListener(e -> {
        Log.e(TAG, "Error getting random questions", e);
        if (onFailure != null) onFailure.onFailure(e);
    });
}

// ===== TEST OPERATIONS =====

/**
 * Create a new test
 */
public void createTest(Test test, OnCompleteListener listener) {
    DocumentReference docRef;
    if (test.getId() != null && !test.getId().isEmpty()) {
        docRef = db.collection(COLLECTION_TESTS).document(test.getId());
    } else {
        docRef = db.collection(COLLECTION_TESTS).document();
        test.setId(docRef.getId());
    }

    docRef.set(test)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Test created: " + test.getId());
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error creating test", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Get test by ID
 */
public void getTest(String testId, OnSuccessListener<Test> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_TESTS)
        .document(testId)
        .get()
        .addOnSuccessListener(document -> {
            if (document.exists()) {
                Test test = document.toObject(Test.class);
                if (onSuccess != null) onSuccess.onSuccess(test);
            } else {
                if (onFailure != null) onFailure.onFailure(new Exception("Test not found"));
            }
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting test", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
 * Get all tests
 */
public void getAllTests(OnSuccessListener<List<Test>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_TESTS)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<Test> tests = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                tests.add(doc.toObject(Test.class));
            }
            if (onSuccess != null) onSuccess.onSuccess(tests);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting all tests", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
 * Get active tests only
 */
public void getActiveTests(OnSuccessListener<List<Test>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_TESTS)
        .whereEqualTo("active", true)
        .get()
        .addOnSuccessListener(querySnapshot -> {

```

```

        List<Test> tests = new ArrayList<>();
        for (QueryDocumentSnapshot doc : querySnapshot) {
            tests.add(doc.toObject(Test.class));
        }
        if (onSuccess != null) onSuccess.onSuccess(tests);
    })
    .addOnFailureListener(e -> {
        Log.e(TAG, "Error getting active tests", e);
        if (onFailure != null) onFailure.onFailure(e);
    });
}

/**
 * Get tests by creator (teacher)
 */
public void getTestsByCreator(String creatorId, OnSuccessListener<List<Test>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_TESTS)
        .whereEqualTo("createdBy", creatorId)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<Test> tests = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                tests.add(doc.toObject(Test.class));
            }
            if (onSuccess != null) onSuccess.onSuccess(tests);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting tests by creator", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
 * Update test
 */
public void updateTest(String testId, Map<String, Object> updates, OnCompleteListener listener) {
    updates.put("updatedAt", Timestamp.now());
    db.collection(COLLECTION_TESTS)
        .document(testId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Test updated: " + testId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error updating test", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Delete test
 */
public void deleteTest(String testId, OnCompleteListener listener) {
    db.collection(COLLECTION_TESTS)
        .document(testId)
        .delete()
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Test deleted: " + testId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error deleting test", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Toggle test active status
 */
public void toggleTestActive(String testId, boolean isActive, OnCompleteListener listener) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("active", isActive);
    updateTest(testId, updates, listener);
}

// ===== UTILITY OPERATIONS =====

/**
 * Check if a document exists in a collection
 */
public void documentExists(String collection, String documentId, OnSuccessListener<Boolean> onSuccess, OnFailureListener onFailure) {
    db.collection(collection)
        .document(documentId)

```

```

        .get()
        .addOnSuccessListener(document -> {
            if (onSuccess != null) onSuccess.onSuccess(document.exists());
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error checking document exists", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
    }

    /**
     * Get document count in a collection
     */
    public void getCollectionCount(String collection, OnSuccessListener<Integer> onSuccess, OnFailureListener onFailure) {
        db.collection(collection)
            .get()
            .addOnSuccessListener(querySnapshot -> {
                if (onSuccess != null) onSuccess.onSuccess(querySnapshot.size());
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error getting collection count", e);
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    /**
     * Delete multiple documents by IDs
     */
    public void deleteDocuments(String collection, List<String> documentIds, OnCompleteListener listener) {
        WriteBatch batch = db.batch();

        for (String docId : documentIds) {
            DocumentReference docRef = db.collection(collection).document(docId);
            batch.delete(docRef);
        }

        batch.commit()
            .addOnSuccessListener(aVoid -> {
                Log.d(TAG, "Documents deleted from " + collection);
                if (listener != null) listener.onComplete(true, null);
            })
            .addOnFailureListener(e -> {
                Log.e(TAG, "Error deleting documents", e);
                if (listener != null) listener.onComplete(false, e);
            });
    }

    /**
     * Generate a new document ID
     */
    public String generateDocumentId(String collection) {
        return db.collection(collection).document().getId();
    }

    /**
     * Get statistics for dashboard
     */
    public void getDashboardStats(OnSuccessListener<Map<String, Integer>> onSuccess, OnFailureListener onFailure) {
        Map<String, Integer> stats = new HashMap<>();

        // Get student count
        db.collection(COLLECTION_USERS)
            .whereEqualTo("role", "student")
            .get()
            .addOnSuccessListener(studentSnapshot -> {
                stats.put("studentCount", studentSnapshot.size());

                // Get question count
                db.collection(COLLECTION_QUESTIONS)
                    .get()
                    .addOnSuccessListener(questionSnapshot -> {
                        stats.put("questionCount", questionSnapshot.size());
                        if (onSuccess != null) onSuccess.onSuccess(stats);
                    })
                    .addOnFailureListener(e -> {
                        if (onFailure != null) onFailure.onFailure(e);
                    });
            })
            .addOnFailureListener(e -> {
                if (onFailure != null) onFailure.onFailure(e);
            });
    }

    // ===== FCM TOKEN OPERATIONS =====

```

```

/**
 * Save FCM token for a user
 */
public void saveFCMToken(String userId, String token, OnCompleteListener listener) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("fcmToken", token);
    updates.put("lastTokenUpdate", Timestamp.now());

    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "FCM token saved for user: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error saving FCM token", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Delete FCM token for a user
 */
public void deleteFCMToken(String userId, OnCompleteListener listener) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("fcmToken", FieldValue.delete());

    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "FCM token deleted for user: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error deleting FCM token", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Get all student FCM tokens
 */
public void getAllStudentTokens(OnSuccessListener<List<String>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .whereEqualTo("role", "student")
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<String> tokens = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                String token = doc.getString("fcmToken");
                if (token != null && !token.isEmpty()) {
                    tokens.add(token);
                }
            }
            if (onSuccess != null) onSuccess.onSuccess(tokens);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting student tokens", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

/**
 * Get all teacher FCM tokens
 */
public void getTeacherTokens(OnSuccessListener<List<String>> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .whereEqualTo("role", "teacher")
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<String> tokens = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                String token = doc.getString("fcmToken");
                if (token != null && !token.isEmpty()) {
                    tokens.add(token);
                }
            }
            if (onSuccess != null) onSuccess.onSuccess(tokens);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting teacher tokens", e);
        });
}

```

```

        if (onFailure != null) onFailure.onFailure(e);
    });
}

// ===== NOTIFICATION PREFERENCES OPERATIONS =====

/**
 * Save notification preferences for a user
 */
public void saveNotificationPreferences(String userId, NotificationPreferences prefs, OnCompleteListener listener) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("notificationPreferences", prefs);

    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Notification preferences saved for user: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error saving notification preferences", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Get notification preferences for a user
 */
public void getNotificationPreferences(String userId, OnSuccessListener<NotificationPreferences> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_USERS)
        .document(userId)
        .get()
        .addOnSuccessListener(document -> {
            if (document.exists()) {
                NotificationPreferences prefs = null;
                try {
                    Map<String, Object> prefsMap = (Map<String, Object>) document.get("notificationPreferences");
                    if (prefsMap != null) {
                        prefs = document.toObject(User.class).getNotificationPreferences();
                    }
                } catch (Exception e) {
                    Log.w(TAG, "Error parsing notification preferences", e);
                }
                if (prefs == null) {
                    prefs = NotificationPreferences.createDefault();
                }
                if (onSuccess != null) onSuccess.onSuccess(prefs);
            } else {
                if (onSuccess != null) onSuccess.onSuccess(NotificationPreferences.createDefault());
            }
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting notification preferences", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

// ===== QUIZ SETTINGS OPERATIONS =====

public static final String COLLECTION_SETTINGS = "settings";
public static final String DOC_QUIZ_SETTINGS = "quiz";
public static final int DEFAULT_QUESTION_COUNT = 5;

/**
 * Save quiz question count setting
 */
public void saveQuizQuestionCount(int count, OnCompleteListener listener) {
    Map<String, Object> settings = new HashMap<>();
    settings.put("numberOfQuestions", count);

    db.collection(COLLECTION_SETTINGS)
        .document(DOC_QUIZ_SETTINGS)
        .set(settings)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Quiz question count saved: " + count);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error saving quiz question count", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

```



```

/**
 * Get quiz question count setting
 */
public void getQuizQuestionCount(OnSuccessListener<Integer> onSuccess, OnFailureListener onFailure) {
    db.collection(COLLECTION_SETTINGS)
        .document(DOC_QUIZ_SETTINGS)
        .get()
        .addOnSuccessListener(document -> {
            int count = DEFAULT_QUESTION_COUNT;
            if (document.exists()) {
                Long value = document.getLong("numberOfQuestions");
                if (value != null) {
                    count = value.intValue();
                }
            }
            if (onSuccess != null) onSuccess.onSuccess(count);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting quiz question count", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}

// ===== ACTIVITY TRACKING OPERATIONS =====

/**
 * Update last activity timestamp for a user
 */
public void updateLastActivity(String userId, OnCompleteListener listener) {
    Map<String, Object> updates = new HashMap<>();
    updates.put("lastActivityAt", Timestamp.now());

    db.collection(COLLECTION_USERS)
        .document(userId)
        .update(updates)
        .addOnSuccessListener(aVoid -> {
            Log.d(TAG, "Last activity updated for user: " + userId);
            if (listener != null) listener.onComplete(true, null);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error updating last activity", e);
            if (listener != null) listener.onComplete(false, e);
        });
}

/**
 * Update last activity for current user
 */
public void updateCurrentUserLastActivity(OnCompleteListener listener) {
    String userId = getCurrentUserId();
    if (userId == null) {
        if (listener != null) listener.onComplete(false, new Exception("No authenticated user"));
        return;
    }
    updateLastActivity(userId, listener);
}

/**
 * Get inactive users (no activity in specified days)
 */
public void getInactiveUsers(int daysSinceLastActivity, OnSuccessListener<List<User>> onSuccess, OnFailureListener onFailure) {
    // Calculate the cutoff timestamp
    long cutoffMillis = System.currentTimeMillis() - (daysSinceLastActivity * 24 * 60 * 60 * 1000L);
    Timestamp cutoffTimestamp = new Timestamp(cutoffMillis / 1000, 0);

    db.collection(COLLECTION_USERS)
        .whereLessThan("lastActivityAt", cutoffTimestamp)
        .get()
        .addOnSuccessListener(querySnapshot -> {
            List<User> users = new ArrayList<>();
            for (QueryDocumentSnapshot doc : querySnapshot) {
                users.add(doc.toObject(User.class));
            }
            if (onSuccess != null) onSuccess.onSuccess(users);
        })
        .addOnFailureListener(e -> {
            Log.e(TAG, "Error getting inactive users", e);
            if (onFailure != null) onFailure.onFailure(e);
        });
}
}

```

NotificationHelper.java

app/src/main/java/com/example/phiz/helpers/NotificationHelper.java

```
package com.example.phiz.helpers;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.os.Build;

import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;

import com.example.phiz.R;
import com.example.phiz.activities.GradesActivity;
import com.example.phiz.activities.QuizActivity;
import com.example.phiz.activities.StudentHomeActivity;

/**
 * Singleton helper class for managing notifications.
 * Handles notification channel creation and displaying notifications.
 */
public class NotificationHelper {
    private static final String TAG = "NotificationHelper";

    // Notification Channel IDs
    public static final String CHANNEL_QUIZ = "quiz_notifications";
    public static final String CHANNEL_GRADE = "grade_notifications";
    public static final String CHANNEL_ACHIEVEMENT = "achievement_notifications";
    public static final String CHANNEL_REMINDER = "reminder_notifications";
    public static final String CHANNEL_TEACHER = "teacher_notifications";

    // Notification IDs
    public static final int NOTIFICATION_ID_QUIZ = 1001;
    public static final int NOTIFICATION_ID_GRADE = 1002;
    public static final int NOTIFICATION_ID_ACHIEVEMENT = 1003;
    public static final int NOTIFICATION_ID_REMINDER = 1004;
    public static final int NOTIFICATION_ID_TEACHER = 1005;
    public static final int NOTIFICATION_ID_NEW_CONTENT = 1006;
    public static final int NOTIFICATION_ID_INACTIVITY = 1007;
    public static final int NOTIFICATION_ID_WEEKLY_PROGRESS = 1008;

    // Deep link actions
    public static final String ACTION_OPEN_QUIZ = "com.example.phiz.OPEN_QUIZ";
    public static final String ACTION_OPEN_GRADES = "com.example.phiz.OPEN_GRADES";
    public static final String ACTION_OPEN_HOME = "com.example.phiz.OPEN_HOME";

    // Singleton instance
    private static NotificationHelper instance;
    private final Context context;

    private NotificationHelper(Context context) {
        this.context = context.getApplicationContext();
    }

    public static synchronized NotificationHelper getInstance(Context context) {
        if (instance == null) {
            instance = new NotificationHelper(context);
        }
        return instance;
    }

    /**
     * Create all notification channels. Call this in Application.onCreate()
     */
    public void createNotificationChannels() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationManager notificationManager =
                context.getSystemService(NotificationManager.class);

            // Quiz notifications channel (HIGH importance)
            NotificationChannel quizChannel = new NotificationChannel(
                CHANNEL_QUIZ,
                "Quiz Alerts",
                NotificationManager.IMPORTANCE_HIGH
            );
            quizChannel.setDescription("Notifications about new quizzes and quiz deadlines");
            quizChannel.enableVibration(true);
            notificationManager.createNotificationChannel(quizChannel);

            // Grade notifications channel (HIGH importance)
```

```

NotificationChannel gradeChannel = new NotificationChannel(
    CHANNEL_GRADE,
    "Grade Updates",
    NotificationManager.IMPORTANCE_HIGH
);
gradeChannel.setDescription("Notifications about grades and score updates");
gradeChannel.enableVibration(true);
notificationManager.createNotificationChannel(gradeChannel);

// Achievement notifications channel (DEFAULT importance)
NotificationChannel achievementChannel = new NotificationChannel(
    CHANNEL_ACHIEVEMENT,
    "Achievements",
    NotificationManager.IMPORTANCE_DEFAULT
);
achievementChannel.setDescription("Notifications about achievements and milestones");
notificationManager.createNotificationChannel(achievementChannel);

// Reminder notifications channel (DEFAULT importance)
NotificationChannel reminderChannel = new NotificationChannel(
    CHANNEL_REMINDER,
    "Study Reminders",
    NotificationManager.IMPORTANCE_DEFAULT
);
reminderChannel.setDescription("Daily study reminders and inactivity notifications");
notificationManager.createNotificationChannel(reminderChannel);

// Teacher notifications channel (HIGH importance)
NotificationChannel teacherChannel = new NotificationChannel(
    CHANNEL_TEACHER,
    "Teacher Alerts",
    NotificationManager.IMPORTANCE_HIGH
);
teacherChannel.setDescription("Notifications for teachers about student activity");
teacherChannel.enableVibration(true);
notificationManager.createNotificationChannel(teacherChannel);
}

// ===== STUDENT NOTIFICATIONS =====

/**
 * Show notification when a new quiz is available
 */
public void showNewQuizNotification(String quizTitle) {
    Intent intent = new Intent(context, QuizActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_QUIZ)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("New Quiz Available!")
        .setContentText(quizTitle != null ? quizTitle : "A new physics quiz is ready for you")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_QUIZ, builder);
}

/**
 * Show notification when grade is posted (after quiz completion)
 */
public void showGradePostedNotification(int score, int totalQuestions) {
    Intent intent = new Intent(context, GradesActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    String message = "You scored " + score + " points!";
    if (score >= totalQuestions * 20) { // Perfect score check
        message += " Perfect score!";
    }

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_GRADE)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Quiz Complete!")
        .setContentText(message)
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_GRADE, builder);
}

```

```
}
```

```
/**
```

```
 * Show achievement unlocked notification
```

```
 */
```

```
public void showAchievementNotification(String achievementTitle, String achievementDescription) {
    Intent intent = new Intent(context, StudentHomeActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ACHIEVEMENT)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Achievement Unlocked!")
        .setContentText(achievementTitle)
        .setStyle(new NotificationCompat.BigTextStyle()
            .bigText(achievementTitle + "\n" + achievementDescription))
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_ACHIEVEMENT, builder);
}
```

```
/**
```

```
 * Show daily study reminder notification
```

```
 */
```

```
public void showStudyReminderNotification() {
    Intent intent = new Intent(context, QuizActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_REMINDER)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Time to Study!")
        .setContentText("Take a quick physics quiz to keep your skills sharp")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_REMINDER, builder);
}
```

```
/**
```

```
 * Show inactivity reminder notification
```

```
 */
```

```
public void showInactivityReminderNotification(int daysSinceLastActivity) {
    Intent intent = new Intent(context, StudentHomeActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_REMINDER)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("We Miss You!")
        .setContentText("It's been " + daysSinceLastActivity + " days since your last visit. Come back and learn some physics")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_INACTIVITY, builder);
}
```

```
/**
```

```
 * Show weekly progress notification
```

```
 */
```

```
public void showWeeklyProgressNotification(int scoreImprovement, int currentScore) {
    Intent intent = new Intent(context, GradesActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    String message;
    if (scoreImprovement > 0) {
        message = "You improved by " + scoreImprovement + " points this week! Current score: " + currentScore;
    } else {
        message = "Your current score is " + currentScore + " points. Keep practicing!";
    }

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_GRADE)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Weekly Progress Report")
        .setContentText(message)
}
```

```

        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_WEEKLY_PROGRESS, builder);
}

/**
 * Show new content available notification
 */
public void showNewContentNotification(String contentType) {
    Intent intent = new Intent(context, QuizActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_QUIZ)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("New Content Available!")
        .setContentText("Your teacher added new " + contentType + ". Check it out!")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_NEW_CONTENT, builder);
}

// ===== TEACHER NOTIFICATIONS =====

/**
 * Show notification to teacher when a student completes a quiz
 */
public void showQuizCompletionNotificationForTeacher(String studentName, int score) {
    Intent intent = new Intent(context, GradesActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_TEACHER)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Quiz Completed")
        .setContentText(studentName + " completed a quiz with " + score + " points")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_TEACHER, builder);
}

/**
 * Show low score alert notification to teacher
 */
public void showLowScoreAlertForTeacher(String studentName, int score, int percentage) {
    Intent intent = new Intent(context, GradesActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

    NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_TEACHER)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Low Score Alert")
        .setContentText(studentName + " scored " + percentage + "% on their quiz. They may need help.")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setContentIntent(pendingIntent)
        .setAutoCancel(true);

    showNotification(NOTIFICATION_ID_TEACHER + 1, builder);
}

// ===== HELPER METHODS =====

/**
 * Show a notification using NotificationManagerCompat
 */
private void showNotification(int notificationId, NotificationCompat.Builder builder) {
    try {
        NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
        notificationManager.notify(notificationId, builder.build());
    } catch (SecurityException e) {
        // Permission not granted - ignore
    }
}

/**

```

```

    * Cancel a specific notification
    */
    public void cancelNotification(int notificationId) {
        NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
        notificationManager.cancel(notificationId);
    }

    /**
    * Cancel all notifications
    */
    public void cancelAllNotifications() {
        NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
        notificationManager.cancelAll();
    }

    /**
    * Check if notifications are enabled for the app
    */
    public boolean areNotificationsEnabled() {
        NotificationManagerCompat notificationManager = NotificationManagerCompat.from(context);
        return notificationManager.areNotificationsEnabled();
    }

    /**
    * Check if a specific channel is enabled (Android O+)
    */
    public boolean isChannelEnabled(String channelId) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationManager notificationManager =
                context.getSystemService(NotificationManager.class);
            NotificationChannel channel = notificationManager.getNotificationChannel(channelId);
            return channel != null && channel.getImportance() != NotificationManager.IMPORTANCE_NONE;
        }
        return true;
    }
}

```

WorkerScheduler.java

app/src/main/java/com/example/phiz/helpers/WorkerScheduler.java

```
package com.example.phiz.helpers;

import android.content.Context;
import android.util.Log;

import androidx.work.Constraints;
import androidx.work.ExistingPeriodicWorkPolicy;
import androidx.work.NetworkType;
import androidx.work.PeriodicWorkRequest;
import androidx.work.WorkManager;

import com.example.phiz.models.NotificationPreferences;
import com.example.phiz.workers.InactivityCheckWorker;
import com.example.phiz.workers.StudyReminderWorker;
import com.example.phiz.workers.WeeklyProgressWorker;

import java.util.Calendar;
import java.util.concurrent.TimeUnit;

/**
 * Helper class for scheduling WorkManager workers for notifications.
 */
public class WorkerScheduler {
    private static final String TAG = "WorkerScheduler";

    /**
     * Schedule all notification workers based on user preferences.
     */
    public static void scheduleAllWorkers(Context context, NotificationPreferences prefs) {
        if (prefs == null) {
            prefs = NotificationPreferences.createDefault();
        }

        if (prefs.isStudyReminders()) {
            scheduleStudyReminder(context, prefs);
        } else {
            cancelStudyReminder(context);
        }

        // Always schedule inactivity check
        scheduleInactivityCheck(context);

        if (prefs.isWeeklyProgress()) {
            scheduleWeeklyProgress(context);
        } else {
            cancelWeeklyProgress(context);
        }

        Log.d(TAG, "All workers scheduled");
    }

    /**
     * Schedule daily study reminder at user's preferred time.
     */
    public static void scheduleStudyReminder(Context context, NotificationPreferences prefs) {
        // Calculate initial delay to the next reminder time
        long initialDelay = calculateInitialDelay(prefs.getReminderHour(), prefs.getReminderMinute());

        PeriodicWorkRequest reminderRequest = new PeriodicWorkRequest.Builder(
            StudyReminderWorker.class,
            1, TimeUnit.DAYS)
            .setInitialDelay(initialDelay, TimeUnit.MILLISECONDS)
            .addTag(StudyReminderWorker.WORK_NAME)
            .build();

        WorkManager.getInstance(context)
            .enqueueUniquePeriodicWork(
                StudyReminderWorker.WORK_NAME,
                ExistingPeriodicWorkPolicy.UPDATE,
                reminderRequest
            );

        Log.d(TAG, "Study reminder scheduled with initial delay: " + initialDelay + "ms");
    }

    /**
     * Cancel study reminder worker.
     */
    public static void cancelStudyReminder(Context context) {
        WorkManager.getInstance(context).cancelUniqueWork(StudyReminderWorker.WORK_NAME);
    }
}
```

```

        Log.d(TAG, "Study reminder cancelled");
    }

    /**
     * Schedule inactivity check worker to run daily.
     */
    public static void scheduleInactivityCheck(Context context) {
        PeriodicWorkRequest inactivityRequest = new PeriodicWorkRequest.Builder(
            InactivityCheckWorker.class,
            1, TimeUnit.DAYS)
            .setInitialDelay(1, TimeUnit.HOURS) // Start after 1 hour
            .addTag(InactivityCheckWorker.WORK_NAME)
            .build();

        WorkManager.getInstance(context)
            .enqueueUniquePeriodicWork(
                InactivityCheckWorker.WORK_NAME,
                ExistingPeriodicWorkPolicy.KEEP,
                inactivityRequest
            );

        Log.d(TAG, "Inactivity check scheduled");
    }

    /**
     * Cancel inactivity check worker.
     */
    public static void cancelInactivityCheck(Context context) {
        WorkManager.getInstance(context).cancelUniqueWork(InactivityCheckWorker.WORK_NAME);
        Log.d(TAG, "Inactivity check cancelled");
    }

    /**
     * Schedule weekly progress worker to run once a week.
     */
    public static void scheduleWeeklyProgress(Context context) {
        Constraints constraints = new Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
            .build();

        PeriodicWorkRequest progressRequest = new PeriodicWorkRequest.Builder(
            WeeklyProgressWorker.class,
            7, TimeUnit.DAYS)
            .setConstraints(constraints)
            .setInitialDelay(calculateDelayToSunday(), TimeUnit.MILLISECONDS)
            .addTag(WeeklyProgressWorker.WORK_NAME)
            .build();

        WorkManager.getInstance(context)
            .enqueueUniquePeriodicWork(
                WeeklyProgressWorker.WORK_NAME,
                ExistingPeriodicWorkPolicy.KEEP,
                progressRequest
            );

        Log.d(TAG, "Weekly progress scheduled");
    }

    /**
     * Cancel weekly progress worker.
     */
    public static void cancelWeeklyProgress(Context context) {
        WorkManager.getInstance(context).cancelUniqueWork(WeeklyProgressWorker.WORK_NAME);
        Log.d(TAG, "Weekly progress cancelled");
    }

    /**
     * Cancel all notification workers.
     */
    public static void cancelAllWorkers(Context context) {
        WorkManager workManager = WorkManager.getInstance(context);
        workManager.cancelUniqueWork(StudyReminderWorker.WORK_NAME);
        workManager.cancelUniqueWork(InactivityCheckWorker.WORK_NAME);
        workManager.cancelUniqueWork(WeeklyProgressWorker.WORK_NAME);
        Log.d(TAG, "All workers cancelled");
    }

    /**
     * Calculate the initial delay until the specified time.
     */
    private static long calculateInitialDelay(int targetHour, int targetMinute) {
        Calendar now = Calendar.getInstance();
        Calendar target = Calendar.getInstance();
    }

```



```

        target.set(Calendar.HOUR_OF_DAY, targetHour);
        target.set(Calendar.MINUTE, targetMinute);
        target.set(Calendar.SECOND, 0);
        target.set(Calendar.MILLISECOND, 0);

        // If the target time has passed today, schedule for tomorrow
        if (target.before(now)) {
            target.add(Calendar.DAY_OF_MONTH, 1);
        }

        return target.getTimeInMillis() - now.getTimeInMillis();
    }

    /**
     * Calculate delay until next Sunday at 10:00 AM.
     */
    private static long calculateDelayToSunday() {
        Calendar now = Calendar.getInstance();
        Calendar target = Calendar.getInstance();

        target.set(Calendar.DAY_OF_WEEK, Calendar.SUNDAY);
        target.set(Calendar.HOUR_OF_DAY, 10);
        target.set(Calendar.MINUTE, 0);
        target.set(Calendar.SECOND, 0);
        target.set(Calendar.MILLISECOND, 0);

        // If it's already past Sunday 10 AM this week, go to next week
        if (target.before(now)) {
            target.add(Calendar.WEEK_OF_YEAR, 1);
        }

        return target.getTimeInMillis() - now.getTimeInMillis();
    }
}

```

NotificationPreferences.java

app/src/main/java/com/example/phiz/models/NotificationPreferences.java

```
package com.example.phiz.models;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * Model class for user notification preferences.
 * Stores settings for different notification types and reminder schedules.
 */
public class NotificationPreferences {
    private boolean quizNotifications;
    private boolean gradeNotifications;
    private boolean achievementNotifications;
    private boolean studyReminders;
    private boolean weeklyProgress;
    private String reminderTime; // Format: "HH:mm" (e.g., "18:00")
    private List<String> reminderDays; // Days of week: "MONDAY", "TUESDAY", etc.

    // Default constructor for Firestore
    public NotificationPreferences() {
        // Set defaults - all enabled
        this.quizNotifications = true;
        this.gradeNotifications = true;
        this.achievementNotifications = true;
        this.studyReminders = true;
        this.weeklyProgress = true;
        this.reminderTime = "18:00";
        this.reminderDays = new ArrayList<>(Arrays.asList(
            "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY"
        ));
    }

    /**
     * Create preferences with all notifications enabled (default)
     */
    public static NotificationPreferences createDefault() {
        return new NotificationPreferences();
    }

    /**
     * Create preferences with all notifications disabled
     */
    public static NotificationPreferences createDisabled() {
        NotificationPreferences prefs = new NotificationPreferences();
        prefs.setQuizNotifications(false);
        prefs.setGradeNotifications(false);
        prefs.setAchievementNotifications(false);
        prefs.setStudyReminders(false);
        prefs.setWeeklyProgress(false);
        return prefs;
    }

    // Getters and setters
    public boolean isQuizNotifications() {
        return quizNotifications;
    }

    public void setQuizNotifications(boolean quizNotifications) {
        this.quizNotifications = quizNotifications;
    }

    public boolean isGradeNotifications() {
        return gradeNotifications;
    }

    public void setGradeNotifications(boolean gradeNotifications) {
        this.gradeNotifications = gradeNotifications;
    }

    public boolean isAchievementNotifications() {
        return achievementNotifications;
    }

    public void setAchievementNotifications(boolean achievementNotifications) {
        this.achievementNotifications = achievementNotifications;
    }

    public boolean isStudyReminders() {
        return studyReminders;
    }
}
```

```

}

public void setStudyReminders(boolean studyReminders) {
    this.studyReminders = studyReminders;
}

public boolean isWeeklyProgress() {
    return weeklyProgress;
}

public void setWeeklyProgress(boolean weeklyProgress) {
    this.weeklyProgress = weeklyProgress;
}

public String getReminderTime() {
    return reminderTime;
}

public void setReminderTime(String reminderTime) {
    this.reminderTime = reminderTime;
}

public List<String> getReminderDays() {
    return reminderDays;
}

public void setReminderDays(List<String> reminderDays) {
    this.reminderDays = reminderDays;
}

/**
 * Get reminder hour from reminderTime
 */
public int getReminderHour() {
    if (reminderTime == null || !reminderTime.contains(":")) {
        return 18; // Default to 6 PM
    }
    try {
        return Integer.parseInt(reminderTime.split(":")[0]);
    } catch (NumberFormatException e) {
        return 18;
    }
}

/**
 * Get reminder minute from reminderTime
 */
public int getReminderMinute() {
    if (reminderTime == null || !reminderTime.contains(":")) {
        return 0;
    }
    try {
        return Integer.parseInt(reminderTime.split(":")[1]);
    } catch (NumberFormatException e) {
        return 0;
    }
}

/**
 * Check if reminders are enabled for a specific day
 */
public boolean isReminderEnabledForDay(String dayOfWeek) {
    if (reminderDays == null) {
        return false;
    }
    return reminderDays.contains(dayOfWeek.toUpperCase());
}

/**
 * Check if any notification type is enabled
 */
public boolean hasAnyNotificationEnabled() {
    return quizNotifications || gradeNotifications || achievementNotifications
        || studyReminders || weeklyProgress;
}
}

```

Question.java

app/src/main/java/com/example/phiz/models/Question.java

```
package com.example.phiz.models;

import java.util.List;

/**
 * Represents a single quiz/test question.
 * This model is designed to be stored in and retrieved from Firestore.
 */
public class Question {
    private String questionId;
    private String questionText;
    private List<String> options; // Using List for Firestore compatibility
    private int correctAnswerIndex; // 0-based index of correct answer
    private String explanation; // Optional explanation for the correct answer
    private int pointValue; // Points awarded for correct answer
    private String difficulty; // "easy", "medium", "hard"

    /**
     * Default constructor required for Firestore deserialization
     */
    public Question() {
    }

    /**
     * Constructor for creating a new question
     */
    public Question(String questionId, String questionText, List<String> options,
                    int correctAnswerIndex, String explanation, int pointValue, String difficulty) {
        this.questionId = questionId;
        this.questionText = questionText;
        this.options = options;
        this.correctAnswerIndex = correctAnswerIndex;
        this.explanation = explanation;
        this.pointValue = pointValue;
        this.difficulty = difficulty;
    }

    /**
     * Simplified constructor with default values
     */
    public Question(String questionId, String questionText, List<String> options, int correctAnswerIndex) {
        this.questionId = questionId;
        this.questionText = questionText;
        this.options = options;
        this.correctAnswerIndex = correctAnswerIndex;
        this.explanation = "";
        this.pointValue = 20; // Default point value
        this.difficulty = "medium";
    }

    // Getters and setters
    public String getQuestionId() {
        return questionId;
    }

    public void setQuestionId(String questionId) {
        this.questionId = questionId;
    }

    public String getQuestionText() {
        return questionText;
    }

    public void setQuestionText(String questionText) {
        this.questionText = questionText;
    }

    public List<String> getOptions() {
        return options;
    }

    public void setOptions(List<String> options) {
        this.options = options;
    }

    public int getCorrectAnswerIndex() {
        return correctAnswerIndex;
    }

    public void setCorrectAnswerIndex(int correctAnswerIndex) {
    }
}
```

```

        this.correctAnswerIndex = correctAnswerIndex;
    }

    public String getExplanation() {
        return explanation;
    }

    public void setExplanation(String explanation) {
        this.explanation = explanation;
    }

    public int getPointValue() {
        return pointValue;
    }

    public void setPointValue(int pointValue) {
        this.pointValue = pointValue;
    }

    public String getDifficulty() {
        return difficulty;
    }

    public void setDifficulty(String difficulty) {
        this.difficulty = difficulty;
    }

    /**
     * Checks if a given answer index is correct
     */
    public boolean isCorrect(int answerIndex) {
        return answerIndex == correctAnswerIndex;
    }

    /**
     * Gets the correct answer text
     */
    public String getCorrectAnswer() {
        if (options != null && correctAnswerIndex >= 0 && correctAnswerIndex < options.size()) {
            return options.get(correctAnswerIndex);
        }
        return null;
    }
}

```

QuizResult.java

app/src/main/java/com/example/phiz/models/QuizResult.java

```
package com.example.phiz.models;

import com.google.firebase.Timestamp;

public class QuizResult {
    private String quizId;
    private String userId;
    private String quizName;
    private int score;
    private int totalQuestions;
    private Timestamp timestamp;

    public QuizResult() {
        // Default constructor required for Firestore
    }

    public QuizResult(String quizId, String userId, String quizName, int score, int totalQuestions) {
        this.quizId = quizId;
        this.userId = userId;
        this.quizName = quizName;
        this.score = score;
        this.totalQuestions = totalQuestions;
        this.timestamp = Timestamp.now();
    }

    // Getters and setters
    public String getQuizId() {
        return quizId;
    }

    public void setQuizId(String quizId) {
        this.quizId = quizId;
    }

    public String getUserId() {
        return userId;
    }

    public void setUserId(String userId) {
        this.userId = userId;
    }

    public String getQuizName() {
        return quizName;
    }

    public void setQuizName(String quizName) {
        this.quizName = quizName;
    }

    public int getScore() {
        return score;
    }

    public void setScore(int score) {
        this.score = score;
    }

    public int getTotalQuestions() {
        return totalQuestions;
    }

    public void setTotalQuestions(int totalQuestions) {
        this.totalQuestions = totalQuestions;
    }

    public Timestamp getTimestamp() {
        return timestamp;
    }

    public void setTimestamp(Timestamp timestamp) {
        this.timestamp = timestamp;
    }
}
```

Test.java

app/src/main/java/com/example/phiz/models/Test.java

```
package com.example.phiz.models;

import com.google.firebase.Timestamp;
import java.util.List;

/**
 * Represents a complete test/quiz with metadata and questions.
 * This model is designed to be stored in Firestore's "tests" collection.
 */
public class Test {
    private String testId;
    private String testName;
    private String description;
    private String subject; // e.g., "Newton's Laws", "Kinematics", "Energy"
    private List<Question> questions;
    private int totalPoints;
    private int passingScore; // Minimum score to pass
    private int timeLimit; // Time limit in minutes (0 = no limit)
    private String difficulty; // "easy", "medium", "hard"
    private boolean isActive; // Whether the test is currently available
    private String createdBy; // Teacher/creator user ID
    private Timestamp createdAt;
    private Timestamp updatedAt;

    /**
     * Default constructor required for Firestore deserialization
     */
    public Test() {
    }

    /**
     * Full constructor for creating a new test
     */
    public Test(String testId, String testName, String description, String subject,
        List<Question> questions, int totalPoints, int passingScore,
        int timeLimit, String difficulty, boolean isActive, String createdBy) {
        this.testId = testId;
        this.testName = testName;
        this.description = description;
        this.subject = subject;
        this.questions = questions;
        this.totalPoints = totalPoints;
        this.passingScore = passingScore;
        this.timeLimit = timeLimit;
        this.difficulty = difficulty;
        this.isActive = isActive;
        this.createdBy = createdBy;
        this.createdAt = Timestamp.now();
        this.updatedAt = Timestamp.now();
    }

    /**
     * Simplified constructor with default values
     */
    public Test(String testId, String testName, String subject, List<Question> questions, String createdBy) {
        this.testId = testId;
        this.testName = testName;
        this.description = "";
        this.subject = subject;
        this.questions = questions;
        this.totalPoints = calculateTotalPoints();
        this.passingScore = (int) (totalPoints * 0.6); // 60% passing by default
        this.timeLimit = 0; // No time limit by default
        this.difficulty = "medium";
        this.isActive = true;
        this.createdBy = createdBy;
        this.createdAt = Timestamp.now();
        this.updatedAt = Timestamp.now();
    }

    // Getters and setters
    public String getTestId() {
        return testId;
    }

    public void setTestId(String testId) {
        this.testId = testId;
    }

    public String getTestName() {
```

```

        return testName;
    }

    public void setTestName(String testName) {
        this.testName = testName;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getSubject() {
        return subject;
    }

    public void setSubject(String subject) {
        this.subject = subject;
    }

    public List<Question> getQuestions() {
        return questions;
    }

    public void setQuestions(List<Question> questions) {
        this.questions = questions;
    }

    public int getTotalPoints() {
        return totalPoints;
    }

    public void setTotalPoints(int totalPoints) {
        this.totalPoints = totalPoints;
    }

    public int getPassingScore() {
        return passingScore;
    }

    public void setPassingScore(int passingScore) {
        this.passingScore = passingScore;
    }

    public int getTimeLimit() {
        return timeLimit;
    }

    public void setTimeLimit(int timeLimit) {
        this.timeLimit = timeLimit;
    }

    public String getDifficulty() {
        return difficulty;
    }

    public void setDifficulty(String difficulty) {
        this.difficulty = difficulty;
    }

    public boolean isActive() {
        return isActive;
    }

    public void setActive(boolean active) {
        isActive = active;
    }

    public String getCreatedBy() {
        return createdBy;
    }

    public void setCreatedBy(String createdBy) {
        this.createdBy = createdBy;
    }

    public Timestamp getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(Timestamp createdAt) {

```



```

        this.createdAt = createdAt;
    }

    public Timestamp getUpdatedAt() {
        return updatedAt;
    }

    public void setUpdatedAt(Timestamp updatedAt) {
        this.updatedAt = updatedAt;
    }

    // Helper methods

    /**
     * Calculates total points based on all questions
     */
    public int calculateTotalPoints() {
        if (questions == null || questions.isEmpty()) {
            return 0;
        }
        int total = 0;
        for (Question q : questions) {
            total += q.getPointValue();
        }
        return total;
    }

    /**
     * Gets the number of questions in the test
     */
    public int getQuestionCount() {
        return questions != null ? questions.size() : 0;
    }

    /**
     * Checks if a given score is passing
     */
    public boolean isPassing(int score) {
        return score >= passingScore;
    }

    /**
     * Gets a specific question by index
     */
    public Question getQuestion(int index) {
        if (questions != null && index >= 0 && index < questions.size()) {
            return questions.get(index);
        }
        return null;
    }

    /**
     * Updates the updatedAt timestamp
     */
    public void updateTimestamp() {
        this.updatedAt = Timestamp.now();
    }
}

```

User.java

app/src/main/java/com/example/phiz/models/User.java

```
package com.example.phiz.models;

import com.google.firebase.Timestamp;

public class User {
    private String uid;
    private String email;
    private String name;
    private String role; // "student" or "teacher"
    private int totalScore;
    private int testsCompleted;

    // Notification-related fields
    private String fcmToken;
    private Timestamp lastActivityAt;
    private NotificationPreferences notificationPreferences;

    public User() {
        // Default constructor required for Firestore
    }

    public User(String uid, String email, String name, String role) {
        this.uid = uid;
        this.email = email;
        this.name = name;
        this.role = role;
        this.totalScore = 0;
        this.testsCompleted = 0;
        this.notificationPreferences = NotificationPreferences.createDefault();
    }

    // Getters and setters
    public String getUid() {
        return uid;
    }

    public void setUid(String uid) {
        this.uid = uid;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }

    public int getTotalScore() {
        return totalScore;
    }

    public void setTotalScore(int totalScore) {
        this.totalScore = totalScore;
    }

    public int getTestsCompleted() {
        return testsCompleted;
    }

    public void setTestsCompleted(int testsCompleted) {
        this.testsCompleted = testsCompleted;
    }
}
```

```

// Get user level based on total score
public int getLevel() {
    if (totalScore >= 1000) return 5;
    if (totalScore >= 500) return 4;
    if (totalScore >= 250) return 3;
    if (totalScore >= 100) return 2;
    return 1;
}

// Get level title based on score
public String getLevelTitle() {
    int level = getLevel();
    switch (level) {
        case 5: return "Physics Master";
        case 4: return "Expert";
        case 3: return "Advanced";
        case 2: return "Intermediate";
        default: return "Beginner";
    }
}

// FCM Token
public String getFcmToken() {
    return fcmToken;
}

public void setFcmToken(String fcmToken) {
    this.fcmToken = fcmToken;
}

// Last Activity
public Timestamp getLastActivityAt() {
    return lastActivityAt;
}

public void setLastActivityAt(Timestamp lastActivityAt) {
    this.lastActivityAt = lastActivityAt;
}

// Notification Preferences
public NotificationPreferences getNotificationPreferences() {
    return notificationPreferences;
}

public void setNotificationPreferences(NotificationPreferences notificationPreferences) {
    this.notificationPreferences = notificationPreferences;
}
}

```

PhizApplication.java

app/src/main/java/com/example/phiz/PhizApplication.java

```
package com.example.phiz;

import android.app.Application;

import androidx.work.Configuration;
import androidx.work.WorkManager;

import com.example.phiz.helpers.NotificationHelper;
import com.google.firebase.FirebaseApp;

public class PhizApplication extends Application {
    @Override
    public void onCreate() {
        super.onCreate();

        // Initialize Firebase
        FirebaseApp.initializeApp(this);

        // Create notification channels
        NotificationHelper.getInstance(this).createNotificationChannels();

        // Initialize WorkManager with default configuration
        if (!isWorkManagerInitialized()) {
            WorkManager.initialize(this, new Configuration.Builder().build());
        }
    }

    private boolean isWorkManagerInitialized() {
        try {
            WorkManager.getInstance(this);
            return true;
        } catch (IllegalStateException e) {
            return false;
        }
    }
}
```

BootReceiver.java

app/src/main/java/com/example/phiz/receivers/BootReceiver.java

```
package com.example.phiz.receivers;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.util.Log;

import com.example.phiz.helpers.WorkerScheduler;
import com.example.phiz.models.NotificationPreferences;

/**
 * Broadcast receiver that reschedules notification workers after device reboot.
 */
public class BootReceiver extends BroadcastReceiver {
    private static final String TAG = "BootReceiver";

    @Override
    public void onReceive(Context context, Intent intent) {
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())) {
            Log.d(TAG, "Device boot completed, rescheduling notification workers");

            // Reschedule workers with default preferences
            // The actual preferences will be loaded when the user opens the app
            WorkerScheduler.scheduleAllWorkers(context, NotificationPreferences.createDefault());
        }
    }
}
```

NotificationActionReceiver.java

app/src/main/java/com/example/phiz/receivers/NotificationActionReceiver.java

```
package com.example.phiz.receivers;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.util.Log;

import com.example.phiz.activities.GradesActivity;
import com.example.phiz.activities.QuizActivity;
import com.example.phiz.activities.StudentHomeActivity;
import com.example.phiz.helpers.NotificationHelper;

/**
 * Broadcast receiver for handling notification button actions.
 */
public class NotificationActionReceiver extends BroadcastReceiver {
    private static final String TAG = "NotificationActionReceiver";

    public static final String ACTION_OPEN QUIZ = "com.example.phiz.action.OPEN QUIZ";
    public static final String ACTION_OPEN GRADES = "com.example.phiz.action.OPEN GRADES";
    public static final String ACTION_OPEN HOME = "com.example.phiz.action.OPEN HOME";
    public static final String ACTION_DISMISS = "com.example.phiz.action.DISMISS";

    public static final String EXTRA_NOTIFICATION_ID = "notification_id";

    @Override
    public void onReceive(Context context, Intent intent) {
        String action = intent.getAction();
        int notificationId = intent.getIntExtra(EXTRA_NOTIFICATION_ID, -1);

        Log.d(TAG, "Received action: " + action + ", notification ID: " + notificationId);

        if (action == null) {
            return;
        }

        // Cancel the notification if we have an ID
        if (notificationId != -1) {
            NotificationHelper.getInstance(context).cancelNotification(notificationId);
        }

        switch (action) {
            case ACTION_OPEN QUIZ:
                openActivity(context, QuizActivity.class);
                break;

            case ACTION_OPEN GRADES:
                openActivity(context, GradesActivity.class);
                break;

            case ACTION_OPEN HOME:
                openActivity(context, StudentHomeActivity.class);
                break;

            case ACTION_DISMISS:
                // Just dismiss the notification (already cancelled above)
                Log.d(TAG, "Notification dismissed");
                break;

            default:
                Log.w(TAG, "Unknown action: " + action);
        }
    }

    private void openActivity(Context context, Class<?> activityClass) {
        Intent intent = new Intent(context, activityClass);
        intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TOP);
        context.startActivity(intent);
    }
}
```

PhizFirebaseMessagingService.java

app/src/main/java/com/example/phiz/services/PhizFirebaseMessagingService.java

```
package com.example.phiz.services;

import android.util.Log;

import androidx.annotation.NonNull;

import com.example.phiz.helpers.FCMTokenManager;
import com.example.phiz.helpers.NotificationHelper;
import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;

import java.util.Map;

/**
 * Firebase Cloud Messaging service for handling push notifications.
 * Handles both token refresh events and incoming messages.
 */
public class PhizFirebaseMessagingService extends FirebaseMessagingService {
    private static final String TAG = "PhizFCMService";

    // Notification types from FCM data payload
    public static final String TYPE_NEW_QUIZ = "new_quiz";
    public static final String TYPE_QUIZ_DEADLINE = "quiz_deadline";
    public static final String TYPE_NEW_CONTENT = "new_content";
    public static final String TYPE_QUIZ_COMPLETED = "quiz_completed";
    public static final String TYPE_LOW_SCORE_ALERT = "low_score_alert";
    public static final String TYPE_CLASS_PROGRESS = "class_progress";

    @Override
    public void onNewToken(@NonNull String token) {
        super.onNewToken(token);
        Log.d(TAG, "New FCM token: " + token);

        // Save the new token to Firestore
        FCMTokenManager.getInstance(this).saveTokenToFirestore(token);
    }

    @Override
    public void onMessageReceived(@NonNull RemoteMessage remoteMessage) {
        super.onMessageReceived(remoteMessage);
        Log.d(TAG, "FCM message received from: " + remoteMessage.getFrom());

        // Check if message contains a notification payload (shown automatically when app in background)
        if (remoteMessage.getNotification() != null) {
            Log.d(TAG, "Notification payload: " + remoteMessage.getNotification().getBody());
            handleNotificationPayload(remoteMessage.getNotification());
        }

        // Check if message contains a data payload (always handled by us)
        if (remoteMessage.getData().size() > 0) {
            Log.d(TAG, "Data payload: " + remoteMessage.getData());
            handleDataPayload(remoteMessage.getData());
        }
    }

    /**
     * Handle notification payload from FCM.
     * This is typically used for simple notifications that show automatically.
     */
    private void handleNotificationPayload(RemoteMessage.Notification notification) {
        String title = notification.getTitle();
        String body = notification.getBody();

        // Show notification using our helper for consistent styling
        NotificationHelper helper = NotificationHelper.getInstance(this);

        // Default to quiz channel if we can't determine type
        NotificationHelper.getInstance(this).showNewQuizNotification(body);
    }

    /**
     * Handle data payload from FCM.
     * This allows for more complex notification logic.
     */
    private void handleDataPayload(Map<String, String> data) {
        String type = data.get("type");
        NotificationHelper helper = NotificationHelper.getInstance(this);

        if (type == null) {
            Log.w(TAG, "No notification type in data payload");
        }
    }
}
```

```

        return;
    }

    switch (type) {
        case TYPE_NEW_QUIZ:
            handleNewQuizNotification(data, helper);
            break;

        case TYPE_QUIZ_DEADLINE:
            handleQuizDeadlineNotification(data, helper);
            break;

        case TYPE_NEW_CONTENT:
            handleNewContentNotification(data, helper);
            break;

        case TYPE_QUIZ_COMPLETED:
            handleQuizCompletedNotification(data, helper);
            break;

        case TYPE_LOW_SCORE_ALERT:
            handleLowScoreAlertNotification(data, helper);
            break;

        case TYPE_CLASS_PROGRESS:
            handleClassProgressNotification(data, helper);
            break;

        default:
            Log.w(TAG, "Unknown notification type: " + type);
    }
}

private void handleNewQuizNotification(Map<String, String> data, NotificationHelper helper) {
    String quizTitle = data.get("quiz_title");
    helper.showNewQuizNotification(quizTitle != null ? quizTitle : "New Physics Quiz");
}

private void handleQuizDeadlineNotification(Map<String, String> data, NotificationHelper helper) {
    String quizTitle = data.get("quiz_title");
    String deadline = data.get("deadline");
    // For now, reuse the new quiz notification
    helper.showNewQuizNotification("Deadline approaching: " + (quizTitle != null ? quizTitle : "Quiz"));
}

private void handleNewContentNotification(Map<String, String> data, NotificationHelper helper) {
    String contentType = data.get("content_type");
    helper.showNewContentNotification(contentType != null ? contentType : "questions");
}

private void handleQuizCompletedNotification(Map<String, String> data, NotificationHelper helper) {
    String studentName = data.get("student_name");
    String scoreStr = data.get("score");
    int score = 0;
    try {
        score = Integer.parseInt(scoreStr != null ? scoreStr : "0");
    } catch (NumberFormatException e) {
        Log.e(TAG, "Invalid score value: " + scoreStr);
    }
    helper.showQuizCompletionNotificationForTeacher(
        studentName != null ? studentName : "A student",
        score
    );
}

private void handleLowScoreAlertNotification(Map<String, String> data, NotificationHelper helper) {
    String studentName = data.get("student_name");
    String scoreStr = data.get("score");
    String percentageStr = data.get("percentage");
    int score = 0;
    int percentage = 0;
    try {
        score = Integer.parseInt(scoreStr != null ? scoreStr : "0");
        percentage = Integer.parseInt(percentageStr != null ? percentageStr : "0");
    } catch (NumberFormatException e) {
        Log.e(TAG, "Invalid score/percentage value");
    }
    helper.showLowScoreAlertForTeacher(
        studentName != null ? studentName : "A student",
        score,
        percentage
    );
}
}

```



```
private void handleClassProgressNotification(Map<String, String> data, NotificationHelper helper) {  
    // This would typically be a weekly summary for teachers  
    // For now, we'll just show a simple notification  
    String summary = data.get("summary");  
    if (summary != null) {  
        helper.showQuizCompletionNotificationForTeacher("Class Progress", 0);  
    }  
}
```

DoodleView.java

app/src/main/java/com/example/phiz/views/DoodleView.java

```
package com.example.phiz.views;

import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.graphics.Path;
import android.graphics.PixelFormat;
import android.graphics.PorterDuff;
import android.util.AttributeSet;
import android.view.MotionEvent;
import android.view.SurfaceHolder;
import android.view.SurfaceView;

import java.util.ArrayList;
import java.util.List;

public class DoodleView extends SurfaceView implements SurfaceHolder.Callback {
    private Paint paint;
    private List<Path> paths;
    private List<Paint> paints;
    private Path currentPath;
    private boolean isDrawingEnabled = false;
    private SurfaceHolder surfaceHolder;

    public DoodleView(Context context) {
        super(context);
        init();
    }

    public DoodleView(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
    }

    public DoodleView(Context context, AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
        init();
    }

    private void init() {
        surfaceHolder = getHolder();
        surfaceHolder.addCallback(this);
        surfaceHolder.setFormat(PixelFormat.TRANSPARENT);
        setZOrderOnTop(true);

        paint = new Paint();
        paint.setColor(Color.RED);
        paint.setStrokeWidth(8f);
        paint.setStyle(Paint.Style.STROKE);
        paint.setStrokeCap(Paint.Cap.ROUND);
        paint.setStrokeJoin(Paint.Join.ROUND);
        paint.setAntiAlias(true);

        paths = new ArrayList<>();
        paints = new ArrayList<>();
    }

    public void setDrawingEnabled(boolean enabled) {
        isDrawingEnabled = enabled;
    }

    public boolean isDrawingEnabled() {
        return isDrawingEnabled;
    }

    public void clearDoodle() {
        paths.clear();
        paints.clear();
        drawCanvas();
    }

    public void setColor(int color) {
        paint.setColor(color);
    }

    public void setStrokeWidth(float width) {
        paint.setStrokeWidth(width);
    }
}
```

```

@Override
public boolean onTouchEvent(MotionEvent event) {
    if (!isDrawingEnabled) {
        return false;
    }

    float x = event.getX();
    float y = event.getY();

    switch (event.getAction()) {
        case MotionEvent.ACTION_DOWN:
            currentPath = new Path();
            currentPath.moveTo(x, y);

            // Create a new paint with current settings
            Paint pathPaint = new Paint(Paint);
            paths.add(currentPath);
            paints.add(pathPaint);
            return true;

        case MotionEvent.ACTION_MOVE:
            if (currentPath != null) {
                currentPath.lineTo(x, y);
                drawCanvas();
            }
            return true;

        case MotionEvent.ACTION_UP:
            currentPath = null;
            return true;
    }

    return false;
}

private void drawCanvas() {
    if (surfaceHolder == null || !surfaceHolder.getSurface().isValid()) {
        return;
    }

    Canvas canvas = null;
    try {
        canvas = surfaceHolder.lockCanvas();
        if (canvas != null) {
            // Clear the canvas with transparent
            canvas.drawColor(Color.TRANSPARENT, PorterDuff.Mode.CLEAR);

            // Draw all paths
            for (int i = 0; i < paths.size(); i++) {
                canvas.drawPath(paths.get(i), paints.get(i));
            }
        }
    } catch (Exception e) {
        // Ignore drawing errors
    } finally {
        if (canvas != null) {
            try {
                surfaceHolder.unlockCanvasAndPost(canvas);
            } catch (Exception e) {
                // Ignore unlock errors
            }
        }
    }
}

@Override
public void surfaceCreated(SurfaceHolder holder) {
    drawCanvas();
}

@Override
public void surfaceChanged(SurfaceHolder holder, int format, int width, int height) {
    drawCanvas();
}

@Override
public void surfaceDestroyed(SurfaceHolder holder) {
    // Clean up if needed
}
}

```

PhysicsSimulationView.java

app/src/main/java/com/example/phiz/views/PhysicsSimulationView.java

```
package com.example.phiz.views;

import android.content.Context;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.graphics.Path;
import android.util.AttributeSet;
import android.view.View;

import androidx.core.content.ContextCompat;

import com.example.phiz.R;

public class PhysicsSimulationView extends View {
    private Paint objectPaint, surfacePaint, vectorPaint, textPaint;
    private float objectX, objectY;
    private float objectSize = 80f;
    private float mass = 5.0f;
    private float appliedForce = 30.0f;
    private float frictionCoefficient = 0.2f;
    private float angle = 0.0f;
    private boolean isAnimating = false;

    private float velocity = 0.0f;
    private float acceleration = 0.0f;
    private float position = 0.0f;
    private static final float GRAVITY = 9.8f;

    private OnPhysicsUpdateListener listener;

    public interface OnPhysicsUpdateListener {
        void onUpdate(float acceleration, float velocity, float position);
    }

    public PhysicsSimulationView(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
    }

    private void init() {
        objectPaint = new Paint();
        objectPaint.setColor(ContextCompat.getColor(getContext(), R.color.blue_primary));
        objectPaint.setStyle(Paint.Style.FILL);

        surfacePaint = new Paint();
        surfacePaint.setColor(ContextCompat.getColor(getContext(), R.color.brown_surface));
        surfacePaint.setStrokeWidth(5f);

        vectorPaint = new Paint();
        vectorPaint.setStrokeWidth(4f);
        vectorPaint.setStyle(Paint.Style.STROKE);

        textPaint = new Paint();
        textPaint.setTextSize(30f);
        textPaint.setAntiAlias(true);
    }

    public void setParameters(float mass, float force, float friction, float angle) {
        this.mass = mass;
        this.appliedForce = force;
        this.frictionCoefficient = friction;
        this.angle = angle;
        calculatePhysics();
        invalidate();
    }

    public void startAnimation() {
        isAnimating = true;
        velocity = 0.0f;
        position = 0.0f;
        calculatePhysics();
        post(animationRunnable);
    }

    public void stopAnimation() {
        isAnimating = false;
        removeCallbacks(animationRunnable);
    }
}
```

```

public void reset() {
    stopAnimation();
    velocity = 0.0f;
    position = 0.0f;
    acceleration = 0.0f;
    objectX = 100f;
    if (listener != null) {
        listener.onUpdate(acceleration, velocity, position);
    }
    invalidate();
}

public void setOnPhysicsUpdateListener(OnPhysicsUpdateListener listener) {
    this.listener = listener;
}

private void calculatePhysics() {
    double angleRad = Math.toRadians(angle);
    float gravityComponent = (float) (mass * GRAVITY * Math.sin(angleRad));
    float normalForce = (float) (mass * GRAVITY * Math.cos(angleRad));
    float frictionForce = frictionCoefficient * normalForce;

    float netForce = appliedForce - frictionForce - gravityComponent;
    acceleration = netForce / mass;

    if (listener != null) {
        listener.onUpdate(acceleration, velocity, position);
    }
}

private Runnable animationRunnable = new Runnable() {
    @Override
    public void run() {
        if (isAnimating) {
            float deltaTime = 0.016f; // ~60 FPS
            velocity += acceleration * deltaTime;
            position += velocity * deltaTime;

            objectX = 100f + position * 20f; // Scale for visualization

            if (objectX > getWidth() - objectSize) {
                objectX = getWidth() - objectSize;
                velocity = 0;
            }

            if (objectX < 100f) {
                objectX = 100f;
                velocity = 0;
            }

            if (listener != null) {
                listener.onUpdate(acceleration, velocity, position);
            }

            invalidate();
            postDelayed(this, 16);
        }
    }
};

@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);

    int width = getWidth();
    int height = getHeight();

    float surfaceY = height * 0.7f;
    double angleRad = Math.toRadians(angle);

    canvas.drawLine(0, surfaceY, width, (float) (surfaceY - width * Math.tan(angleRad)),
        surfacePaint);

    if (objectX == 0) {
        objectX = 100f;
    }
    objectY = (float) (surfaceY - objectSize - (objectX * Math.tan(angleRad)));

    canvas.drawRect(objectX, objectY, objectX + objectSize, objectY + objectSize, objectPaint);

    float centerX = objectX + objectSize / 2;
    float centerY = objectY + objectSize / 2;

    float gravityForce = mass * GRAVITY;

```

```

drawVector(canvas, centerX, centerY, 0, gravityForce * 2, Color.RED, "Fg");

float normalForce = (float) (mass * GRAVITY * Math.cos(angleRad));
drawVector(canvas, centerX, centerY, 0, -normalForce * 2, Color.BLUE, "Fn");

if (appliedForce > 0) {
    float forceX = (float) (appliedForce * Math.cos(angleRad)) * 3;
    drawVector(canvas, centerX, centerY, forceX, 0, Color.GREEN, "Fa");
}

if (frictionCoefficient > 0 && velocity != 0) {
    float frictionForce = frictionCoefficient * normalForce;
    float frictionX = -(frictionForce * 3);
    drawVector(canvas, centerX, centerY, frictionX, 0, Color.YELLOW, "Ff");
}
}

private void drawVector(Canvas canvas, float startX, float startY, float dx, float dy,
    int color, String label) {
    vectorPaint.setColor(color);
    textPaint.setColor(color);

    float endX = startX + dx;
    float endY = startY + dy;

    canvas.drawLine(startX, startY, endX, endY, vectorPaint);

    double angle = Math.atan2(dy, dx);
    float arrowSize = 20f;
    float arrowX1 = (float) (endX - arrowSize * Math.cos(angle - Math.PI / 6));
    float arrowY1 = (float) (endY - arrowSize * Math.sin(angle - Math.PI / 6));
    float arrowX2 = (float) (endX - arrowSize * Math.cos(angle + Math.PI / 6));
    float arrowY2 = (float) (endY - arrowSize * Math.sin(angle + Math.PI / 6));

    Path arrowPath = new Path();
    arrowPath.moveTo(endX, endY);
    arrowPath.lineTo(arrowX1, arrowY1);
    arrowPath.moveTo(endX, endY);
    arrowPath.lineTo(arrowX2, arrowY2);
    canvas.drawPath(arrowPath, vectorPaint);

    canvas.drawText(label, endX + 10, endY - 10, textPaint);
}
}

```

InactivityCheckWorker.java

app/src/main/java/com/example/phiz/workers/InactivityCheckWorker.java

```
package com.example.phiz.workers;

import android.content.Context;
import android.content.SharedPreferences;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.work.Worker;
import androidx.work.WorkerParameters;

import com.example.phiz.helpers.NotificationHelper;

import java.util.concurrent.TimeUnit;

/**
 * Worker that checks for user inactivity and shows reminder notifications.
 * Runs periodically to check if user hasn't used the app in a while.
 */
public class InactivityCheckWorker extends Worker {
    private static final String TAG = "InactivityCheckWorker";

    public static final String WORK_NAME = "inactivity_check_work";
    private static final String PREFS_NAME = "activity_prefs";
    private static final String KEY_LAST_ACTIVITY = "last_activity_timestamp";
    private static final int INACTIVITY_THRESHOLD_DAYS = 3;

    public InactivityCheckWorker(@NonNull Context context, @NonNull WorkerParameters workerParams) {
        super(context, workerParams);
    }

    @NonNull
    @Override
    public Result doWork() {
        Log.d(TAG, "Inactivity check worker running");

        try {
            SharedPreferences prefs = getApplicationContext()
                .getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);

            long lastActivityTime = prefs.getLong(KEY_LAST_ACTIVITY, System.currentTimeMillis());
            long currentTime = System.currentTimeMillis();
            long daysSinceLastActivity = TimeUnit.MILLISECONDS.toDays(currentTime - lastActivityTime);

            Log.d(TAG, "Days since last activity: " + daysSinceLastActivity);

            if (daysSinceLastActivity >= INACTIVITY_THRESHOLD_DAYS) {
                // Show inactivity reminder notification
                NotificationHelper.getInstance(getApplicationContext())
                    .showInactivityReminderNotification((int) daysSinceLastActivity);

                Log.d(TAG, "Inactivity notification shown");
            }

            return Result.success();
        } catch (Exception e) {
            Log.e(TAG, "Error checking inactivity", e);
            return Result.failure();
        }
    }

    /**
     * Update the last activity timestamp.
     * Call this whenever the user performs an action in the app.
     */
    public static void updateLastActivity(Context context) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        prefs.edit().putLong(KEY_LAST_ACTIVITY, System.currentTimeMillis()).apply();
    }
}
```

StudyReminderWorker.java

app/src/main/java/com/example/phiz/workers/StudyReminderWorker.java

```
package com.example.phiz.workers;

import android.content.Context;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.work.Worker;
import androidx.work.WorkerParameters;

import com.example.phiz.helpers.NotificationHelper;

/**
 * Worker that shows daily study reminder notifications.
 * Scheduled via WorkManager based on user's notification preferences.
 */
public class StudyReminderWorker extends Worker {
    private static final String TAG = "StudyReminderWorker";

    public static final String WORK_NAME = "study_reminder_work";

    public StudyReminderWorker(@NonNull Context context, @NonNull WorkerParameters workerParams) {
        super(context, workerParams);
    }

    @NonNull
    @Override
    public Result doWork() {
        Log.d(TAG, "Study reminder worker running");

        try {
            // Show the study reminder notification
            NotificationHelper.getInstance(getApplicationContext())
                .showStudyReminderNotification();

            Log.d(TAG, "Study reminder notification shown");
            return Result.success();
        } catch (Exception e) {
            Log.e(TAG, "Error showing study reminder", e);
            return Result.failure();
        }
    }
}
```


WeeklyProgressWorker.java

app/src/main/java/com/example/phiz/workers/WeeklyProgressWorker.java

```
package com.example.phiz.workers;

import android.content.Context;
import android.content.SharedPreferences;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.work.Worker;
import androidx.work.WorkerParameters;

import com.example.phiz.helpers.NotificationHelper;

/**
 * Worker that checks weekly score progress and shows notifications.
 * Runs weekly to report on score improvements.
 */
public class WeeklyProgressWorker extends Worker {
    private static final String TAG = "WeeklyProgressWorker";

    public static final String WORK_NAME = "weekly_progress_work";
    private static final String PREFS_NAME = "progress_prefs";
    private static final String KEY_LAST_WEEK_SCORE = "last_week_score";
    private static final String KEY_CURRENT_SCORE = "current_score";

    public WeeklyProgressWorker(@NonNull Context context, @NonNull WorkerParameters workerParams) {
        super(context, workerParams);
    }

    @NonNull
    @Override
    public Result doWork() {
        Log.d(TAG, "Weekly progress worker running");

        try {
            SharedPreferences prefs = getApplicationContext()
                .getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);

            int lastWeekScore = prefs.getInt(KEY_LAST_WEEK_SCORE, 0);
            int currentScore = prefs.getInt(KEY_CURRENT_SCORE, 0);
            int improvement = currentScore - lastWeekScore;

            Log.d(TAG, "Last week score: " + lastWeekScore + ", Current score: " + currentScore);

            // Show weekly progress notification
            NotificationHelper.getInstance(getApplicationContext())
                .showWeeklyProgressNotification(improvement, currentScore);

            // Update last week's score for next comparison
            prefs.edit().putInt(KEY_LAST_WEEK_SCORE, currentScore).apply();

            Log.d(TAG, "Weekly progress notification shown");
            return Result.success();
        } catch (Exception e) {
            Log.e(TAG, "Error showing weekly progress", e);
            return Result.failure();
        }
    }

    /**
     * Update the current score.
     * Call this after a quiz is completed.
     */
    public static void updateCurrentScore(Context context, int score) {
        SharedPreferences prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
        prefs.edit().putInt(KEY_CURRENT_SCORE, score).apply();
    }
}
```

card_background_blue.xml

app/src/main/res/drawable/card_background_blue.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/blue_tint_bg"
        android:endColor="@color/blue_light"
        android:angle="135"/>
    <corners android:radius="16dp"/>
</shape>
```

card_background_green.xml

app/src/main/res/drawable/card_background_green.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/green_tint_bg"
        android:endColor="@color/green_light"
        android:angle="135"/>
    <corners android:radius="16dp"/>
</shape>
```

card_background_orange.xml

app/src/main/res/drawable/card_background_orange.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/orange_tint_bg"
        android:endColor="@color/orange_light"
        android:angle="135"/>
    <corners android:radius="16dp"/>
</shape>
```

circle_blue.xml

app/src/main/res/drawable/circle_blue.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <gradient
        android:startColor="@color/blue_primary"
        android:endColor="@color/blue_dark"
        android:angle="135"/>
    <size android:width="60dp" android:height="60dp"/>
</shape>
```

circle_green.xml

app/src/main/res/drawable/circle_green.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <gradient
        android:startColor="@color/green_primary"
        android:endColor="@color/green_dark"
        android:angle="135"/>
    <size android:width="60dp" android:height="60dp"/>
</shape>
```

circle_orange.xml

app/src/main/res/drawable/circle_orange.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <gradient
        android:startColor="@color/orange_primary"
        android:endColor="@color/orange_dark"
        android:angle="135"/>
    <size android:width="60dp" android:height="60dp"/>
</shape>
```

circle_purple.xml

app/src/main/res/drawable/circle_purple.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <gradient
        android:startColor="@color/purple_primary"
        android:endColor="@color/purple_dark"
        android:angle="135"/>
    <size android:width="60dp" android:height="60dp"/>
</shape>
```


difficulty_badge_background.xml

app/src/main/res/drawable/difficulty_badge_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/orange_primary"/>
    <corners android:radius="4dp"/>
</shape>
```

gradient_blue.xml

app/src/main/res/drawable/gradient_blue.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/blue_primary"
        android:endColor="@color/blue_dark"
        android:angle="135"/>
</shape>
```

gradient_green.xml

app/src/main/res/drawable/gradient_green.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/green_primary"
        android:endColor="@color/green_dark"
        android:angle="135"/>
</shape>
```

gradient_login_bg.xml

app/src/main/res/drawable/gradient_login_bg.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/blue_tint_bg"
        android:centerColor="@color/background_surface"
        android:endColor="@color/green_tint_bg"
        android:angle="160"/>
</shape>
```

gradient_orange.xml

app/src/main/res/drawable/gradient_orange.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/orange_primary"
        android:endColor="@color/orange_dark"
        android:angle="135"/>
</shape>
```

gradient_purple.xml

app/src/main/res/drawable/gradient_purple.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/purple_primary"
        android:endColor="@color/purple_dark"
        android:angle="135"/>
</shape>
```

gradient_red.xml

app/src/main/res/drawable/gradient_red.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <gradient
        android:startColor="@color/red_primary"
        android:endColor="@color/red_dark"
        android:angle="135"/>
</shape>
```

ic_add.xml

app/src/main/res/drawable/ic_add.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M19,13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/>
</vector>
```


ic_arrow_back.xml

app/src/main/res/drawable/ic_arrow_back.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="?attr/colorControlNormal">
    <path
        android:fillColor="@android:color/white"
        android:pathData="M20,11H7.83L5.59,-5.59L12,4l-8,8 8,8 1.41,-1.41L7.83,13H20v-2z"/>
</vector>
```

ic_clear.xml

app/src/main/res/drawable/ic_clear.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="@android:color/white">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M19,6.41L17.59,5 12,10.59 6.41,5 5,6.41 10.59,12 5,17.59 6.41,19 12,13.41 17.59,19 19,17.59 13.41,12z"/>
</vector>
```

ic_delete.xml

app/src/main/res/drawable/ic_delete.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="@color/red_primary"
        android:pathData="M6,19c0,1.1 0.9,2 2,2h8c1.1,0 2,-0.9 2,-2V7H6v12zM19,4h-3.5l-1,-1h-5l-1,1H5v2h14V4z"/>
</vector>
```

ic_edit.xml

app/src/main/res/drawable/ic_edit.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="@android:color/white">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M3,17.25V21h3.75L17.81,9.94l-3.75,-3.75L3,17.25z"
    />
</vector>
```

ic_email.xml

app/src/main/res/drawable/ic_email.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:width="24dp"
```

```
    android:height="24dp"
```

```
    android:viewportWidth="24"
```

```
    android:viewportHeight="24">
```

```
    <path
```

```
        android:fillColor="@color/text_tertiary"
```

```
        android:pathData="M20,4H4C2.9,4 2.01,4.9 2.01,6L2,18c0,1.1 0.9,2 2,2h16c1.1,0 2,-0.9 2,-2V6C22,4.9 21.1,4 20,4zM20,8l-8,5"
```

```
</vector>
```

ic_grades.xml

app/src/main/res/drawable/ic_grades.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M19,3H5C3.9,3 3,3.9 3,5v14c0,1.1 0.9,2 2,2h14c1.1,0 2,-0.9 2,-2V5C21,3.9 20.1,3 19,3zM9,17H7v-7h2V17zM19,17H17v-7h2V17z" />
</vector>
```

ic_launcher_background.xml

app/src/main/res/drawable/ic_launcher_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path
        android:fillColor="#3DDC84"
        android:pathData="M0,0h108v108h-108z" />
    <path
        android:fillColor="#00000000"
        android:pathData="M9,0L9,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M19,0L19,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M29,0L29,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M39,0L39,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M49,0L49,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M59,0L59,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M69,0L69,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M79,0L79,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M89,0L89,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M99,0L99,108"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,9L108,9"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,19L108,19"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,29L108,29"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
        android:fillColor="#00000000"
        android:pathData="M0,39L108,39"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
    <path
```

```

        android:fillColor="#00000000"
        android:pathData="M0,49L108,49"
        android:strokeWidth="0.8"
        android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,59L108,59"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,69L108,69"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,79L108,79"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,89L108,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M0,99L108,99"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,29L89,29"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,39L89,39"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,49L89,49"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,59L89,59"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,69L89,69"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M19,79L89,79"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M29,19L29,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M39,19L39,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M49,19L49,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M59,19L59,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />
<path
    android:fillColor="#00000000"
    android:pathData="M69,19L69,89"
    android:strokeWidth="0.8"
    android:strokeColor="#33FFFFFF" />

```



```
<path
  android:fillColor="#00000000"
  android:pathData="M79,19L79,89"
  android:strokeWidth="0.8"
  android:strokeColor="#33FFFFFF" />
</vector>
```

ic_launcher_foreground.xml

app/src/main/res/drawable/ic_launcher_foreground.xml

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <path android:pathData="M31,63.928c0,0 6.4, -11 12.1, -13.1c7.2, -2.6 26, -1.4 26, -1.4l38.1,38.1l107,108.928l-32, -1L31,63.928z">
        <aapt:attr name="android:fillColor">
            <gradient
                android:endX="85.84757"
                android:endY="92.4963"
                android:startX="42.9492"
                android:startY="49.59793"
                android:type="linear">
                    <item
                        android:color="#44000000"
                        android:offset="0.0" />
                    <item
                        android:color="#00000000"
                        android:offset="1.0" />
                </gradient>
            </aapt:attr>
        </path>
        <path
            android:fillColor="#FFFFFF"
            android:fillType="nonZero"
            android:pathData="M65.3,45.828l3.8, -6.6c0.2, -0.4 0.1, -0.9 -0.3, -1.1c-0.4, -0.2 -0.9, -0.1 -1.1,0.3l-3.9,6.7c-6.3, -2.8 -13.4"
            android:strokeWidth="1"
            android:strokeColor="#00000000" />
    </vector>
```

ic_list.xml

app/src/main/res/drawable/ic_list.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M3,13h2v-2H3v2zM3,17h2v-2H3v2zM3,9h2V7H3v2zM7,13h14v-2H7v2zM7,17h14v-2H7v2zM7,7v2h14V7H7z"/>
</vector>
```

ic_lock.xml

app/src/main/res/drawable/ic_lock.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="@color/text_tertiary"
        android:pathData="M18,8h-1V6c0,-2.76 -2.24,-5 -5,-5S7,3.24 7,6v2H6c-1.1,0 -2,0.9 -2,2v10c0,1.1 0.9,2 2,2h12c1.1,0 2,-0.9
    </vector>
```

ic_logout.xml

app/src/main/res/drawable/ic_logout.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M17,7l-1.41,1.41L18.17,11H8v2h10.17l-2.58,2.58L17,17l5,-5zM4,5h8V3H4c-1.1,0 -2,0.9 -2,2v14c0,1.1 0.9,2
    </vector>
```

ic_notification.xml

app/src/main/res/drawable/ic_notification.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24"
    android:tint="#FFFFFF">
    <path
        android:fillColor="@android:color/white"
        android:pathData="M12,22c1.1,0 2,-0.9 2,-2h-4c0,1.1 0.89,2 2,2z"
    />
</vector>
```

ic_person.xml

app/src/main/res/drawable/ic_person.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M12,12c2.21,0 4,-1.79 4,-4s-1.79,-4 -4,-4 -4,-4 -4,1.79 -4,4 1.79,4 4,4zM12,14c-2.67,0 -8,1.34 -8,4v2h16v-2c0,-2.67 -2.67,-4 -8,-4"
    />
</vector>
```

ic_physics.xml

app/src/main/res/drawable/ic_physics.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M19.8,18.4L14,10.67V6.5l1.35,-1.69C15.61,4.48 15.38,4 14.96,4H9.04C8.62,4 8.39,4.48 8.65,4.81L10,6.5v4.
</vector>
```


ic_quiz.xml

app/src/main/res/drawable/ic_quiz.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M4,6H2v14c0,1.1 0.9,2 2,2h14v-2H4V6z"
    />
</vector>
```

ic_star.xml

app/src/main/res/drawable/ic_star.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFD700"
        android:pathData="M12,17.27L18.18,21l-1.64,-7.03L22,9.24l-7.19,-0.61L12,2 9.19,8.63 2,9.24L5.46,4.73L5.82,21z"/>
</vector>
```

ic_students.xml

app/src/main/res/drawable/ic_students.xml

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M16,11c1.66,0 2.99,-1.34 2.99,-3S17.66,5 16,5c-1.66,0 -3,1.34 -3,3s1.34,3 3,3zM8,11c1.66,0 2.99,-1.34 2.99,-3S8.66,5 8,5c-1.66,0 -3,1.34 -3,3s1.34,3 3,3Z" />
</vector>
```

ic_trophy.xml

app/src/main/res/drawable/ic_trophy.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<vector xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:width="24dp"
```

```
    android:height="24dp"
```

```
    android:viewportWidth="24"
```

```
    android:viewportHeight="24">
```

```
    <path
```

```
        android:fillColor="#FFD700"
```

```
        android:pathData="M19,5h-2V3H7v2H5C3.9,5 3,5.9 3,7v1c0,2.55 1.92,4.63 4.39,4.94c0.63,1.5 1.98,2.63 3.61,2.96V19H7v2h10v-2"
```

```
</vector>
```

rounded_button_blue.xml

app/src/main/res/drawable/rounded_button_blue.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ripple xmlns:android="http://schemas.android.com/apk/res/android"
    android:color="@color/blue_dark">
    <item>
        <shape android:shape="rectangle">
            <gradient
                android:startColor="@color/blue_primary"
                android:endColor="@color/blue_medium"
                android:angle="90"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</ripple>
```

rounded_button_gray.xml

app/src/main/res/drawable/rounded_button_gray.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ripple xmlns:android="http://schemas.android.com/apk/res/android"
    android:color="@color/gray_darker">
    <item>
        <shape android:shape="rectangle">
            <gradient
                android:startColor="@color/gray_medium"
                android:endColor="@color/gray_dark"
                android:angle="90"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</ripple>
```

rounded_button_green.xml

app/src/main/res/drawable/rounded_button_green.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ripple xmlns:android="http://schemas.android.com/apk/res/android"
    android:color="@color/green_dark">
    <item>
        <shape android:shape="rectangle">
            <gradient
                android:startColor="@color/green_primary"
                android:endColor="@color/green_medium"
                android:angle="90"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</ripple>
```

rounded_button_orange.xml

app/src/main/res/drawable/rounded_button_orange.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ripple xmlns:android="http://schemas.android.com/apk/res/android"
    android:color="@color/orange_dark">
    <item>
        <shape android:shape="rectangle">
            <gradient
                android:startColor="@color/orange_primary"
                android:endColor="@color/orange_medium"
                android:angle="90"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</ripple>
```


rounded_button_red.xml

app/src/main/res/drawable/rounded_button_red.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ripple xmlns:android="http://schemas.android.com/apk/res/android"
    android:color="@color/red_dark">
    <item>
        <shape android:shape="rectangle">
            <gradient
                android:startColor="@color/red_primary"
                android:endColor="@color/red_medium"
                android:angle="90"/>
            <corners android:radius="12dp"/>
        </shape>
    </item>
</ripple>
```

timer_background.xml

app/src/main/res/drawable/timer_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="@color/orange_tint_bg"/>
    <corners android:radius="16dp"/>
    <stroke
        android:width="2dp"
        android:color="@color/deep_orange"/>
</shape>
```

activity_all_grades.xml

app/src/main/res/layout/activity_all_grades.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Header -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:padding="16dp"
        android:background="@drawable/gradient_blue">

        <com.google.android.material.card.MaterialCardView
            android:id="@+id/backButton"
            android:layout_width="40dp"
            android:layout_height="40dp"
            app:cardCornerRadius="20dp"
            app:cardElevation="4dp"
            android:clickable="true"
            android:focusable="true">

            <ImageView
                android:layout_width="24dp"
                android:layout_height="24dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_arrow_back"
                app:tint="@color/text_primary"
                android:contentDescription="Back"/>
        </com.google.android.material.card.MaterialCardView>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:id="@+id/titleTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="All Grades"
                android:textSize="22sp"
                android:textStyle="bold"
                android:textColor="@android:color/white"/>

            <TextView
                android:id="@+id/subtitleTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="View all student quiz results"
                android:textSize="14sp"
                android:textColor="@color/text_on_gradient"/>
        </LinearLayout>
    </LinearLayout>

    <!-- Grades RecyclerView -->
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/allGradesRecyclerView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="16dp"
        android:clipToPadding="false"/>

    <!-- Empty State -->
    <LinearLayout
        android:id="@+id/emptyStateLayout"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center"
        android:visibility="gone"
        android:padding="32dp">
```

```

<FrameLayout
    android:layout_width="100dp"
    android:layout_height="100dp">

    <View
        android:layout_width="100dp"
        android:layout_height="100dp"
        android:background="@drawable/circle_blue"/>

    <ImageView
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:layout_gravity="center"
        android:src="@drawable/ic_grades"
        android:contentDescription="No grades"/>
</FrameLayout>

<TextView
    android:id="@+id/emptyTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="No quiz results yet"
    android:textSize="18sp"
    android:textColor="@color/text_tertiary"
    android:layout_marginTop="24dp"/>

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Students haven't taken any quizzes"
    android:textSize="14sp"
    android:textColor="@color/text_disabled"
    android:layout_marginTop="8dp"/>
</LinearLayout>

<!-- Loading indicator -->
<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_gravity="center"
    android:visibility="gone"
    android:indeterminateTint="@color/blue_primary"/>

</LinearLayout>

```

activity_create_question.xml

app/src/main/res/layout/activity_create_question.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background_primary">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="@color/blue_primary"
            android:elevation="4dp"
            app:navigationIcon="@drawable/ic_arrow_back"
            app:navigationIconTint="@android:color/white"
            app:titleTextColor="@android:color/white">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Create Question"
                android:textColor="@android:color/white"
                android:textSize="20sp"
                android:textStyle="bold"/>
        </com.google.android.material.appbar.MaterialToolbar>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Question Text"
                style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
                android:layout_marginBottom="16dp">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/questionTextInput"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="textMultiLine"
                    android:minLines="2"/>
            </com.google.android.material.textfield.TextInputLayout>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Answer Options"
                android:textSize="16sp"
                android:textStyle="bold"
                android:textColor="@color/text_primary"
                android:layout_marginBottom="8dp"/>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Option 1"
                style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
                android:layout_marginBottom="8dp">

                <com.google.android.material.textfield.TextInputEditText
                    android:id="@+id/option1Input"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:inputType="text"/>
            </com.google.android.material.textfield.TextInputLayout>

            <com.google.android.material.textfield.TextInputLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Option 2"
                style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox">
```

```

        android:layout_marginBottom="8dp">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/option2Input"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="text"/>
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Option 3"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_marginBottom="8dp">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/option3Input"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="text"/>
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Option 4"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_marginBottom="16dp">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/option4Input"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="text"/>
    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Correct Answer"
        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@color/text_primary"
        android:layout_marginBottom="8dp"/>

    <RadioGroup
        android:id="@+id/correctAnswerRadioGroup"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginBottom="16dp">

        <RadioButton
            android:id="@+id/radioOption1"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="1"/>

        <RadioButton
            android:id="@+id/radioOption2"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="2"/>

        <RadioButton
            android:id="@+id/radioOption3"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="3"/>

        <RadioButton
            android:id="@+id/radioOption4"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="4"/>
    </RadioGroup>

    <TextView
        android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:text="Difficulty"
        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@color/text_primary"
        android:layout_marginBottom="8dp"/>

```

<Spinner

```

        android:id="@+id/difficultySpinner"
        android:layout_width="match_parent"
        android:layout_height="48dp"
        android:background="@android:drawable/btn_dropdown"
        android:layout_marginBottom="16dp"/>

```

<com.google.android.material.textfield.TextInputLayout

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Explanation (Optional)"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_marginBottom="16dp">

```

<com.google.android.material.textfield.TextInputEditText

```

        android:id="@+id/explanationInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textMultiLine"
        android:minLines="2"/>

```

</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Point Value"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
        android:layout_marginBottom="24dp">

```

<com.google.android.material.textfield.TextInputEditText

```

        android:id="@+id/pointValueInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="number"
        android:text="20"/>

```

</com.google.android.material.textfield.TextInputLayout>

<ProgressBar

```

        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:visibility="gone"/>

```

<Button

```

        android:id="@+id/saveQuestionButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save Question"
        android:backgroundTint="@color/green_primary"
        android:textColor="@android:color/white"
        android:textSize="16sp"/>

```

</LinearLayout>

</LinearLayout>

</ScrollView>

activity_grades.xml

app/src/main/res/layout/activity_grades.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Header with gradient -->
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <View
            android:layout_width="match_parent"
            android:layout_height="140dp"
            android:background="@drawable/gradient_orange"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="20dp"
            android:gravity="center_vertical">

            <!-- Back Button -->
            <com.google.android.material.card.MaterialCardView
                android:id="@+id/backButton"
                android:layout_width="44dp"
                android:layout_height="44dp"
                app:cardCornerRadius="22dp"
                app:cardElevation="4dp"
                app:cardBackgroundColor="@color/background_surface"
                android:clickable="true"
                android:focusable="true">

                <ImageView
                    android:layout_width="24dp"
                    android:layout_height="24dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_arrow_back"
                    app:tint="@color/orange_primary"
                    android:contentDescription="Back"/>
            </com.google.android.material.card.MaterialCardView>

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:orientation="vertical"
                android:layout_marginStart="16dp">

                <TextView
                    android:id="@+id/titleTextView"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="My Grades"
                    android:textColor="@android:color/white"
                    android:textSize="24sp"
                    android:textStyle="bold"/>

                <TextView
                    android:id="@+id/subtitleTextView"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="View your quiz results"
                    android:textSize="14sp"
                    android:textColor="@color/orange_light"
                    android:layout_marginTop="4dp"/>
            </LinearLayout>

            <!-- Grades Icon -->
            <FrameLayout
                android:layout_width="56dp"
                android:layout_height="56dp">

                <View
                    android:layout_width="56dp"
                    android:layout_height="56dp"
                    android:background="@drawable/circle_green"/>
            </FrameLayout>
        </LinearLayout>
    </FrameLayout>
</LinearLayout>
```



```

        <ImageView
            android:layout_width="32dp"
            android:layout_height="32dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_grades"
            android:contentDescription="Grades"/>
    </FrameLayout>
</LinearLayout>
</FrameLayout>

<!-- Grades List -->
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/gradesRecyclerView"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:padding="16dp"
    android:clipToPadding="false"/>

<!-- Empty State -->
<LinearLayout
    android:id="@+id/emptyStateLayout"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="vertical"
    android:gravity="center"
    android:visibility="gone"
    android:padding="32dp">

    <FrameLayout
        android:layout_width="100dp"
        android:layout_height="100dp">

        <View
            android:layout_width="100dp"
            android:layout_height="100dp"
            android:background="@drawable/circle_orange"
            android:alpha="0.2"/>

        <ImageView
            android:layout_width="50dp"
            android:layout_height="50dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_grades"
            app:tint="@color/gray_icon"
            android:contentDescription="No grades"/>
    </FrameLayout>

    <TextView
        android:id="@+id/emptyTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="No quiz results yet"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="@color/text_tertiary"
        android:layout_marginTop="24dp"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Take a quiz to see your grades here!"
        android:textSize="14sp"
        android:textColor="@color/text_placeholder"
        android:layout_marginTop="8dp"
        android:gravity="center"/>

    <Button
        android:id="@+id/takeQuizButton"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:layout_marginTop="24dp"
        android:paddingHorizontal="32dp"
        android:text="Take a Quiz"
        android:textAllCaps="false"
        android:background="@drawable/rounded_button_green"
        android:textColor="@android:color/white"/>
</LinearLayout>
</LinearLayout>

```

activity_login.xml

app/src/main/res/layout/activity_login.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@drawable/gradient_login_bg">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp"
        android:gravity="center">

        <!-- Logo/Icon Area -->
        <FrameLayout
            android:layout_width="120dp"
            android:layout_height="120dp"
            android:layout_marginTop="40dp"
            android:layout_marginBottom="24dp">

            <View
                android:layout_width="120dp"
                android:layout_height="120dp"
                android:background="@drawable/circle_blue"/>

            <ImageView
                android:layout_width="60dp"
                android:layout_height="60dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_physics"
                android:contentDescription="Physics icon"/>
        </FrameLayout>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Physics Simulator"
            android:textSize="32sp"
            android:textStyle="bold"
            android:textColor="@color/blue_dark"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Learn Newton's Laws of Motion"
            android:textSize="16sp"
            android:textColor="@color/text_tertiary"
            android:layout_marginBottom="40dp"/>

        <!-- Login Card -->
        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardCornerRadius="20dp"
            app:cardElevation="8dp"
            android:layout_marginBottom="24dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="24dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Welcome Back!"
                    android:textSize="24sp"
                    android:textStyle="bold"
                    android:textColor="@color/text_primary"
                    android:layout_marginBottom="8dp"/>

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Sign in to continue learning"
                    android:textSize="14sp"
                    android:textColor="@color/text_hint">
            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>
    </LinearLayout>
</ScrollView>
```

```

        android:layout_marginBottom="24dp"/>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    app:boxCornerRadiusBottomEnd="12dp"
    app:boxCornerRadiusBottomStart="12dp"
    app:boxCornerRadiusTopEnd="12dp"
    app:boxCornerRadiusTopStart="12dp"
    app:startIconDrawable="@drawable/ic_email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/emailEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email Address"
        android:inputType="textEmailAddress"/>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="24dp"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    app:boxCornerRadiusBottomEnd="12dp"
    app:boxCornerRadiusBottomStart="12dp"
    app:boxCornerRadiusTopEnd="12dp"
    app:boxCornerRadiusTopStart="12dp"
    app:endIconMode="password_toggle"
    app:startIconDrawable="@drawable/ic_lock">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/passwordEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"/>
</com.google.android.material.textfield.TextInputLayout>

<Button
    android:id="@+id/loginButton"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:text="Sign In"
    android:textSize="16sp"
    android:textAllCaps="false"
    android:background="@drawable/rounded_button_blue"
    android:textColor="@android:color/white"/>

</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Register Link -->
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Don't have an account? "
        android:textSize="15sp"
        android:textColor="@color/text_tertiary"/>

    <TextView
        android:id="@+id/registerTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sign Up"
        android:textSize="15sp"
        android:textColor="@color/blue_medium"
        android:textStyle="bold"
        android:clickable="true"
        android:focusable="true"
        android:padding="8dp"/>
</LinearLayout>

<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="48dp"

```

```
android:layout_height="48dp"  
android:layout_marginTop="24dp"  
android:visibility="gone"  
android:indeterminateTint="@color/blue_primary"/>
```

```
</LinearLayout>  
</ScrollView>
```

activity_main.xml

app/src/main/res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

activity_notification_settings.xml

app/src/main/res/layout/activity_notification_settings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Toolbar -->
    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="@drawable/gradient_blue"
        app:navigationIcon="@drawable/ic_arrow_back"
        app:navigationIconTint="@android:color/white"
        app:title="Notification Settings"
        app:titleTextColor="@android:color/white" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <!-- Info Card -->
            <com.google.android.material.card.MaterialCardView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="16dp"
                app:cardCornerRadius="12dp"
                app:cardElevation="2dp"
                app:cardBackgroundColor="@color/blue_tint_bg">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="horizontal"
                    android:padding="16dp"
                    android:gravity="center_vertical">

                    <ImageView
                        android:layout_width="24dp"
                        android:layout_height="24dp"
                        android:src="@drawable/ic_notification"
                        app:tint="@color/blue_dark"
                        android:contentDescription="Info" />

                    <TextView
                        android:layout_width="0dp"
                        android:layout_height="wrap_content"
                        android:layout_weight="1"
                        android:layout_marginStart="12dp"
                        android:text="Manage which notifications you receive from Phiz"
                        android:textSize="14sp"
                        android:textColor="@color/blue_dark" />

                </LinearLayout>
            </com.google.android.material.card.MaterialCardView>

            <!-- Notification Toggles Card -->
            <com.google.android.material.card.MaterialCardView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="16dp"
                app:cardCornerRadius="16dp"
                app:cardElevation="4dp">

                <LinearLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:padding="16dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
```

```

        android:text="Notification Types"
        android:textSize="18sp"
        android:textStyle="bold"
        android:textColor="@color/text_primary"
        android:layout_marginBottom="16dp" />

<!-- Quiz Notifications -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingVertical="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Quiz Notifications"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="New quizzes and deadline reminders"
            android:textSize="12sp"
            android:textColor="@color/text_hint" />
    </LinearLayout>

    <com.google.android.material.switchmaterial.SwitchMaterial
        android:id="@+id/switchQuizNotifications"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true" />
</LinearLayout>

<View
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/divider" />

<!-- Grade Notifications -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingVertical="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Grade Notifications"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Quiz results and score updates"
            android:textSize="12sp"
            android:textColor="@color/text_hint" />
    </LinearLayout>

    <com.google.android.material.switchmaterial.SwitchMaterial
        android:id="@+id/switchGradeNotifications"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true" />
</LinearLayout>

<View

```

```

        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="@color/divider" />

<!-- Achievement Notifications -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingVertical="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Achievement Notifications"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Milestones and achievements unlocked"
            android:textSize="12sp"
            android:textColor="@color/text_hint" />
    </LinearLayout>

    <com.google.android.material.switchmaterial.SwitchMaterial
        android:id="@+id/switchAchievementNotifications"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true" />
</LinearLayout>

<!-- Study Reminders -->
<View
    android:id="@+id/dividerStudyReminders"
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/divider" />

<LinearLayout
    android:id="@+id/studyRemindersRow"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingVertical="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Study Reminders"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Daily reminders to practice physics"
            android:textSize="12sp"
            android:textColor="@color/text_hint" />
    </LinearLayout>

    <com.google.android.material.switchmaterial.SwitchMaterial
        android:id="@+id/switchStudyReminders"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true" />
</LinearLayout>

<!-- Weekly Progress -->

```



```

<View
    android:id="@+id/dividerWeeklyProgress"
    android:layout_width="match_parent"
    android:layout_height="1dp"
    android:background="@color/divider" />

<LinearLayout
    android:id="@+id/weeklyProgressRow"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingVertical="12dp">

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Weekly Progress"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Weekly summary of your improvement"
            android:textSize="12sp"
            android:textColor="@color/text_hint" />
    </LinearLayout>

    <com.google.android.material.switchmaterial.SwitchMaterial
        android:id="@+id/switchWeeklyProgress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:checked="true" />
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Reminder Time Card -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/reminderTimeCard"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    android:clickable="true"
    android:focusable="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Study Reminder Time"
            android:textSize="18sp"
            android:textStyle="bold"
            android:textColor="@color/text_primary"
            android:layout_marginBottom="12dp" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical">

            <FrameLayout
                android:layout_width="48dp"
                android:layout_height="48dp">

                <View
                    android:layout_width="48dp"
                    android:layout_height="48dp"
                    android:background="@drawable/circle_orange" />
            </FrameLayout>
        </LinearLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

```

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_notification"
            android:contentDescription="Time" />
    </FrameLayout>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical"
        android:layout_marginStart="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Reminder Time"
            android:textSize="16sp"
            android:textColor="@color/text_primary" />

        <TextView
            android:id="@+id/reminderTimeText"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="6:00 PM"
            android:textSize="14sp"
            android:textColor="@color/orange_primary" />
    </LinearLayout>

    <ImageView
        android:layout_width="24dp"
        android:layout_height="24dp"
        android:src="@drawable/ic_arrow_back"
        android:rotation="180"
        app:tint="@color/gray_icon"
        android:contentDescription="Change" />
    </LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Save Button -->
<Button
    android:id="@+id/saveButton"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:text="Save Settings"
    android:textColor="@android:color/white"
    android:textAllCaps="false"
    android:textSize="16sp"
    android:background="@drawable/rounded_button_blue"
    android:layout_marginTop="8dp" />

<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:layout_marginTop="16dp"
    android:visibility="gone" />

</LinearLayout>
</ScrollView>
</LinearLayout>

```

activity_physics_simulation.xml

app/src/main/res/layout/activity_physics_simulation.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Header with gradient -->
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <View
            android:layout_width="match_parent"
            android:layout_height="100dp"
            android:background="@drawable/gradient_blue"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:padding="20dp"
            android:gravity="center_vertical">

            <!-- Back Button -->
            <com.google.android.material.card.MaterialCardView
                android:id="@+id/backButton"
                android:layout_width="44dp"
                android:layout_height="44dp"
                app:cardCornerRadius="22dp"
                app:cardElevation="4dp"
                app:cardBackgroundColor="@color/background_surface"
                android:clickable="true"
                android:focusable="true">

                <ImageView
                    android:layout_width="24dp"
                    android:layout_height="24dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_arrow_back"
                    app:tint="@color/blue_primary"
                    android:contentDescription="Back"/>
            </com.google.android.material.card.MaterialCardView>

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:orientation="vertical"
                android:layout_marginStart="16dp">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Physics Simulation"
                    android:textColor="@android:color/white"
                    android:textSize="22sp"
                    android:textStyle="bold"/>

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Newton's Laws of Motion"
                    android:textSize="13sp"
                    android:textColor="@color/text_on_gradient"/>
            </LinearLayout>

            <!-- Physics Icon -->
            <FrameLayout
                android:layout_width="48dp"
                android:layout_height="48dp">

                <View
                    android:layout_width="48dp"
                    android:layout_height="48dp"
                    android:background="@drawable/circle_orange"/>

                <ImageView
                    android:layout_width="28dp"
```

```

        android:layout_height="28dp"
        android:layout_gravity="center"
        android:src="@drawable/ic_physics"
        android:contentDescription="Physics"/>
    </FrameLayout>
</LinearLayout>
</FrameLayout>

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:clipToPadding="false">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <!-- Simulation View Card -->
        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            app:cardCornerRadius="20dp"
            app:cardElevation="6dp">

            <com.example.phiz.views.PhysicsSimulationView
                android:id="@+id/simulationView"
                android:layout_width="match_parent"
                android:layout_height="280dp"
                android:background="@android:color/white"/>
        </com.google.android.material.card.MaterialCardView>

        <!-- Law Description Card -->
        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            app:cardCornerRadius="16dp"
            app:cardElevation="4dp"
            app:cardBackgroundColor="@color/blue_tint_bg">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="16dp">

                <LinearLayout
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:orientation="horizontal"
                    android:gravity="center_vertical"
                    android:layout_marginBottom="8dp">

                    <FrameLayout
                        android:layout_width="32dp"
                        android:layout_height="32dp">

                        <View
                            android:layout_width="32dp"
                            android:layout_height="32dp"
                            android:background="@drawable/circle_blue"/>

                        <ImageView
                            android:layout_width="18dp"
                            android:layout_height="18dp"
                            android:layout_gravity="center"
                            android:src="@drawable/ic_physics"
                            android:contentDescription="Laws"/>
                    </FrameLayout>

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Newton's Laws"
                        android:textSize="16sp"
                        android:textStyle="bold"
                        android:textColor="@color/blue_dark"
                        android:layout_marginStart="8dp"/>
                </LinearLayout>

                <TextView

```

```

        android:id="@+id/lawDescriptionTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Adjust parameters to see forces and motion"
        android:textSize="13sp"
        android:textColor="@color/text_secondary"
        android:lineSpacingExtra="4dp"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Parameters Card -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Parameters"
            android:textSize="18sp"
            android:textStyle="bold"
            android:textColor="@color/text_primary"
            android:layout_marginBottom="16dp"/>

        <!-- Mass -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:layout_marginBottom="4dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Mass"
                android:textSize="15sp"
                android:textColor="@color/text_secondary"/>

            <TextView
                android:id="@+id/massValueTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="5.0 kg"
                android:textSize="15sp"
                android:textStyle="bold"
                android:textColor="@color/blue_primary"/>
        </LinearLayout>

        <SeekBar
            android:id="@+id/massSeekBar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:max="100"
            android:progress="50"
            android:progressTint="@color/blue_primary"
            android:thumbTint="@color/blue_primary"
            android:layout_marginBottom="16dp"/>

        <!-- Force -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:layout_marginBottom="4dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Applied Force"
                android:textSize="15sp"
                android:textColor="@color/text_secondary"/>

```

```

        <TextView
            android:id="@+id/forceValueTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="30.0 N"
            android:textSize="15sp"
            android:textStyle="bold"
            android:textColor="@color/green_primary"/>
    </LinearLayout>

    <SeekBar
        android:id="@+id/forceSeekBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:max="100"
        android:progress="30"
        android:progressTint="@color/green_primary"
        android:thumbTint="@color/green_primary"
        android:layout_marginBottom="16dp"/>

    <!-- Friction -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:layout_marginBottom="4dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Friction Coefficient"
            android:textSize="15sp"
            android:textColor="@color/text_secondary"/>

        <TextView
            android:id="@+id/frictionValueTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="0.2"
            android:textSize="15sp"
            android:textStyle="bold"
            android:textColor="@color/orange_primary"/>
    </LinearLayout>

    <SeekBar
        android:id="@+id/frictionSeekBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:max="100"
        android:progress="20"
        android:progressTint="@color/orange_primary"
        android:thumbTint="@color/orange_primary"
        android:layout_marginBottom="16dp"/>

    <!-- Angle -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical"
        android:layout_marginBottom="4dp">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Angle"
            android:textSize="15sp"
            android:textColor="@color/text_secondary"/>

        <TextView
            android:id="@+id/angleValueTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="0°"
            android:textSize="15sp"
            android:textStyle="bold"
            android:textColor="@color/purple_primary"/>
    </LinearLayout>

    <SeekBar

```

```

        android:id="@+id/angleSeekBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:max="45"
        android:progress="0"
        android:progressTint="@color/purple_primary"
        android:thumbTint="@color/purple_primary"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Results Card -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    app:cardBackgroundColor="@color/green_tint_bg">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Calculated Values"
            android:textSize="16sp"
            android:textStyle="bold"
            android:textColor="@color/green_dark"
            android:layout_marginBottom="12dp"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:orientation="vertical"
                android:gravity="center">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Acceleration"
                    android:textSize="12sp"
                    android:textColor="@color/text_tertiary"/>

                <TextView
                    android:id="@+id/accelerationTextView"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="0.0 m/s2"
                    android:textSize="16sp"
                    android:textStyle="bold"
                    android:textColor="@color/green_dark"
                    android:layout_marginTop="4dp"/>
            </LinearLayout>

            <View
                android:layout_width="1dp"
                android:layout_height="40dp"
                android:background="@color/green_light"/>

            <LinearLayout
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:orientation="vertical"
                android:gravity="center">

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Velocity"
                    android:textSize="12sp"
                    android:textColor="@color/text_tertiary"/>

                <TextView

```

```

        android:id="@+id/velocityTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="0.0 m/s"
        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@color/green_dark"
        android:layout_marginTop="4dp"/>
    </LinearLayout>

    <View
        android:layout_width="1dp"
        android:layout_height="40dp"
        android:background="@color/green_light"/>

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical"
        android:gravity="center">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Position"
            android:textSize="12sp"
            android:textColor="@color/text_tertiary"/>

        <TextView
            android:id="@+id/positionTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="0.0 m"
            android:textSize="16sp"
            android:textStyle="bold"
            android:textColor="@color/green_dark"
            android:layout_marginTop="4dp"/>
    </LinearLayout>
</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Control Buttons -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="16dp">

    <Button
        android:id="@+id/startButton"
        android:layout_width="0dp"
        android:layout_height="56dp"
        android:layout_weight="1"
        android:text="Start"
        android:textAllCaps="false"
        android:textSize="16sp"
        android:background="@drawable/rounded_button_green"
        android:textColor="@android:color/white"
        android:layout_marginEnd="8dp"/>

    <Button
        android:id="@+id/resetButton"
        android:layout_width="0dp"
        android:layout_height="56dp"
        android:layout_weight="1"
        android:text="Reset"
        android:textAllCaps="false"
        android:textSize="16sp"
        android:background="@drawable/rounded_button_orange"
        android:textColor="@android:color/white"
        android:layout_marginStart="8dp"/>
</LinearLayout>

</LinearLayout>
</ScrollView>
</LinearLayout>

```


activity_quiz.xml

app/src/main/res/layout/activity_quiz.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background_primary">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

        <!-- Header with gradient -->
        <FrameLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content">

            <View
                android:layout_width="match_parent"
                android:layout_height="120dp"
                android:background="@drawable/gradient_green"/>

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:padding="20dp"
                android:gravity="center_vertical">

                <!-- Quiz Icon -->
                <FrameLayout
                    android:layout_width="50dp"
                    android:layout_height="50dp">

                    <View
                        android:layout_width="50dp"
                        android:layout_height="50dp"
                        android:background="@drawable/circle_orange"/>

                    <ImageView
                        android:layout_width="28dp"
                        android:layout_height="28dp"
                        android:layout_gravity="center"
                        android:src="@drawable/ic_quiz"
                        android:contentDescription="Quiz"/>
                </FrameLayout>

                <LinearLayout
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:orientation="vertical"
                    android:layout_marginStart="12dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Newton's Laws Quiz"
                        android:textColor="@android:color/white"
                        android:textSize="20sp"
                        android:textStyle="bold"/>

                    <TextView
                        android:id="@+id/questionNumberTextView"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Question 1 of 5"
                        android:textSize="14sp"
                        android:textColor="@color/green_light"
                        android:layout_marginTop="2dp"/>
                </LinearLayout>

                <!-- Timer Badge -->
                <com.google.android.material.card.MaterialCardView
                    android:layout_width="56dp"
                    android:layout_height="56dp"
                    app:cardCornerRadius="28dp"
                    app:cardElevation="4dp"
                    app:cardBackgroundColor="@color/background_surface">
```

```

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical"
            android:gravity="center">

            <TextView
                android:id="@+id/timerTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="30"
                android:textSize="20sp"
                android:textStyle="bold"
                android:textColor="@color/deep_orange"/>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="sec"
                android:textSize="10sp"
                android:textColor="@color/text_hint"/>

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>
</LinearLayout>
</FrameLayout>

<!-- Question Content -->
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:padding="16dp"
    android:clipToPadding="false">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Question Card -->
        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="20dp"
            app:cardCornerRadius="20dp"
            app:cardElevation="6dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="20dp">

                <LinearLayout
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:orientation="horizontal"
                    android:gravity="center_vertical"
                    android:background="@drawable/card_background_green"
                    android:paddingHorizontal="12dp"
                    android:paddingVertical="6dp"
                    android:layout_marginBottom="16dp">

                    <ImageView
                        android:layout_width="16dp"
                        android:layout_height="16dp"
                        android:src="@drawable/ic_quiz"
                        app:tint="@color/green_dark"
                        android:contentDescription="Question"/>

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Question"
                        android:textSize="12sp"
                        android:textColor="@color/green_dark"
                        android:textStyle="bold"
                        android:layout_marginStart="6dp"/>

                </LinearLayout>

                <TextView
                    android:id="@+id/questionTextView"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"

```

```

        android:text="Question text will appear here"
        android:textSize="18sp"
        android:textColor="@color/text_primary"
        android:lineSpacingExtra="4dp"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Answer Options Card -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp">

    <RadioGroup
        android:id="@+id/answersRadioGroup"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="8dp">

        <RadioButton
            android:id="@+id/option1RadioButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textSize="15sp"
            android:text="Option 1"
            android:background="?attr/selectableItemBackground"/>

        <View
            android:layout_width="match_parent"
            android:layout_height="1dp"
            android:background="@color/divider"
            android:layout_marginHorizontal="16dp"/>

        <RadioButton
            android:id="@+id/option2RadioButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textSize="15sp"
            android:text="Option 2"
            android:background="?attr/selectableItemBackground"/>

        <View
            android:layout_width="match_parent"
            android:layout_height="1dp"
            android:background="@color/divider"
            android:layout_marginHorizontal="16dp"/>

        <RadioButton
            android:id="@+id/option3RadioButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textSize="15sp"
            android:text="Option 3"
            android:background="?attr/selectableItemBackground"/>

        <View
            android:layout_width="match_parent"
            android:layout_height="1dp"
            android:background="@color/divider"
            android:layout_marginHorizontal="16dp"/>

        <RadioButton
            android:id="@+id/option4RadioButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textSize="15sp"
            android:text="Option 4"
            android:background="?attr/selectableItemBackground"/>

    </RadioGroup>
</com.google.android.material.card.MaterialCardView>

<!-- Feedback Card -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/feedbackCard"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"

```

```

        android:visibility="gone"
        app:cardCornerRadius="12dp"
        app:cardElevation="2dp"
        android:layout_marginBottom="16dp">

        <TextView
            android:id="@+id/feedbackTextView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text=""
            android:textSize="15sp"
            android:textStyle="bold"
            android:padding="16dp"/>
        </com.google.android.material.card.MaterialCardView>

    </LinearLayout>
</ScrollView>

<!-- Bottom Button -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="@android:color/white"
    android:elevation="8dp">

    <Button
        android:id="@+id/submitButton"
        android:layout_width="match_parent"
        android:layout_height="56dp"
        android:text="Submit Answer"
        android:textAllCaps="false"
        android:textSize="16sp"
        android:background="@drawable/rounded_button_green"
        android:textColor="@android:color/white"/>

    <Button
        android:id="@+id/nextButton"
        android:layout_width="match_parent"
        android:layout_height="56dp"
        android:text="Next Question"
        android:textAllCaps="false"
        android:textSize="16sp"
        android:background="@drawable/rounded_button_blue"
        android:textColor="@android:color/white"
        android:visibility="gone"/>
    </LinearLayout>

</LinearLayout>

<!-- Doodle overlay -->
<com.example.phiz.views.DoodleView
    android:id="@+id/doodleView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>

<!-- Doodle toggle button -->
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/doodleToggleButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_margin="16dp"
    android:layout_marginBottom="90dp"
    android:src="@drawable/ic_edit"
    android:contentDescription="Toggle doodle"
    app:fabSize="mini"
    app:backgroundTint="@color/purple_primary"/>

<!-- Clear doodle button -->
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/clearDoodleButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_margin="16dp"
    android:layout_marginBottom="150dp"
    android:src="@drawable/ic_clear"
    android:contentDescription="Clear doodle"
    android:visibility="gone"
    app:fabSize="mini"
    app:backgroundTint="@color/red_primary"/>

```

</FrameLayout>

activity_register.xml

app/src/main/res/layout/activity_register.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    android:background="@drawable/gradient_login_bg">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="24dp"
        android:gravity="center">

        <!-- Header Icon -->
        <FrameLayout
            android:layout_width="100dp"
            android:layout_height="100dp"
            android:layout_marginTop="20dp"
            android:layout_marginBottom="16dp">

            <View
                android:layout_width="100dp"
                android:layout_height="100dp"
                android:background="@drawable/circle_green"/>

            <ImageView
                android:layout_width="50dp"
                android:layout_height="50dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_person"
                android:contentDescription="Person icon"/>
        </FrameLayout>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Create Account"
            android:textSize="28sp"
            android:textStyle="bold"
            android:textColor="@color/green_dark"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Join us to start learning physics"
            android:textSize="14sp"
            android:textColor="@color/text_tertiary"
            android:layout_marginBottom="24dp"/>

        <!-- Registration Card -->
        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardCornerRadius="20dp"
            app:cardElevation="8dp"
            android:layout_marginBottom="16dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="24dp">

                <com.google.android.material.textfield.TextInputLayout
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginBottom="16dp"
                    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
                    app:boxCornerRadiusBottomEnd="12dp"
                    app:boxCornerRadiusBottomStart="12dp"
                    app:boxCornerRadiusTopEnd="12dp"
                    app:boxCornerRadiusTopStart="12dp"
                    app:startIconDrawable="@drawable/ic_person">

                    <com.google.android.material.textfield.TextInputEditText
                        android:id="@+id/nameEditText"
                        android:layout_width="match_parent"
                        android:layout_height="wrap_content">
```

```

        android:hint="Full Name"
        android:inputType="textPersonName"/>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    app:boxCornerRadiusBottomEnd="12dp"
    app:boxCornerRadiusBottomStart="12dp"
    app:boxCornerRadiusTopEnd="12dp"
    app:boxCornerRadiusTopStart="12dp"
    app:startIconDrawable="@drawable/ic_email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/emailEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email Address"
        android:inputType="textEmailAddress"/>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="20dp"
    style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    app:boxCornerRadiusBottomEnd="12dp"
    app:boxCornerRadiusBottomStart="12dp"
    app:boxCornerRadiusTopEnd="12dp"
    app:boxCornerRadiusTopStart="12dp"
    app:endIconMode="password_toggle"
    app:startIconDrawable="@drawable/ic_lock">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/passwordEditText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password (min 6 characters)"
        android:inputType="textPassword"/>
</com.google.android.material.textfield.TextInputLayout>

<!-- Role Selection -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="I am a..."
    android:textSize="16sp"
    android:textStyle="bold"
    android:textColor="@color/text_primary"
    android:layout_marginBottom="12dp"/>

<LinearLayout
    android:id="@+id/roleRadioGroup"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="24dp">

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/studentCard"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="8dp"
        android:clickable="true"
        android:focusable="true"
        app:cardCornerRadius="12dp"
        app:cardElevation="4dp"
        app:strokeColor="@color/green_primary"
        app:strokeWidth="2dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:gravity="center"
            android:padding="16dp">

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:src="@drawable/ic_person"

```

```

        app:tint="@color/green_primary"
        android:contentDescription="Student icon"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Student"
            android:textSize="15sp"
            android:textColor="@color/text_primary"
            android:layout_marginTop="8dp"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:id="@+id/teacherCard"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:layout_marginStart="8dp"
    android:clickable="true"
    android:focusable="true"
    app:cardCornerRadius="12dp"
    app:cardElevation="2dp"
    app:strokeColor="@color/divider"
    app:strokeWidth="1dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:gravity="center"
        android:padding="16dp">

        <ImageView
            android:layout_width="32dp"
            android:layout_height="32dp"
            android:src="@drawable/ic_students"
            app:tint="@color/blue_primary"
            android:contentDescription="Teacher icon"/>

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Teacher"
            android:textSize="15sp"
            android:textColor="@color/text_primary"
            android:layout_marginTop="8dp"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<Button
    android:id="@+id/registerButton"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:text="Create Account"
    android:textSize="16sp"
    android:textAllCaps="false"
    android:background="@drawable/rounded_button_green"
    android:textColor="@android:color/white"/>

</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Login Link -->
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Already have an account? "
        android:textSize="15sp"
        android:textColor="@color/text_tertiary"/>

    <TextView
        android:id="@+id/loginTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sign In"
        android:textSize="15sp"

```



```
        android:textColor="@color/green_medium"
        android:textStyle="bold"
        android:clickable="true"
        android:focusable="true"
        android:padding="8dp"/>
</LinearLayout>

<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_marginTop="16dp"
    android:visibility="gone"
    android:indeterminateTint="@color/green_primary"/>

</LinearLayout>
</ScrollView>
```

activity_student_home.xml

app/src/main/res/layout/activity_student_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Header with gradient -->
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <View
            android:layout_width="match_parent"
            android:layout_height="200dp"
            android:background="@drawable/gradient_blue"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:gravity="center_vertical">

                <!-- Profile Icon -->
                <FrameLayout
                    android:layout_width="50dp"
                    android:layout_height="50dp">

                    <View
                        android:layout_width="50dp"
                        android:layout_height="50dp"
                        android:background="@drawable/circle_green"/>

                    <ImageView
                        android:layout_width="28dp"
                        android:layout_height="28dp"
                        android:layout_gravity="center"
                        android:src="@drawable/ic_person"
                        android:contentDescription="Profile"/>

                </FrameLayout>

                <LinearLayout
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:orientation="vertical"
                    android:layout_marginStart="12dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Hello, "
                        android:textSize="14sp"
                        android:textColor="@color/text_on_gradient"/>

                    <TextView
                        android:id="@+id/welcomeTextView"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Student"
                        android:textColor="@android:color/white"
                        android:textSize="22sp"
                        android:textStyle="bold"/>

                </LinearLayout>

            <!-- Settings Button -->
            <com.google.android.material.card.MaterialCardView
                android:id="@+id/settingsButton"
                android:layout_width="44dp"
                android:layout_height="44dp"
                android:layout_marginEnd="8dp"
                app:cardCornerRadius="22dp"
                app:cardElevation="4dp">
```

```

        app:cardBackgroundColor="@color/background_surface"
        android:clickable="true"
        android:focusable="true">

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_notification"
            app:tint="@color/blue_primary"
            android:contentDescription="Settings"/>
    </com.google.android.material.card.MaterialCardView>

    <!-- Logout Button -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/logoutButton"
        android:layout_width="44dp"
        android:layout_height="44dp"
        app:cardCornerRadius="22dp"
        app:cardElevation="4dp"
        app:cardBackgroundColor="@color/background_surface"
        android:clickable="true"
        android:focusable="true">

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_logout"
            app:tint="@color/red_primary"
            android:contentDescription="Logout"/>
    </com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Score Card -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="8dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="20dp"
        android:gravity="center_vertical">

        <FrameLayout
            android:layout_width="56dp"
            android:layout_height="56dp">

            <View
                android:layout_width="56dp"
                android:layout_height="56dp"
                android:background="@drawable/circle_orange"/>

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_trophy"
                android:contentDescription="Trophy"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Your Total Score"
                android:textSize="14sp"
                android:textColor="@color/text_hint"/>

            <TextView
                android:id="@+id/scoreTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"

```

```

        android:text="0 points"
        android:textSize="28sp"
        android:textStyle="bold"
        android:textColor="@color/orange_primary"/>
    </LinearLayout>

    <ImageView
        android:layout_width="32dp"
        android:layout_height="32dp"
        android:src="@drawable/ic_star"
        android:contentDescription="Star"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</FrameLayout>

<!-- Menu Cards -->
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:clipToPadding="false"
    android:padding="16dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Physics Simulation Card -->
        <com.google.android.material.card.MaterialCardView
            android:id="@+id/simulationCard"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            android:clickable="true"
            android:focusable="true"
            app:cardCornerRadius="20dp"
            app:cardElevation="6dp"
            app:rippleColor="@color/blue_ripple">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:padding="20dp"
                android:gravity="center_vertical">

                <FrameLayout
                    android:layout_width="64dp"
                    android:layout_height="64dp">

                    <View
                        android:layout_width="64dp"
                        android:layout_height="64dp"
                        android:background="@drawable/circle_blue"/>

                    <ImageView
                        android:layout_width="32dp"
                        android:layout_height="32dp"
                        android:layout_gravity="center"
                        android:src="@drawable/ic_physics"
                        android:contentDescription="Physics"/>
                </FrameLayout>

                <LinearLayout
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:orientation="vertical"
                    android:layout_marginStart="16dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Physics Simulation"
                        android:textSize="20sp"
                        android:textStyle="bold"
                        android:textColor="@color/blue_dark"/>

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Explore Newton's Laws interactively"

```

```

        android:textSize="13sp"
        android:textColor="@color/text_hint"
        android:layout_marginTop="4dp"/>

<View
    android:layout_width="60dp"
    android:layout_height="4dp"
    android:layout_marginTop="12dp"
    android:background="@drawable/gradient_blue"/>
</LinearLayout>

<ImageView
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@drawable/ic_arrow_back"
    android:rotation="180"
    app:tint="@color/gray_icon"
    android:contentDescription="Go"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Quiz Card -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/quizCard"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="16dp"
    android:clickable="true"
    android:focusable="true"
    app:cardCornerRadius="20dp"
    app:cardElevation="6dp"
    app:rippleColor="@color/green_ripple">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="20dp"
        android:gravity="center_vertical">

        <FrameLayout
            android:layout_width="64dp"
            android:layout_height="64dp">

            <View
                android:layout_width="64dp"
                android:layout_height="64dp"
                android:background="@drawable/circle_green"/>

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_quiz"
                android:contentDescription="Quiz"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Take Quiz"
                android:textSize="20sp"
                android:textStyle="bold"
                android:textColor="@color/green_dark"/>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Test your knowledge and earn points"
                android:textSize="13sp"
                android:textColor="@color/text_hint"
                android:layout_marginTop="4dp"/>

            <View
                android:layout_width="60dp"
                android:layout_height="4dp"
                android:layout_marginTop="12dp"

```

```

        android:background="@drawable/gradient_green"/>
</LinearLayout>

<ImageView
    android:layout_width="24dp"
    android:layout_height="24dp"
    android:src="@drawable/ic_arrow_back"
    android:rotation="180"
    app:tint="@color/gray_icon"
    android:contentDescription="Go"/>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- Grades Card -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/gradesCard"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:clickable="true"
    android:focusable="true"
    app:cardCornerRadius="20dp"
    app:cardElevation="6dp"
    app:rippleColor="@color/orange_ripple">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="20dp"
        android:gravity="center_vertical">

        <FrameLayout
            android:layout_width="64dp"
            android:layout_height="64dp">

            <View
                android:layout_width="64dp"
                android:layout_height="64dp"
                android:background="@drawable/circle_orange"/>

            <ImageView
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_grades"
                android:contentDescription="Grades"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="View Grades"
                android:textSize="20sp"
                android:textStyle="bold"
                android:textColor="@color/orange_dark"/>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Check your quiz results and progress"
                android:textSize="13sp"
                android:textColor="@color/text_hint"
                android:layout_marginTop="4dp"/>

            <View
                android:layout_width="60dp"
                android:layout_height="4dp"
                android:layout_marginTop="12dp"
                android:background="@drawable/gradient_orange"/>
        </LinearLayout>

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:src="@drawable/ic_arrow_back"
            android:rotation="180"
            app:tint="@color/gray_icon"

```

```
        android:contentDescription="Go"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

</LinearLayout>
</ScrollView>

</LinearLayout>
```

activity_teacher_home.xml

app/src/main/res/layout/activity_teacher_home.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <!-- Header with gradient -->
    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <View
            android:layout_width="match_parent"
            android:layout_height="180dp"
            android:background="@drawable/gradient_purple"/>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="horizontal"
                android:gravity="center_vertical">

                <!-- Profile Icon -->
                <FrameLayout
                    android:layout_width="50dp"
                    android:layout_height="50dp">

                    <View
                        android:layout_width="50dp"
                        android:layout_height="50dp"
                        android:background="@drawable/circle_orange"/>

                    <ImageView
                        android:layout_width="28dp"
                        android:layout_height="28dp"
                        android:layout_gravity="center"
                        android:src="@drawable/ic_person"
                        android:contentDescription="Profile"/>

                </FrameLayout>

                <LinearLayout
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:orientation="vertical"
                    android:layout_marginStart="12dp">

                    <TextView
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Teacher Dashboard"
                        android:textSize="14sp"
                        android:textColor="@color/purple_light"/>

                    <TextView
                        android:id="@+id/welcomeTextView"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:text="Welcome"
                        android:textColor="@android:color/white"
                        android:textSize="22sp"
                        android:textStyle="bold"/>

                </LinearLayout>

            <!-- Settings Button -->
            <com.google.android.material.card.MaterialCardView
                android:id="@+id/settingsButton"
                android:layout_width="44dp"
                android:layout_height="44dp"
                android:layout_marginEnd="8dp"
                app:cardCornerRadius="22dp"
                app:cardElevation="4dp">
```



```

        app:cardBackgroundColor="@color/background_surface"
        android:clickable="true"
        android:focusable="true">

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_notification"
            app:tint="@color/blue_primary"
            android:contentDescription="Settings"/>
    </com.google.android.material.card.MaterialCardView>

    <!-- Logout Button -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/logoutButton"
        android:layout_width="44dp"
        android:layout_height="44dp"
        app:cardCornerRadius="22dp"
        app:cardElevation="4dp"
        app:cardBackgroundColor="@color/background_surface"
        android:clickable="true"
        android:focusable="true">

        <ImageView
            android:layout_width="24dp"
            android:layout_height="24dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_logout"
            app:tint="@color/red_primary"
            android:contentDescription="Logout"/>

    </com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Action Buttons Row 1 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="20dp">

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/createQuestionButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="8dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        android:clickable="true"
        android:focusable="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp"
        android:gravity="center">

        <FrameLayout
            android:layout_width="48dp"
            android:layout_height="48dp">

            <View
                android:layout_width="48dp"
                android:layout_height="48dp"
                android:background="@drawable/circle_green"/>

            <ImageView
                android:layout_width="24dp"
                android:layout_height="24dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_add"
                android:contentDescription="Add"/>

        </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Create Question"
        android:textSize="13sp"
        android:textStyle="bold"
        android:textColor="@color/green_dark"
        android:layout_marginTop="8dp"/>

```

```

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        android:id="@+id/viewQuestionsButton"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="8dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        android:clickable="true"
        android:focusable="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp"
            android:gravity="center">

            <FrameLayout
                android:layout_width="48dp"
                android:layout_height="48dp">

                <View
                    android:layout_width="48dp"
                    android:layout_height="48dp"
                    android:background="@drawable/circle_orange"/>

                <ImageView
                    android:layout_width="24dp"
                    android:layout_height="24dp"
                    android:layout_gravity="center"
                    android:src="@drawable/ic_list"
                    android:contentDescription="List"/>
            </FrameLayout>

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="View Questions"
                android:textSize="13sp"
                android:textStyle="bold"
                android:textColor="@color/orange_dark"
                android:layout_marginTop="8dp"/>
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- Action Buttons Row 2 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="12dp">

    <!-- All Grades Button -->
    <com.google.android.material.card.MaterialCardView
        android:id="@+id/allGradesButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        android:clickable="true"
        android:focusable="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp"
            android:gravity="center">

            <FrameLayout
                android:layout_width="48dp"
                android:layout_height="48dp">

                <View
                    android:layout_width="48dp"
                    android:layout_height="48dp"
                    android:background="@drawable/circle_blue"/>

                <ImageView

```

```

        android:layout_width="24dp"
        android:layout_height="24dp"
        android:layout_gravity="center"
        android:src="@drawable/ic_grades"
        android:contentDescription="Grades"/>
    </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="View All Grades"
        android:textSize="13sp"
        android:textStyle="bold"
        android:textColor="@color/blue_dark"
        android:layout_marginTop="8dp"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</LinearLayout>
</FrameLayout>

<!-- Students Section Header -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center_vertical"
    android:paddingHorizontal="20dp"
    android:paddingVertical="16dp">

    <FrameLayout
        android:layout_width="36dp"
        android:layout_height="36dp">

        <View
            android:layout_width="36dp"
            android:layout_height="36dp"
            android:background="@drawable/circle_blue"/>

        <ImageView
            android:layout_width="20dp"
            android:layout_height="20dp"
            android:layout_gravity="center"
            android:src="@drawable/ic_students"
            android:contentDescription="Students"/>
    </FrameLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Students"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="@color/text_primary"
        android:layout_marginStart="12dp"/>
</LinearLayout>

<!-- Students List -->
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/studentsRecyclerView"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:paddingHorizontal="16dp"
    android:clipToPadding="false"/>

<!-- Empty State -->
<LinearLayout
    android:id="@+id/emptyStateLayout"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:orientation="vertical"
    android:gravity="center"
    android:visibility="gone"
    android:padding="32dp">

    <FrameLayout
        android:layout_width="80dp"
        android:layout_height="80dp">

        <View
            android:layout_width="80dp"
            android:layout_height="80dp"

```

```
        android:background="@drawable/circle_blue"
        android:alpha="0.2"/>
```

```
    <ImageView
        android:layout_width="40dp"
        android:layout_height="40dp"
        android:layout_gravity="center"
        android:src="@drawable/ic_students"
        app:tint="@color/gray_icon"
        android:contentDescription="No students"/>
```

```
</FrameLayout>
```

```
<TextView
    android:id="@+id/emptyTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="No students yet"
    android:textSize="18sp"
    android:textColor="@color/text_hint"
    android:layout_marginTop="16dp"/>
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Students will appear here once they register"
    android:textSize="14sp"
    android:textColor="@color/gray_icon"
    android:layout_marginTop="8dp"
    android:gravity="center"/>
```

```
</LinearLayout>
```

```
</LinearLayout>
```

activity_view_questions.xml

app/src/main/res/layout/activity_view_questions.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background_primary">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="?attr/actionBarSize"
        android:background="@color/orange_primary"
        android:elevation="4dp"
        app:navigationIcon="@drawable/ic_arrow_back"
        app:navigationIconTint="@android:color/white"
        app:titleTextColor="@android:color/white">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Questions Database"
            android:textColor="@android:color/white"
            android:textSize="20sp"
            android:textStyle="bold"/>
    </com.google.android.material.appbar.MaterialToolbar>

    <ProgressBar
        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="16dp"
        android:visibility="gone"/>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/questionsRecyclerView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:padding="16dp"/>

    <TextView
        android:id="@+id/emptyTextView"
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:layout_gravity="center"
        android:text="No questions yet. Create your first question!"
        android:textSize="16sp"
        android:textColor="@color/text_disabled"
        android:gravity="center"
        android:visibility="gone"/>

</LinearLayout>
```

item_all_grades.xml

app/src/main/res/layout/item_all_grades.xml

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp"
        android:gravity="center_vertical">

        <!-- User Avatar -->
        <FrameLayout
            android:layout_width="52dp"
            android:layout_height="52dp">

            <View
                android:layout_width="52dp"
                android:layout_height="52dp"
                android:background="@drawable/circle_blue"/>

            <ImageView
                android:layout_width="28dp"
                android:layout_height="28dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_person"
                android:contentDescription="Student"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:id="@+id/userNameTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Student Name"
                android:textSize="16sp"
                android:textStyle="bold"
                android:textColor="@color/text_primary"/>

            <TextView
                android:id="@+id/quizNameTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Quiz Name"
                android:textSize="14sp"
                android:textColor="@color/text_tertiary"
                android:layout_marginTop="2dp"/>

            <TextView
                android:id="@+id/dateTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Date"
                android:textSize="12sp"
                android:textColor="@color/text_hint"
                android:layout_marginTop="2dp"/>

            <TextView
                android:id="@+id/scoreTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Score: 80/100"
                android:textSize="14sp"
                android:textColor="@color/green_primary"
                android:textStyle="bold"
                android:layout_marginTop="6dp"/>
        </LinearLayout>

        <!-- Percentage Badge -->
```

```

<FrameLayout
    android:layout_width="60dp"
    android:layout_height="60dp">

    <View
        android:layout_width="60dp"
        android:layout_height="60dp"
        android:background="@drawable/circle_green"/>

    <TextView
        android:id="@+id/percentageTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:text="80%"
        android:textSize="16sp"
        android:textColor="@android:color/white"
        android:textStyle="bold"/>
    </FrameLayout>
</LinearLayout>

</com.google.android.material.card.MaterialCardView>

```

item_grade.xml

app/src/main/res/layout/item_grade.xml

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp"
        android:gravity="center_vertical">

        <!-- Quiz Icon -->
        <FrameLayout
            android:layout_width="52dp"
            android:layout_height="52dp">

            <View
                android:layout_width="52dp"
                android:layout_height="52dp"
                android:background="@drawable/circle_green"/>

            <ImageView
                android:layout_width="28dp"
                android:layout_height="28dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_quiz"
                android:contentDescription="Quiz"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:id="@+id/quizNameTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Quiz Name"
                android:textSize="16sp"
                android:textStyle="bold"
                android:textColor="@color/text_primary"/>

            <TextView
                android:id="@+id/dateTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Date"
                android:textSize="12sp"
                android:textColor="@color/text_hint"
                android:layout_marginTop="2dp"/>

            <TextView
                android:id="@+id/scoreTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Score: 80/100"
                android:textSize="14sp"
                android:textColor="@color/green_primary"
                android:textStyle="bold"
                android:layout_marginTop="6dp"/>
        </LinearLayout>

        <!-- Percentage Badge -->
        <FrameLayout
            android:layout_width="60dp"
            android:layout_height="60dp">

            <View
                android:layout_width="60dp"
                android:layout_height="60dp"
                android:background="@drawable/circle_orange"/>
        </FrameLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```



```
        <TextView
            android:id="@+id/percentageTextView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:text="80%"
            android:textSize="16sp"
            android:textColor="@android:color/white"
            android:textStyle="bold"/>
    </FrameLayout>
</LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

item_question.xml

app/src/main/res/layout/item_question.xml

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    app:cardCornerRadius="8dp"
    app:cardElevation="2dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:layout_marginBottom="8dp">

            <TextView
                android:id="@+id/difficultyBadge"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="medium"
                android:textSize="12sp"
                android:textColor="@android:color/white"
                android:background="@drawable/difficulty_badge_background"
                android:paddingStart="8dp"
                android:paddingEnd="8dp"
                android:paddingTop="2dp"
                android:paddingBottom="2dp"/>

            <TextView
                android:id="@+id/pointsTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginStart="8dp"
                android:text="20 pts"
                android:textSize="12sp"
                android:textColor="@color/text_tertiary"/>

            <View
                android:layout_width="0dp"
                android:layout_height="0dp"
                android:layout_weight="1"/>

            <ImageButton
                android:id="@+id/deleteButton"
                android:layout_width="32dp"
                android:layout_height="32dp"
                android:background="?attr/selectableItemBackgroundBorderless"
                android:src="@drawable/ic_delete"
                android:contentDescription="Delete question"
                app:tint="@color/red_primary"/>
        </LinearLayout>

        <TextView
            android:id="@+id/questionTextView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Question text goes here"
            android:textSize="16sp"
            android:textColor="@color/text_primary"
            android:textStyle="bold"
            android:layout_marginBottom="12dp"/>

        <TextView
            android:id="@+id/optionsLabel"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Options:"
            android:textSize="12sp"
            android:textColor="@color/text_tertiary"
            android:layout_marginBottom="4dp"/>

        <TextView
            android:id="@+id/option1TextView"
            android:layout_width="match_parent"
```

```

        android:layout_height="wrap_content"
        android:text="1. Option 1"
        android:textSize="14sp"
        android:textColor="@color/text_secondary_dark"/>

<TextView
    android:id="@+id/option2TextView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="2. Option 2"
    android:textSize="14sp"
    android:textColor="@color/text_secondary_dark"/>

<TextView
    android:id="@+id/option3TextView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="3. Option 3"
    android:textSize="14sp"
    android:textColor="@color/text_secondary_dark"/>

<TextView
    android:id="@+id/option4TextView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="4. Option 4"
    android:textSize="14sp"
    android:textColor="@color/text_secondary_dark"/>

<TextView
    android:id="@+id/correctAnswerTextView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Correct: Option 1"
    android:textSize="14sp"
    android:textColor="@color/green_primary"
    android:textStyle="bold"/>

</LinearLayout>
</com.google.android.material.card.MaterialCardView>

```

item_student.xml

app/src/main/res/layout/item_student.xml

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    app:rippleColor="@color/blue_ripple">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp"
        android:gravity="center_vertical">

        <!-- Avatar -->
        <FrameLayout
            android:layout_width="52dp"
            android:layout_height="52dp">

            <View
                android:layout_width="52dp"
                android:layout_height="52dp"
                android:background="@drawable/circle_blue"/>

            <ImageView
                android:layout_width="28dp"
                android:layout_height="28dp"
                android:layout_gravity="center"
                android:src="@drawable/ic_person"
                android:contentDescription="Student"/>
        </FrameLayout>

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical"
            android:layout_marginStart="16dp">

            <TextView
                android:id="@+id/studentNameTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Student Name"
                android:textSize="17sp"
                android:textStyle="bold"
                android:textColor="@color/text_primary"/>

            <TextView
                android:id="@+id/studentEmailTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="student@email.com"
                android:textSize="13sp"
                android:textColor="@color/text_hint"
                android:layout_marginTop="2dp"/>
        </LinearLayout>

        <!-- Score Badge -->
        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:gravity="center"
            android:background="@drawable/card_background_orange"
            android:paddingHorizontal="12dp"
            android:paddingVertical="8dp">

            <ImageView
                android:layout_width="20dp"
                android:layout_height="20dp"
                android:src="@drawable/ic_star"
                android:contentDescription="Score"/>

            <TextView
                android:id="@+id/studentScoreTextView"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Score" />
        </LinearLayout>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

```
        android:layout_height="wrap_content"
        android:text="0 pts"
        android:textSize="12sp"
        android:textColor="@color/orange_dark"
        android:textStyle="bold"
        android:layout_marginTop="2dp"/>
    </LinearLayout>
</LinearLayout>

</com.google.android.material.card.MaterialCardView>
```

ic_launcher.xml

app/src/main/res/mipmap-anydpi/ic_launcher.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@drawable/ic_launcher_background" />
  <foreground android:drawable="@drawable/ic_launcher_foreground" />
  <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

ic_launcher_round.xml

app/src/main/res/mipmap-anydpi/ic_launcher_round.xml

```
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@drawable/ic_launcher_background" />
  <foreground android:drawable="@drawable/ic_launcher_foreground" />
  <monochrome android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

colors.xml

app/src/main/res/values-night/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Backgrounds -->
    <color name="background_primary">#121212</color>
    <color name="background_surface">#1E1E1E</color>

    <!-- Text (lighter for dark backgrounds) -->
    <color name="text_primary">#E0E0E0</color>
    <color name="text_secondary_dark">#BDBDBD</color>
    <color name="text_secondary">#B0B0B0</color>
    <color name="text_tertiary">#9E9E9E</color>
    <color name="text_hint">#8A8A8A</color>
    <color name="text_disabled">#757575</color>
    <color name="text_placeholder">#6E6E6E</color>
    <color name="text_on_gradient">#B3FFFFFF</color>
    <color name="text_on_dark">#FFFFFF</color>

    <!-- Divider -->
    <color name="divider">#333333</color>

    <!-- Blue (lighter/desaturated for dark backgrounds) -->
    <color name="blue_primary">#64B5F6</color>
    <color name="blue_medium">#42A5F5</color>
    <color name="blue_dark">#90CAF9</color>
    <color name="blue_tint_bg">#1A3A5C</color>
    <color name="blue_light">#1A4A6C</color>
    <color name="blue_ripple">#1A64B5F6</color>

    <!-- Green -->
    <color name="green_primary">#81C784</color>
    <color name="green_medium">#66BB6A</color>
    <color name="green_dark">#A5D6A7</color>
    <color name="green_tint_bg">#1A3C2A</color>
    <color name="green_light">#2E4A30</color>
    <color name="green_ripple">#1A81C784</color>

    <!-- Orange -->
    <color name="orange_primary">#FFB74D</color>
    <color name="orange_medium">#FFA726</color>
    <color name="orange_dark">#FFCC80</color>
    <color name="orange_tint_bg">#3C2A1A</color>
    <color name="orange_light">#4A3520</color>
    <color name="deep_orange">#FF8A65</color>
    <color name="orange_ripple">#1AFFB74D</color>

    <!-- Purple -->
    <color name="purple_primary">#CE93D8</color>
    <color name="purple_dark">#BA68C8</color>
    <color name="purple_light">#3A2A4A</color>

    <!-- Red -->
    <color name="red_primary">#EF5350</color>
    <color name="red_medium">#E57373</color>
    <color name="red_dark">#EF9A9A</color>

    <!-- Gray (inverted for dark mode) -->
    <color name="gray_icon">#757575</color>
    <color name="gray_medium">#9E9E9E</color>
    <color name="gray_dark">#BDBDBD</color>
    <color name="gray_darker">#E0E0E0</color>

    <!-- Special -->
    <color name="brown_surface">#A1887F</color>
</resources>
```


themes.xml

app/src/main/res/values-night/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Phiz" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="colorPrimary">@color/blue_primary</item>
        <item name="colorOnPrimary">@color/text_on_dark</item>
        <item name="colorPrimaryContainer">@color/blue_tint_bg</item>
        <item name="colorSurface">@color/background_surface</item>
        <item name="colorOnSurface">@color/text_primary</item>
        <item name="colorSurfaceVariant">@color/background_primary</item>
        <item name="android:colorBackground">@color/background_primary</item>
        <item name="android:textColorPrimary">@color/text_primary</item>
        <item name="android:textColorSecondary">@color/text_tertiary</item>
        <item name="android:textColorHint">@color/text_hint</item>
    </style>
</resources>
```

colors.xml

app/src/main/res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFF</color>

    <!-- Backgrounds -->
    <color name="background_primary">#F5F5F5</color>
    <color name="background_surface">#FFFFFF</color>

    <!-- Text -->
    <color name="text_primary">#333333</color>
    <color name="text_secondary_dark">#444444</color>
    <color name="text_secondary">#555555</color>
    <color name="text_tertiary">#666666</color>
    <color name="text_hint">#888888</color>
    <color name="text_disabled">#999999</color>
    <color name="text_placeholder">#AAAAAA</color>
    <color name="text_on_gradient">#B3FFFFFF</color>
    <color name="text_on_dark">#FFFFFF</color>

    <!-- Divider -->
    <color name="divider">#E0E0E0</color>

    <!-- Blue -->
    <color name="blue_primary">#2196F3</color>
    <color name="blue_medium">#1976D2</color>
    <color name="blue_dark">#1565C0</color>
    <color name="blue_tint_bg">#E3F2FD</color>
    <color name="blue_light">#BBDEFB</color>
    <color name="blue_ripple">#1A2196F3</color>

    <!-- Green -->
    <color name="green_primary">#4CAF50</color>
    <color name="green_medium">#388E3C</color>
    <color name="green_dark">#2E7D32</color>
    <color name="green_tint_bg">#E8F5E9</color>
    <color name="green_light">#C8E6C9</color>
    <color name="green_ripple">#1A4CAF50</color>

    <!-- Orange -->
    <color name="orange_primary">#FF9800</color>
    <color name="orange_medium">#F57C00</color>
    <color name="orange_dark">#E65100</color>
    <color name="orange_tint_bg">#FFF3E0</color>
    <color name="orange_light">#FFE0B2</color>
    <color name="deep_orange">#FF5722</color>
    <color name="orange_ripple">#1AFF9800</color>

    <!-- Purple -->
    <color name="purple_primary">#9C27B0</color>
    <color name="purple_dark">#6A1B9A</color>
    <color name="purple_light">#D1C4E9</color>

    <!-- Red -->
    <color name="red_primary">#F44336</color>
    <color name="red_medium">#D32F2F</color>
    <color name="red_dark">#C62828</color>

    <!-- Gray -->
    <color name="gray_icon">#BDBDBD</color>
    <color name="gray_medium">#757575</color>
    <color name="gray_dark">#616161</color>
    <color name="gray_darker">#424242</color>

    <!-- Special -->
    <color name="brown_surface">#795548</color>
</resources>
```

strings.xml

app/src/main/res/values/strings.xml

```
<resources>  
  <string name="app_name">phiz</string>  
</resources>
```

themes.xml

app/src/main/res/values/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Phiz" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="colorPrimary">@color/blue_primary</item>
        <item name="colorOnPrimary">@color/text_on_dark</item>
        <item name="colorPrimaryContainer">@color/blue_tint_bg</item>
        <item name="colorSurface">@color/background_surface</item>
        <item name="colorOnSurface">@color/text_primary</item>
        <item name="colorSurfaceVariant">@color/background_primary</item>
        <item name="android:colorBackground">@color/background_primary</item>
        <item name="android:textColorPrimary">@color/text_primary</item>
        <item name="android:textColorSecondary">@color/text_tertiary</item>
        <item name="android:textColorHint">@color/text_hint</item>
    </style>

    <style name="Theme.Phiz" parent="Base.Theme.Phiz" />
</resources>
```

backup_rules.xml

app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
    Sample backup rules file; uncomment and customize as necessary.
    See https://developer.android.com/guide/topics/data/autobackup
    for details.
    Note: This file is ignored for devices older than API 31
    See https://developer.android.com/about/versions/12/backup-restore
-->
<full-backup-content>
    <!--
        <include domain="sharedpref" path="."/>
        <exclude domain="sharedpref" path="device.xml"/>
    -->
</full-backup-content>
```

data_extraction_rules.xml

app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?><!--
  Sample data extraction rules file; uncomment and customize as necessary.
  See https://developer.android.com/about/versions/12/backup-restore#xml-changes
  for details.
-->
<data-extraction-rules>
  <cloud-backup>
    <!-- TODO: Use <include> and <exclude> to control what is backed up.
    <include .../>
    <exclude .../>
    -->
  </cloud-backup>
  <!--
  <device-transfer>
    <include .../>
    <exclude .../>
  </device-transfer>
  -->
</data-extraction-rules>
```

ExampleUnitTest.java

app/src/test/java/com/example/phiz/ExampleUnitTest.java

```
package com.example.phiz;

import org.junit.Test;

import static org.junit.Assert.*;

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * @see <a href="http://d.android.com/tools/testing">Testing documentation</a>
 */
public class ExampleUnitTest {
    @Test
    public void addition_isCorrect() {
        assertEquals(4, 2 + 2);
    }
}
```

build.gradle.kts

build.gradle.kts

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.google.gms.google.services) apply false
}
```


firebase.json

firebase.json

```
{
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  },
  "functions": {
    "source": "functions",
    "codebase": "default",
    "ignore": [
      "node_modules",
      ".git",
      "firebase-debug.log",
      "firebase-debug.*.log",
      "**.local"
    ],
    "predeploy": [
      "npm --prefix \"$RESOURCE_DIR\" run lint"
    ]
  }
}
```

firebase.indexes.json

firebase.indexes.json

```
{
  "indexes": [
    {
      "collectionGroup": "users",
      "queryScope": "COLLECTION",
      "fields": [
        { "fieldPath": "role", "order": "ASCENDING" },
        { "fieldPath": "totalScore", "order": "DESCENDING" }
      ]
    }
  ],
  "fieldOverrides": []
}
```

firebase.rules

firebase.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Helper function to check if the current user is a teacher
    function isTeacher() {
      return request.auth != null &&
        get(/databases/{database}/documents/users/{request.auth.uid}).data.role == 'teacher';
    }

    // Users collection
    match /users/{userId} {
      // Users can read/write their own documents
      allow read, write: if request.auth != null && request.auth.uid == userId;

      // Teachers can read all user documents for viewing student progress
      allow read: if isTeacher();

      // Grades subcollection
      match /grades/{gradeId} {
        // Users can read/write their own grades
        allow read, write: if request.auth != null && request.auth.uid == userId;

        // Teachers can read all student grades
        allow read: if isTeacher();
      }
    }

    // Questions collection - teachers can create/edit, students can read
    match /questions/{questionId} {
      allow read: if request.auth != null;
      allow create, update, delete: if isTeacher();
    }

    // Tests collection - teachers can manage, students can read
    match /tests/{testId} {
      allow read: if request.auth != null;
      allow create, update, delete: if isTeacher();
    }

    // Settings collection - teachers can manage, students can read
    match /settings/{settingId} {
      allow read: if request.auth != null;
      allow create, update, delete: if isTeacher();
    }

    // Notifications collection for storing notification history (optional)
    match /notifications/{notificationId} {
      allow read: if request.auth != null &&
        resource.data.userId == request.auth.uid;
      allow create: if request.auth != null;
    }
  }
}
```

index.js

functions/index.js

```
/**
 * Cloud Functions for Phiz Push Notifications
 *
 * These functions handle server-triggered notifications for:
 * - New question created (notify all students)
 * - Quiz completed (notify teachers)
 * - Low score alerts (notify teachers)
 * - Weekly digest (scheduled)
 */

const { onDocumentCreated } = require("firebase-functions/v2/firestore");
const { onSchedule } = require("firebase-functions/v2/scheduler");
const { initializeApp } = require("firebase-admin/app");
const { getFirestore } = require("firebase-admin/firestore");
const { getMessaging } = require("firebase-admin/messaging");

// Initialize Firebase Admin
initializeApp();

const db = getFirestore();
const messaging = getMessaging();

/**
 * Triggered when a new question is created.
 * Sends a notification to all students via the "all_students" topic.
 */
exports.onQuestionCreated = onDocumentCreated("questions/{questionId}", async (event) => {
  const questionData = event.data.data();
  const questionId = event.params.questionId;

  console.log(`New question created: ${questionId}`);

  // Build the notification message
  const message = {
    topic: "all_students",
    notification: {
      title: "New Question Available!",
      body: `A new ${questionData.difficulty} || "physics" question has been added. Test your knowledge!`,
    },
    data: {
      type: "new_content",
      content_type: "question",
      question_id: questionId,
    },
    android: {
      notification: {
        channelId: "quiz_notifications",
        priority: "high",
        clickAction: "OPEN_QUIZ",
      },
    },
  };

  try {
    const response = await messaging.send(message);
    console.log("Successfully sent notification to all_students topic:", response);
    return { success: true, messageId: response };
  } catch (error) {
    console.error("Error sending notification:", error);
    return { success: false, error: error.message };
  }
});

/**
 * Triggered when a new grade is created in a user's grades subcollection.
 * Notifies teachers when a student completes a quiz.
 */
exports.onGradeCreated = onDocumentCreated("users/{userId}/grades/{gradeId}", async (event) => {
  const gradeData = event.data.data();
  const userId = event.params.userId;

  console.log(`New grade created for user: ${userId}`);

  // Get user info for the notification
  const userDoc = await db.collection("users").doc(userId).get();
  if (!userDoc.exists) {
    console.log("User not found:", userId);
    return null;
  }
}
```

```

const userData = userDoc.data();

// Only notify for students
if (userData.role !== "student") {
  return null;
}

const studentName = userData.name || "A student";
const score = gradeData.score || 0;
const totalQuestions = gradeData.totalQuestions || 5;
const percentage = Math.round((score / (totalQuestions * 20)) * 100);

// Build notification for teachers
const message = {
  topic: "all_teachers",
  notification: {
    title: "Quiz Completed",
    body: `${studentName} completed a quiz with ${score} points (${percentage}%)\`,
  },
  data: {
    type: "quiz_completed",
    student_id: userId,
    student_name: studentName,
    score: String(score),
    percentage: String(percentage),
  },
  android: {
    notification: {
      channelId: "teacher_notifications",
      priority: "high",
    },
  },
};

try {
  const response = await messaging.send(message);
  console.log("Successfully notified teachers:", response);

  // Check for low score alert (below 50%)
  if (percentage < 50) {
    await sendLowScoreAlert(studentName, score, percentage);
  }

  return { success: true, messageId: response };
} catch (error) {
  console.error("Error sending notification:", error);
  return { success: false, error: error.message };
}
});

/**
 * Send a low score alert to teachers
 */
async function sendLowScoreAlert(studentName, score, percentage) {
  const message = {
    topic: "all_teachers",
    notification: {
      title: "Low Score Alert",
      body: `${studentName} scored ${percentage}% on their quiz. They may need additional help.\`,
    },
    data: {
      type: "low_score_alert",
      student_name: studentName,
      score: String(score),
      percentage: String(percentage),
    },
    android: {
      notification: {
        channelId: "teacher_notifications",
        priority: "high",
      },
    },
  };

  try {
    const response = await messaging.send(message);
    console.log("Low score alert sent:", response);
    return response;
  } catch (error) {
    console.error("Error sending low score alert:", error);
    throw error;
  }
}

```

```

/**
 * Weekly digest for teachers - runs every Sunday at 10:00 AM
 * Summarizes student activity for the week
 */
exports.weeklyTeacherDigest = onSchedule("every sunday 10:00", async (event) => {
  console.log("Running weekly teacher digest...");

  // Get all students
  const studentsSnapshot = await db.collection("users")
    .where("role", "=", "student")
    .get();

  if (studentsSnapshot.empty) {
    console.log("No students found");
    return null;
  }

  // Calculate weekly stats
  const oneWeekAgo = new Date();
  oneWeekAgo.setDate(oneWeekAgo.getDate() - 7);

  let totalQuizzes = 0;
  let totalScore = 0;
  let activeStudents = 0;

  for (const studentDoc of studentsSnapshot.docs) {
    const gradesSnapshot = await db.collection("users")
      .doc(studentDoc.id)
      .collection("grades")
      .where("timestamp", ">=", oneWeekAgo)
      .get();

    if (!gradesSnapshot.empty) {
      activeStudents++;
      totalQuizzes += gradesSnapshot.size;
      gradesSnapshot.forEach((gradeDoc) => {
        totalScore += gradeDoc.data().score || 0;
      });
    }
  }

  const avgScore = totalQuizzes > 0 ? Math.round(totalScore / totalQuizzes) : 0;

  // Send digest to teachers
  const message = {
    topic: "all_teachers",
    notification: {
      title: "Weekly Class Summary",
      body: `${activeStudents} students completed ${totalQuizzes} quizzes. Average score: ${avgScore} points.`,
    },
    data: {
      type: "weekly_digest",
      active_students: String(activeStudents),
      total_quizzes: String(totalQuizzes),
      average_score: String(avgScore),
    },
    android: {
      notification: {
        channelId: "teacher_notifications",
        priority: "default",
      },
    },
  },
  };

  try {
    const response = await messaging.send(message);
    console.log("Weekly digest sent:", response);
    return { success: true, messageId: response };
  } catch (error) {
    console.error("Error sending weekly digest:", error);
    return { success: false, error: error.message };
  }
});

/**
 * Inactivity reminder - runs daily at 6 PM
 * Checks for students who haven't used the app in 3+ days
 */
exports.inactivityReminder = onSchedule("every day 18:00", async (event) => {
  console.log("Running inactivity check...");

  const threeDaysAgo = new Date();
  threeDaysAgo.setDate(threeDaysAgo.getDate() - 3);

```

```

// Find inactive students
const inactiveSnapshot = await db.collection("users")
  .where("role", "==", "student")
  .where("lastActivityAt", "<", threeDaysAgo)
  .get();

console.log(`Found ${inactiveSnapshot.size} inactive students`);

// Send personalized reminders to each inactive student
const promises = inactiveSnapshot.docs.map(async (studentDoc) => {
  const studentData = studentDoc.data();
  const token = studentData.fcmToken;

  if (!token) {
    console.log(`No FCM token for user: ${studentDoc.id}`);
    return null;
  }

  // Calculate days since last activity
  const lastActivity = studentData.lastActivityAt ? studentData.lastActivityAt.toDate() : new Date(0);
  const daysSince = Math.floor((Date.now() - lastActivity.getTime()) / (1000 * 60 * 60 * 24));

  const message = {
    token: token,
    notification: {
      title: "We Miss You!",
      body: `It's been ${daysSince} days since your last visit. Come back and keep learning physics!`,
    },
    data: {
      type: "inactivity_reminder",
      days_inactive: String(daysSince),
    },
    android: {
      notification: {
        channelId: "reminder_notifications",
        priority: "default",
      },
    },
  };

  try {
    const response = await messaging.send(message);
    console.log(`Reminder sent to ${studentDoc.id}:`, response);
    return response;
  } catch (error) {
    console.error(`Error sending reminder to ${studentDoc.id}:`, error);
    return null;
  }
});

const results = await Promise.all(promises);
const successCount = results.filter((r) => r !== null).length;

console.log(`Sent ${successCount} inactivity reminders`);
return { success: true, sentCount: successCount };
});

/**
 * Send notification to a specific user by their FCM token
 * This can be called from client-side via HTTPS callable function
 */
exports.sendNotificationToUser = require("firebase-functions/v2/https").onCall(async (request) => {
  const { userId, title, body, data } = request.data;

  if (!userId || !title || !body) {
    throw new Error("Missing required parameters: userId, title, body");
  }

  // Get user's FCM token
  const userDoc = await db.collection("users").doc(userId).get();
  if (!userDoc.exists) {
    throw new Error("User not found");
  }

  const token = userDoc.data().fcmToken;
  if (!token) {
    throw new Error("User has no FCM token");
  }

  const message = {
    token: token,
    notification: {
      title: title,
      body: body,
    },
  };

```

```

    },
    data: data || {},
    android: {
      notification: {
        channelId: data?.channelId || "quiz_notifications",
        priority: "high",
      },
    },
  },
};

try {
  const response = await messaging.send(message);
  return { success: true, messageId: response };
} catch (error) {
  console.error("Error sending notification:", error);
  throw new Error(`Failed to send notification: ${error.message}`);
}
});

```


package.json

functions/package.json

```
{
  "name": "phiz-functions",
  "description": "Cloud Functions for Phiz push notifications",
  "version": "1.0.0",
  "engines": {
    "node": "20"
  },
  "main": "index.js",
  "scripts": {
    "lint": "eslint .",
    "serve": "firebase emulators:start --only functions",
    "shell": "firebase functions:shell",
    "start": "npm run shell",
    "deploy": "firebase deploy --only functions",
    "logs": "firebase functions:log"
  },
  "dependencies": {
    "firebase-admin": "^12.0.0",
    "firebase-functions": "^5.0.0"
  },
  "devDependencies": {
    "eslint": "^8.57.0",
    "eslint-config-google": "^0.14.0"
  },
  "private": true
}
```

gradle.properties

gradle.properties

```
# Project-wide Gradle settings.
# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.
# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html
# Specifies the JVM arguments used for the daemon process.
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. For more details, visit
# https://developer.android.com/r/tools/gradle-multi-project-decoupled-projects
# org.gradle.parallel=true
# AndroidX package structure to make it clearer which packages are bundled with the
# Android operating system, and which are packaged with your app's APK
# https://developer.android.com/topic/libraries/support-library/androidx-rn
android.useAndroidX=true
# Enables namespacing of each library's R class so that its R class includes only the
# resources declared in the library itself and none from the library's dependencies,
# thereby reducing the size of the R class for that library
android.nonTransitiveRClass=true
```

settings.gradle.kts

settings.gradle.kts

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.\\.android.*")
                includeGroupByRegex("com\\.\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "phiz"
include(":app")
```